

C# từ Cơ bản đến Nâng cao — Tập 1

Sách lập trình C# và .NET hiện đại cho người mới bắt đầu —
dạy OOP từ số 0 (Tập 2: ASP.NET Core, Tập 3: Web nâng cao
& design pattern)

https://github.com/phamtuanchip/cshap_book

Mục lục

Phần 0 — Chuẩn bị môi trường

1. Giới thiệu C# và .NET
2. Cài đặt .NET SDK và IDE
3. Chương trình C# đầu tiên
4. Làm quen IDE, solution và NuGet

Phần 1 — Nền tảng lập trình

5. Biến, kiểu dữ liệu, ép kiểu
6. Toán tử và biểu thức
7. Cấu trúc điều khiển: if/else, switch
8. Vòng lặp
9. Mảng
10. Phương thức
11. Chuỗi (string)

Phần 2 — Nhập môn OOP

12. Vì sao cần OOP
13. Class và Object
14. Đóng gói và Property
15. Namespace, using và tổ chức project

Phần 3 — OOP nâng cao

16. Kế thừa
17. Đa hình
18. Abstract class và Interface
19. Static, partial, extension method, enum
20. Lớp object, struct và record

Phần 4 — Xử lý lỗi và Generics

21. Xử lý ngoại lệ

- 22. Generics
- 23. Nullable value type và nullable reference type

Phần 5 — Collection và LINQ

- 24. Tổng quan collection
- 25. List và LinkedList
- 26. Dictionary, HashSet và họ hàng
- 27. IEnumerable, yield và sắp xếp
- 28. Thuật toán tìm kiếm
- 29. Duyệt đồ thị/cây: BFS và DFS
- 30. LINQ

Phần 6 — C# hiện đại và bất đồng bộ

- 31. Delegate, event và lambda
- 32. Record, pattern matching và cú pháp mới
- 33. Lập trình bất đồng bộ async/await
- 34. Đa luồng và đồng bộ hoá

Phần 7 — Nhập/Xuất và xử lý dữ liệu

- 35. File I/O
- 36. JSON và CSV
- 37. Reflection, Attribute và Serialization

Phần 8 — Công cụ, kiểm thử, quản lý dự án

- 38. Unit test với xUnit
- 39. Debug và logging
- 40. Git, NuGet và cấu hình

Phần 9 — Kiến trúc cơ bản và dự án tổng hợp

- 41. Design pattern cơ bản
- 42. Nguyên lý SOLID
- 43. Dự án tổng hợp Tập 1

Phụ lục

- 44. Phụ lục A — Lỗi thường gặp
- 45. Phụ lục B — Tài liệu tham khảo
- 46. Phụ lục C — Cheat sheet C#

Chương 1 — Giới thiệu C# và .NET

Mục tiêu học

Sau chương này, bạn sẽ:

- Phân biệt được **C#** (ngôn ngữ) và **.NET** (nền tảng chạy và thư viện).
- Hiểu SDK, runtime, CLR, BCL là gì và chúng đóng vai trò gì khi bạn viết và chạy chương trình.
- Biết vì sao sách này dùng **.NET hiện đại** (đa nền tảng) và chọn bản LTS.
- Hình dung được lộ trình cả bộ sách: từ console đến web với [ASP.NET Core](#).

C# là gì, .NET là gì?

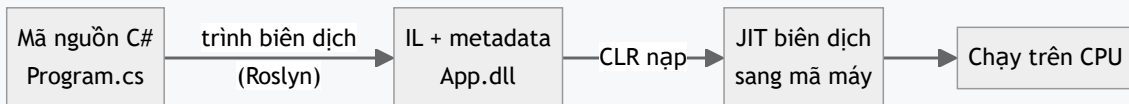
- **C#** (đọc "xi-sharp") là một **ngôn ngữ lập trình** hướng đối tượng, kiểu tĩnh, do Microsoft thiết kế, hiện là chuẩn mở. Bạn viết code C# trong các file `.cs`.
- **.NET** là **nền tảng** để biên dịch, chạy chương trình và cung cấp sẵn kho thư viện khổng lồ (đọc file, gọi mạng, làm web, truy cập CSDL...). Ngoài C#, .NET còn chạy F# và Visual Basic, nhưng C# là ngôn ngữ phổ biến nhất.

Hình dung đơn giản: C# là *tiếng bạn nói*, .NET là *môi trường và bộ công cụ* giúp lời bạn nói được máy tính thực thi.

Những thuật ngữ bạn sẽ gặp mãi

Thuật ngữ	Ý nghĩa
SDK (Software Development Kit)	Bộ công cụ cho người viết code: trình biên dịch, lệnh dotnet, template project. Cài SDK là đủ để vừa viết vừa chạy.
Runtime	Phần tối thiểu để chạy chương trình đã biên dịch (máy người dùng chỉ cần runtime).
CLR (Common Language Runtime)	"Động cơ" bên trong runtime: nạp code, quản lý bộ nhớ (garbage collector), biên dịch JIT, kiểm tra an toàn.
BCL (Base Class Library)	Thư viện chuẩn có sẵn: <code>System.Console</code> , <code>System.IO</code> , <code>System.Collections</code> , <code>System.Linq</code> ...
NuGet	Kho thư viện bên thứ ba (như npm của JavaScript). Sẽ dùng ở Chương 4.

Từ file `.cs` đến chương trình chạy được



1. Trình biên dịch **Roslyn** dịch C# thành **IL** (Intermediate Language) lưu trong file `.dll`. IL không phụ thuộc hệ điều hành hay CPU.
2. Khi chạy, **CLR** nạp file `.dll`, và bộ **JIT** (Just-In-Time) dịch IL sang mã máy của đúng máy đang chạy.

Nhờ vậy cùng một chương trình `.dll` chạy được trên Windows, macOS và Linux — miễn là máy có .NET runtime tương ứng.

.NET hiện đại: một nền tảng duy nhất

Trước đây có nhiều "nhánh" .NET tách rời. Từ **.NET 5** trở đi, Microsoft hợp nhất thành một nền tảng duy nhất, gọi đơn giản là **.NET**, mã nguồn mở, đa nền tảng, phát hành mỗi năm một bản lớn vào tháng 11:

- Năm **chẵn**: bản **LTS** (Long-Term Support) — hỗ trợ dài hạn (3 năm), phù hợp học tập và sản xuất. Ví dụ .NET 8, **.NET 10**.
- Năm **lẻ**: bản STS — hỗ trợ ngắn hơn.

Sách này dùng .NET 10 (LTS) cho toàn bộ code mẫu, và chỉ dạy công nghệ hiện đại. Kiến thức C# bạn học vẫn đúng với các bản sau; tính năng chỉ có ở phiên bản mới sẽ được ghi chú rõ.

Ghi chú phiên bản: mỗi bản .NET đi kèm một phiên bản C# (.NET 10 ↔ C# 14). Bạn không cần nhớ số hiệu — dự án mặc định dùng phiên bản C# phù hợp với `TargetFramework`.

C# dùng để làm gì?

- **Web & API** với **ASP.NET Core** — trọng tâm Tập 2 và Tập 3 của bộ sách.
- **Dịch vụ nền, công cụ dòng lệnh** (console app, worker service).
- **Ứng dụng đám mây** (Azure, container), **game** (Unity), **AI/ML** với các thư viện .NET...

Lộ trình bộ sách

Tập	Nội dung
Tập 1 (sách này)	Ngôn ngữ C#, OOP, collection, LINQ, async, I/O, test — viết được ứng dụng console hoàn chỉnh

Tập	Nội dung
Tập 2	ASP.NET Core: Web API, MVC, Razor Pages, Blazor, Entity Framework Core
Tập 3	Web nâng cao: kiến trúc phân lớp, design pattern, bảo mật, deploy

Lỗi thường gặp

- **Nhầm C# với .NET:** khi tìm tài liệu, "C# tutorial" nói về ngôn ngữ, "[ASP.NET](#) Core tutorial" nói về framework web. Hai thứ bổ trợ nhau, không thay thế nhau.
- **Nhầm SDK với Runtime:** lỗi "You must install .NET to run this application" nghĩa là máy thiếu runtime; lỗi "No .NET SDKs were found" nghĩa là thiếu SDK — nếu bạn muốn viết code, hãy cài SDK.
- **Đọc tài liệu cũ:** nhiều bài viết cũ nói về .NET Framework với cách làm khác. Ưu tiên tài liệu từ learn.microsoft.com và kiểm tra phiên bản.

Bài tập

1. Tự giải thích bằng lời của bạn: SDK khác runtime ở chỗ nào?
2. Vẽ lại sơ đồ từ file `.cs` đến lúc chương trình chạy, không nhìn sách.
3. Vào trang dotnet.microsoft.com/download, tìm phiên bản LTS mới nhất và ngày hết hỗ trợ của nó.

Chương 2 — Cài đặt .NET SDK và IDE

Mục tiêu học

Sau chương này, bạn sẽ:

- Cài được **.NET 10 SDK** và kiểm tra cài đặt thành công bằng `dotnet --version`.
- Chọn và cài một trình soạn thảo/IDE phù hợp: Visual Studio, VS Code hoặc JetBrains Rider.
- Biết vài lệnh `dotnet` nền tảng để dùng suốt cuốn sách.

Bước 1 — Cài .NET SDK

Tải SDK (không phải chỉ Runtime) tại dotnet.microsoft.com/download — chọn **.NET 10 (LTS)**.

- **Windows:** chạy file cài đặt `.exe`, hoặc dùng winget:

```
winget install Microsoft.DotNet.SDK.10
```

- **macOS:** dùng file `.pkg`, hoặc `brew install --cask dotnet-sdk`.
- **Linux:** cài theo hướng dẫn của bản phân phối (Ubuntu: `sudo apt install dotnet-sdk-10.0`, tên gói có thể khác nhau tùy phiên bản hệ điều hành).

Mở terminal **mới** (để nhận biến môi trường `PATH`) và kiểm tra:

```
dotnet --version
```

Kết quả sẽ là một số bắt đầu bằng `10.` (ví dụ `10.0.100`). Xem đầy đủ những gì đã cài:

```
dotnet --info  
dotnet --list-sdks
```

Nếu báo `No .NET SDKs were found` hoặc không nhận lệnh `dotnet`: đóng hẳn terminal rồi mở lại; nếu vẫn lỗi, cài lại SDK và kiểm tra `PATH` có chứa thư mục cài `dotnet`.

Bước 2 — Chọn công cụ soạn thảo

Bạn có thể viết C# bằng bất kỳ editor nào, nhưng IDE giúp gợi ý code, gỡ lỗi, refactor. Ba lựa chọn phổ biến:

Công cụ	Nền tảng	Đặc điểm
Visual Studio Community	Windows	IDE đầy đủ nhất, miễn phí cho cá nhân/học tập. Khi cài chọn workload " ASP.NET and web development " (cần cho Tập 2).
VS Code + C# Dev Kit	Windows / macOS / Linux	Nhẹ, đa nền tảng. Cài extension C# Dev Kit của Microsoft.
JetBrains Rider	Windows / macOS / Linux	IDE mạnh, miễn phí cho mục đích phi thương mại.

Gợi ý cho người mới: Visual Studio nếu bạn dùng Windows; VS Code nếu dùng macOS/Linux hoặc muốn nhẹ. Sách này luôn kèm **lệnh dotnet dòng lệnh** tương đương, nên dùng công cụ nào cũng theo được.

Bước 3 — Vài lệnh dotnet cần biết

Lệnh	Tác dụng
<code>dotnet new list</code>	Liệt kê template project có sẵn
<code>dotnet new console -n TenProject</code>	Tạo project console mới
<code>dotnet run</code>	Biên dịch và chạy project trong thư mục hiện tại
<code>dotnet build</code>	Chỉ biên dịch
<code>dotnet test</code>	Chạy unit test
<code>dotnet add package <tên></code>	Thêm thư viện NuGet

Ta sẽ dùng chúng ngay ở chương sau.

Thử nhanh cho chắc

```
dotnet new console -n Thu
cd Thu
dotnet run
```

Nếu thấy `Hello, World!` in ra, môi trường của bạn đã sẵn sàng.

Lỗi thường gặp

- **Lệnh dotnet không nhận:** chưa mở terminal mới sau khi cài, hoặc `PATH` thiếu thư mục `dotnet`.

- **Cài nhiều SDK, chạy nhằm phiên bản:** kiểm tra bằng `dotnet --list-sdks` . Có thể ghim phiên bản cho một thư mục bằng file `global.json` (tạo bằng `dotnet new globaljson --sdk-version 10.0.100`).
- **VS Code không gợi ý code C#:** chưa cài **C# Dev Kit**, hoặc bạn mở *một file lẻ* thay vì mở *thư mục project*.
- **Visual Studio thiếu template web:** chạy lại Visual Studio Installer, thêm workload [ASP.NET](#).

Bài tập

1. Chạy `dotnet --info` , ghi lại phiên bản SDK và hệ điều hành mà nó báo.
2. Chạy `dotnet new list` , đếm xem có bao nhiêu template và tìm ra template `webapi` , `mvc` , `classlib` .
3. Tạo một project console tên `Thu2` và chạy nó bằng IDE bạn đã chọn (nút Run/Start).

Chương 3 — Chương trình C# đầu tiên

Mục tiêu học

Sau chương này, bạn sẽ:

- Tạo, biên dịch và chạy một chương trình C# chỉ bằng dòng lệnh `dotnet`, không cần IDE.
- Đọc hiểu file `Program.cs` dạng **top-level statements** và biết dạng `Main` truyền thống tương ứng.
- Đọc hiểu file dự án `.csproj` và các thư mục sinh ra (`bin/` , `obj/`).
- Nhận đối số dòng lệnh qua `args`.

Code mẫu đầy đủ: [code/ch03-hello-world/](https://github.com/lorenzobianchi/csharp-hello-world/blob/main/code/ch03-hello-world/).

Tạo project

```
dotnet new console -n HelloWorld
cd HelloWorld
```

Lệnh này sinh ra thư mục `HelloWorld` gồm:

```
HelloWorld/
├── HelloWorld.csproj    # file mô tả dự án
└── Program.cs          # mã nguồn
```

Chương trình đầu tiên

Mở `Program.cs`, nội dung chỉ gồm một dòng:

```
Console.WriteLine("Xin chào, C#!");
```

Chạy:

```
dotnet run
```

Kết quả:

```
Xin chào, C#!
```

Chỉ một dòng — vậy `class`, `Main`, `using System`; đâu rồi? Câu trả lời ở phần tiếp theo.

Top-level statements và dạng Main truyền thống

Từ C# 9, chương trình nhỏ có thể viết **top-level statements**: lệnh đặt thẳng trong file, trình biên dịch tự sinh ra class và phương thức `Main` cho bạn. Đây là dạng dùng trong sách để ví dụ ngắn gọn.

Dạng tương đương viết đầy đủ (cũng hợp lệ, và bạn sẽ gặp nhiều trong code cũ):

```
namespace HelloWorld;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Xin chào, C#!");
    }
}
```

Giải phẫu:

- `class Program` — mọi phương thức trong C# đều thuộc một class (học kỹ ở Chương 13).
- `static void Main(string[] args)` — **điểm vào** (entry point): nơi chương trình bắt đầu chạy. `static` nghĩa là gọi được mà không cần tạo đối tượng; `void` nghĩa là không trả về giá trị; `args` là mảng đối số dòng lệnh.
- `Console.WriteLine(...)` — gọi phương thức `WriteLine` của class `Console` (thuộc namespace `System`) để in một dòng ra màn hình.

Dòng `using System`; không còn cần viết: mẫu project bật **ImplicitUsings**, tự thêm sẵn các `using` phổ biến (`System`, `System.IO`, `System.Linq`, `System.Collections.Generic` ...).

Mỗi project chỉ được có **một** điểm vào. Top-level statements chỉ được dùng trong **một** file.

File .csproj — hồ sơ của dự án

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
```

```
<Nullable>enable</Nullable>
</PropertyGroup>

</Project>
```

Thuộc tính	Ý nghĩa
Sdk="Microsoft.NET.Sdk"	Dùng bộ build tiêu chuẩn của .NET
OutputType	Exe = chương trình chạy được; thư viện dùng Library (mặc định)
TargetFramework	Phiên bản .NET nhắm tới: net10.0
ImplicitUsings	Tự thêm các using phổ biến (đã nói ở trên)
Nullable	Bật kiểm tra tham chiếu null của trình biên dịch (Chương 23)

Không cần liệt kê từng file `.cs` — SDK tự lấy mọi file `.cs` trong thư mục.

dotnet run làm gì?

`dotnet run` gồm hai bước: **build** rồi **chạy**. Khi build, các thư mục mới xuất hiện:

- `obj/` — file trung gian của quá trình build.
- `bin/Debug/net10.0/` — kết quả build: `HelloWorld.dll` (IL) và file `HelloWorld / HelloWorld.exe` (bộ khởi chạy). Có thể chạy trực tiếp file này mà không cần lệnh `dotnet run`.

Hai thư mục này sinh ra tự động và **không đưa vào Git** (file `.gitignore` của repo đã loại trừ). Xóa chúng đi bất kỳ lúc nào cũng an toàn — build lại sẽ tạo lại.

Để phát hành bản dùng thật: `dotnet publish -c Release`.

Đối số dòng lệnh

Trong top-level statements, biến `args` có sẵn (kiểu `string[]`):

```
string ten = args.Length > 0 ? args[0] : "hoc vien";

Console.WriteLine($"Xin chao, {ten}!");
Console.WriteLine($"Day la doi so dong lenh nhan duoc: {args.Length} doi so.");
```

- `args.Length` là số đối số. Biểu thức `điều_kiện ? a : b` chọn `a` nếu đúng, `b` nếu sai.
- `$"...{ten}..."` là **chuỗi nội suy**: chèn giá trị biến vào chuỗi (chi tiết ở Chương 11).

Chạy, lưu ý dấu `--` tách tham số của `dotnet run` khỏi đối số của chương trình:

```
dotnet run -- Tuan
```

Xin chào, Tuan!

Đây là đối số dòng lệnh nhận được: 1 đối số.

Lỗi thường gặp

- **Couldn't find a project to run**: bạn đang đứng sai thư mục. Chạy `dotnet run` trong thư mục có file `.csproj`, hoặc dùng `dotnet run --project HelloWorld`.
- **Thiếu dấu ; cuối lệnh** → lỗi biên dịch `CS1002: ; expected`. Mỗi câu lệnh C# kết thúc bằng `;`.
- **Sai hoa/thường**: `console.writeline` là sai; đúng là `Console.WriteLine`. C# **phân biệt** hoa/thường.
- **Hai điểm vào** (`CS0017: Program has more than one entry point`): dùng cả top-level statements ở nhiều file, hoặc vừa có `Main` vừa có top-level statements.
- **Quên -- khi truyền đối số**, khiến `dotnet run` hiểu nhầm đối số là tùy chọn của chính nó.

Bài tập

1. Sửa chương trình in ra tên và tuổi của bạn trên hai dòng.
2. Viết lại `Greeter` bằng dạng `class Program + Main` truyền thống, kết quả phải giống hệt.
3. Cố tình đổi `WriteLine` thành `Writeline` và đọc thông báo lỗi. Trình biên dịch nói gì, chỉ vào dòng nào?
4. Sửa `Greeter` để ghép **mọi** đối số thành một câu chào (gợi ý: `string.Join(" ", args)`).

Chương 4 — Làm quen IDE, solution và NuGet

Mục tiêu học

Sau chương này, bạn sẽ:

- Tạo được **solution** chứa nhiều project (một ứng dụng + một thư viện) bằng cả IDE lẫn `dotnet CLI`.
- Chạy và **gỡ lỗi** (debug) cơ bản: breakpoint, step over/into, xem biến.
- Thêm thư viện từ **NuGet** và hiểu nó được ghi vào đâu trong `.csproj`.

Code mẫu: [code/ch04-lam-quen-ide/](https://github.com/lorenzobianchi/code/ch04-lam-quen-ide/).

Solution và project

- **Project** (`.csproj`): một đơn vị biên dịch, cho ra một `.dll` hoặc `.exe`.
- **Solution** (`.sln` hoặc `.slnx`): "hộp" gom nhiều project lại để build/mở cùng nhau.

Dự án thực tế hiếm khi chỉ có một project. Ta sẽ dựng solution gồm ứng dụng `App` dùng thư viện `MayTinh`:

```
dotnet new sln -n LamQuenIde
dotnet new console -n App
dotnet new classlib -n MayTinh
dotnet sln add App MayTinh
dotnet add App reference MayTinh
```

- `classlib` tạo **thư viện lớp** (không chạy độc lập, để project khác dùng).
- `dotnet add App reference MayTinh` khai báo `App` phụ thuộc `MayTinh` (thêm dòng `ProjectReference` vào `App.csproj`).

Trong Visual Studio: **File** → **New** → **Project**, làm tương tự qua giao diện. Trong VS Code: mở thư mục solution, dùng **Solution Explorer** của C# Dev Kit.

Thư viện `MayTinh/PhepTinh.cs`:

```
namespace MayTinh;

public static class PhepTinh
{
    public static int Cong(int a, int b) => a + b;
}
```

App/Program.cs :

```
using MayTinh;

Console.WriteLine($"7 + 5 = {PhepTinh.Cong(7, 5)}");
```

Từ khoá `public` cho phép project khác truy cập class — nếu thiếu, `App` sẽ báo không thấy `PhepTinh` (học kỹ ở Chương 14). Chạy: `dotnet run --project App`.

Chạy và debug trong IDE

Debug là kỹ năng quan trọng nhất sau khi viết được code. Quy trình cơ bản:

1. **Đặt breakpoint:** bấm vào lề trái cạnh số dòng (hoặc phím `F9`). Chương trình sẽ dừng *trước khi* chạy dòng đó.
2. **Chạy chế độ debug:** `F5` (Visual Studio / VS Code), không phải `Ctrl+F5` (chạy không debug).
3. Khi dừng ở breakpoint, dùng:

Thao tác	Phím (VS)	Ý nghĩa
Step Over	F10	Chạy hết dòng hiện tại, không đi vào phương thức được gọi
Step Into	F11	Đi vào bên trong phương thức đang gọi
Step Out	Shift+F11	Chạy nốt phương thức hiện tại, quay về nơi gọi
Continue	F5	Chạy tiếp đến breakpoint kế

4. **Xem giá trị biến:** rê chuột lên biến, hoặc mở cửa sổ **Locals / Watch**. Thử đặt breakpoint ở dòng gọi `PhepTinh.Cong`, nhấn `F11` để đi vào phương thức và xem `a`, `b`.

Debug chi tiết hơn (điều kiện breakpoint, logging) sẽ học ở Chương 39.

NuGet: dùng thư viện của người khác

NuGet là kho thư viện .NET (tại nuget.org). Ví dụ thêm thư viện **Humanizer** (chuyển đổi thời gian/số thành văn bản dễ đọc) vào `App` :

```
dotnet add App package Humanizer.Core
```

Lệnh này (1) tải thư viện về bộ nhớ đệm cục bộ, (2) ghi một dòng vào `App.csproj` :

```
<ItemGroup>
  <PackageReference Include="Humanizer.Core" Version="..." />
</ItemGroup>
```

```
</ItemGroup>
```

Dùng thử:

```
using Humanizer;
using MayTinh;

Console.WriteLine($"7 + 5 = {PhepTinh.Cong(7, 5)}");
Console.WriteLine($"Khoang thoi gian: {TimeSpan.FromDays(3).Humanize()}");
```

```
7 + 5 = 12
Khoang thoi gian: 3 days
```

Các lệnh liên quan: `dotnet list package` (xem đã dùng gì), `dotnet remove package <tên>` (gỡ), `dotnet restore` (tải lại các gói theo `.csproj` — `dotnet build / run` tự gọi khi cần). Trong IDE có giao diện **Manage NuGet Packages**.

Quy tắc chọn thư viện: xem số lượt tải, ngày cập nhật gần nhất, giấy phép và mức độ được cộng đồng tin dùng trước khi thêm vào dự án. Mỗi gói thêm vào là một phụ thuộc bạn phải gánh lâu dài.

Cấu trúc thư mục nên có

```
LamQuenIde/
├─ LamQuenIde.sln
├─ App/           (console, tham chiếu MayTinh)
└─ MayTinh/       (class library)
```

Đặt code dùng chung vào thư viện, code khởi chạy vào ứng dụng — kiểu tách này là nền của kiến trúc phân lớp ở Tập 3.

Lỗi thường gặp

- `The type or namespace name 'MayTinh' could not be found`: quên `dotnet add App reference MayTinh`, hoặc quên `using MayTinh;`.
- `'PhepTinh' is inaccessible due to its protection level`: class/phương thức thiếu `public`.
- **Breakpoint có hình tròn rỗng, không dừng**: đang chạy bản Release, hoặc file nguồn không khớp bản build. Chạy lại ở cấu hình **Debug**.

- **Restore package thất bại:** kiểm tra mạng, hoặc chạy `dotnet nuget locals all --clear` rồi thử lại.
- **Lỗi `dotnet run` ở thư mục solution có nhiều project:** chỉ định `--project <tên>`.

Bài tập

1. Thêm phương thức `Nhan(int a, int b)` vào `PhepTinh`, gọi từ `App`, và debug bước vào nó bằng `F11`.
2. Dùng Humanizer để in `1234567` dưới dạng chữ (tìm trong tài liệu phương thức `ToWords()`).
3. Thêm project thứ ba `dotnet new console -n App2` vào solution, cho nó cũng tham chiếu `MayTinh`.
4. Mở `App.csproj` bằng editor và chỉ ra `ProjectReference` và `PackageReference`.

Chương 5 — Biến, kiểu dữ liệu, ép kiểu

Mục tiêu học

Sau chương này, bạn sẽ:

- Khai báo biến, hằng số và hiểu **kiểu dữ liệu** dùng để làm gì.
- Chọn đúng kiểu số (`int` , `long` , `double` , `decimal`), `bool` , `char` , `string` .
- Dùng `var` đúng lúc, ép kiểu ngầm định/tường minh, chuyển chuỗi thành số an toàn.
- Hiểu **tràn số** và **sai số số thực** — hai lỗi âm thầm rất hay gặp.

Code mẫu đầy đủ: [code/ch05-bien-kieu-du-lieu/](#) .

Biến là gì?

Biến là một ô nhớ có tên, dùng để lưu một giá trị. C# là ngôn ngữ **kiểu tĩnh**: mỗi biến có một kiểu cố định ngay lúc biên dịch, và trình biên dịch sẽ báo lỗi nếu bạn dùng sai kiểu.

```
int tuoi = 25;           // kiểu tên_biến = giá_trị;
string ten = "Tuan";
tuoi = 26;               // gán lại giá trị mới: OK
// tuoi = "hai mươi";   // LỖI biên dịch: không gán string cho int
```

Quy ước đặt tên: dùng chữ, số, `_` ; không bắt đầu bằng số; phân biệt hoa/thường. Biến cục bộ dùng `camelCase` (`soLuong`); tên kiểu, phương thức dùng `PascalCase` (`TinhTong`).

Các kiểu dữ liệu cơ bản

Kiểu	Kích thước	Dùng cho	Ví dụ
<code>int</code>	32 bit	số nguyên thông thường ($\approx \pm 2,1$ tỷ)	<code>int tuoi = 25;</code>
<code>long</code>	64 bit	số nguyên rất lớn	<code>long danSo = 8_000_000_000L;</code>
<code>byte</code>	8 bit	0..255 (dữ liệu nhị phân)	<code>byte b = 200;</code>
<code>double</code>	64 bit	số thực (khoa học, đo lường)	<code>double h = 1.75;</code>
<code>float</code>	32 bit	số thực ít chính xác hơn	<code>float f = 1.5f;</code>
<code>decimal</code>	128 bit	số thực chính xác thập phân — tiền tệ	<code>decimal gia = 19.99m;</code>

Kiểu	Kích thước	Dùng cho	Ví dụ
bool	1 bit logic	true / false	bool ok = true;
char	16 bit	một ký tự Unicode	char c = 'A';
string	—	chuỗi ký tự	string s = "chao";

Hệ thống quyết định kiểu của hằng số: `L` cho `long`, `f` cho `float`, `m` cho `decimal`. Dấu `_` chỉ để dễ đọc (`8_000_000_000`). Mỗi kiểu số có `MinValue` / `MaxValue`: `int.MaxValue` là 2147483647.

Chọn kiểu nào? Mặc định `int` cho số nguyên, `double` cho số thực, `decimal` cho **tiền**, `string` cho văn bản, `bool` cho điều kiện.

Các kiểu trên (trừ `string`) là **kiểu giá trị** (value type): biến chứa trực tiếp giá trị. `string` và mảng là **kiểu tham chiếu**: biến chứa địa chỉ trỏ tới dữ liệu. Sự khác biệt này quan trọng, sẽ quay lại ở Chương 9 và 20.

var — để trình biên dịch suy ra kiểu

```
var soLuong = 3;           // int
var thanhTien = soLuong * 19.99m; // decimal
var ten = "Tuan";         // string
```

`var` **không** phải là kiểu động: kiểu vẫn cố định lúc biên dịch, chỉ là bạn không phải viết ra. Dùng `var` khi kiểu đã rõ ràng từ về phải (`var list = new List<int>();`); nếu đọc code mà không thấy ngay kiểu là gì, hãy viết kiểu tường minh.

Hằng số

```
const double Pi = 3.14159; // giá trị cố định, phải gán ngay lúc khai báo
// Pi = 3;                 // LỖI: không đổi được hằng
```

Ép kiểu

Ngầm định (an toàn, không mất dữ liệu — tự động):

```
int a = 10;
long b = a;           // int -> long: OK
```

```
double c = a;          // int -> double: OK
```

Tường minh (có thể mất dữ liệu — phải viết (kiểu)):

```
double d = 9.99;
int n = (int)d;      // 9 – cắt bỏ phần thập phân, KHÔNG làm tròn
int m = (int)Math.Round(d); // 10 – muốn làm tròn thì dùng Math.Round
```

Chuyển chuỗi thành số:

```
int n1 = int.Parse("123");          // ném FormatException nếu chuỗi sai
if (int.TryParse("abc", out int n2)) { } // trả false thay vì ném lỗi
```

Với dữ liệu người dùng nhập, **luôn dùng** `TryParse`, vì người dùng có thể gõ bất kỳ thứ gì.

Hai cái bẫy âm thầm

1. Tràn số (overflow)

```
int lon = int.MaxValue;
Console.WriteLine(lon + 1); // -2147483648 – không báo lỗi!
```

Mặc định C# **không** báo lỗi khi tràn số mà "quay vòng". Muốn phát hiện, bọc trong `checked`:

```
checked { int x = lon + 1; } // ném OverflowException
```

2. Sai số số thực

```
Console.WriteLine(0.1 + 0.2 == 0.3); // False !
Console.WriteLine(0.1m + 0.2m == 0.3m); // True
```

`double` / `float` lưu số dạng nhị phân nên không biểu diễn chính xác `0.1`. Với **tiền tệ** hoặc mọi thứ cần đúng thập phân, dùng `decimal`. Với `double`, không so sánh bằng `==` mà so sánh trong sai số: `Math.Abs(a - b) < 1e-9`.

Lỗi thường gặp

- CS0266: Cannot implicitly convert type 'double' to 'int' — cần ép kiểu tường minh (và tự hỏi có chấp nhận mất phần thập phân không).
- CS0165: Use of unassigned local variable — dùng biến chưa gán giá trị.
- `FormatException` khi `int.Parse("12a")` — dùng `TryParse`.

- Chia số nguyên: `1 / 2` bằng `0` , không phải `0.5` (Chương 6).
- Dùng `double` cho tiền rồi ra kết quả lẻ `0.30000000000000004` .

Bài tập

1. Khai báo biến lưu tên, tuổi, chiều cao, đã kết hôn hay chưa — chọn kiểu phù hợp và in ra.
2. `int.MaxValue + 1` cho kết quả gì? Bọc `checked` để bắt lỗi.
3. Viết chương trình đọc một số từ bàn phím bằng `Console.ReadLine()` và `int.TryParse` ; báo lỗi thân thiện nếu nhập sai.
4. Tính `100 * 1.1` bằng `double` và `decimal` , so sánh kết quả với `110` .

Chương 6 — Toán tử và biểu thức

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng thành thạo toán tử số học, so sánh, logic, gán, tăng/giảm.
- Hiểu **độ ưu tiên** và **ngắn mạch** (short-circuit).
- Dùng các toán tử tiện lợi: `?:`, `??`, `??=`, `?.` và toán tử bit ở mức cơ bản.
- Tránh bẫy **chia nguyên** và `i++` vs `++i`.

Code mẫu: [code/ch06-toan-tu-bieu-thuc/](#).

Biểu thức là thứ có giá trị (`2 + 3`, `tuoi >= 18`); **toán tử** là ký hiệu tạo biểu thức từ các toán hạng.

Toán tử số học

Toán tử	Ý nghĩa	Ví dụ (a=17, b=5)
+ - *	cộng, trừ, nhân	<code>a + b</code> → 22
/	chia	<code>a / b</code> → 3 (chia nguyên)
%	chia lấy dư	<code>a % b</code> → 2

Bẫy chia nguyên: nếu cả hai toán hạng là số nguyên, kết quả là số nguyên (bỏ phần thập phân). Muốn chia thực, ép ít nhất một vế: `a / (double)b` → 3.4. Chia số nguyên cho `0` ném `DivideByZeroException`; chia số thực cho `0` cho `Infinity`.

% rất hữu ích: `n % 2 == 0` kiểm tra số chẵn; `n % 10` lấy chữ số cuối.

Tăng, giảm và gán kết hợp

```
int i = 5;
int x = i++; // x = 5, rồi i = 6 (hậu tố: dùng giá trị cũ trước)
int y = ++i; // i = 7 rồi y = 7 (tiền tố: tăng trước rồi dùng)
```

Nên tránh nhồi `i++` vào biểu thức phức tạp — khó đọc. Gán kết hợp: `+=`, `-=`, `*=`, `/=`, `%=`.

So sánh và logic

`==` `!=` `<` `>` `<=` `>=` cho kết quả `bool`. Toán tử logic:

Toán tử	Ý nghĩa
&&	và (AND)
	hoặc (OR)
!	phủ định (NOT)

Ngắn mạch: && dừng ngay nếu vế trái sai; || dừng ngay nếu vế trái đúng. Nhờ vậy viết được:

```
bool ok = mang.Length > 0 && mang[0] > 0;    // không lỗi dù mảng rỗng
```

Đối thứ tự hai vế sẽ gây lỗi khi mảng rỗng. Nhớ rằng = là **gán**, == là **so sánh**.

Độ ưu tiên

Từ cao đến thấp (rút gọn): ! ++ -- → * / % → + - → < > <= >= → == != → && → || → ?: → gán. Ví dụ `2 + 3 * 4 = 14`. **Khi nghi ngờ, dùng ngoặc ()** — code rõ hơn là nhớ bảng ưu tiên.

Toán tử điều kiện và null

```
string kq = tuoi >= 18 ? "Người lớn" : "Trẻ em";    // ?: chọn một trong hai

string? ten = null;
string hienThi = ten ?? "(không tên)";    // ?? dùng vế phải nếu vế trái là null
ten ??= "Mặc định";                      // ??= gán nếu đang là null
int? len = ten?.Length;                  // ?. trả null thay vì ném lỗi nếu ten là null
```

?. và ?? là công cụ chính chống `NullReferenceException` — sẽ học sâu ở Chương 23.

Toán tử bit (giới thiệu)

Làm việc trên từng bit của số nguyên: & (và), | (hoặc), ^ (xor), ~ (đảo), <<, >> (dịch bit).

```
6 & 3    // 0b110 & 0b011 = 0b010 = 2
1 << 4    // 16 (nhân 2 bốn lần)
```

Thường gặp ở cờ (flags), quyền hạn, xử lý nhị phân (Chương 19 với `enum [Flags]`).

Lỗi thường gặp

- `if (x = 5)` thay vì `x == 5` — C# chặn được ở hầu hết trường hợp (lỗi biên dịch), nhưng với `bool` thì `if (ok = true)` vẫn hợp lệ và gán luôn: hãy viết `if (ok)` .
- `1 / 2` ra 0; `7 / 2 * 2.0` ra 6, không phải 7 . Chuyển sang thực **trước** khi chia.
- Quên ngoặc: `a + b / 2` khác `(a + b) / 2` .
- So sánh số thực bằng `==` (Chương 5).
- Quên rằng `%` với số âm cho dư âm: `-7 % 3` bằng `-1` .

Bài tập

1. Đọc một số, in ra "chẵn" hay "lẻ" (dùng `%` và `?:`).
2. Tính điểm trung bình 3 môn (nhập từ bàn phím) với 2 chữ số thập phân — chú ý chia nguyên.
3. Cho `int a = 5; int b = a++ + ++a;` — dự đoán `a` và `b` rồi chạy kiểm tra.
4. Viết biểu thức kiểm tra năm nhuận: chia hết cho 4 nhưng không chia hết cho 100, hoặc chia hết cho 400.

Chương 7 — Cấu trúc điều khiển: if/else và switch

Mục tiêu học

Sau chương này, bạn sẽ:

- Điều khiển luồng chương trình bằng `if / else if / else`.
- Dùng `switch` statement và **switch expression** hiện đại.
- Dùng **pattern matching** (`when`, `and`, `or`, so khớp theo kiểu, tuple) để thay chuỗi `if` dài.
- Biết khi nào chọn `if`, khi nào chọn `switch`.

Code mẫu: <code/ch07-cau-truc-dieu-khien/>.

if / else if / else

```
int diem = 78;
if (diem >= 90) Console.WriteLine("Xuất sắc");
else if (diem >= 75) Console.WriteLine("Giỏi");
else if (diem >= 50) Console.WriteLine("Trung bình");
else Console.WriteLine("Yếu");
```

Các điều kiện được xét **từ trên xuống**, nhánh đầu tiên đúng sẽ chạy rồi thoát. Vì vậy phải xếp điều kiện từ **hẹp đến rộng** (kiểm `>= 90` trước `>= 75`). Điều kiện bắt buộc là `bool` (khác C/C++ — số `1` không tự thành `true`).

Với khối lệnh nhiều dòng, dùng `{ }`. Ngay cả khi chỉ một dòng, nhiều đội vẫn bắt buộc ngoặc để tránh lỗi khi sửa sau này.

switch statement

Khi so sánh **một giá trị** với nhiều hằng số:

```
switch (thu)
{
    case 2: case 3: case 4: case 5: case 6:
        Console.WriteLine("Ngày làm việc");
        break;
    case 7:
        Console.WriteLine("Thứ bảy");
        break;
    default:
```

```
        Console.WriteLine("Khác");
        break;
    }
```

Khác C/C++: mỗi `case` **bắt buộc kết thúc** bằng `break` (hoặc `return`, `throw ...`); C# không cho "chạy tràn" sang case kế. Nhiều nhãn `case` xếp liền nhau (không có lệnh giữa chúng) là được phép.

Switch expression — cách viết hiện đại

Khi mục đích là **tính ra một giá trị**, switch expression ngắn và an toàn hơn:

```
string TenThu(int t) => t switch
{
    2 => "Thứ hai",
    3 => "Thứ ba",
    >= 4 and <= 6 => "Giữa tuần",
    7 => "Thứ bảy",
    8 => "Chủ nhật",
    _ => "Không hợp lệ",      // _ là "mọi trường hợp còn lại"
};
```

- Mỗi nhánh dạng `mẫu => giá trị`. Không cần `break`.
- Trình biên dịch cảnh báo nếu bạn **quên phủ hết** các trường hợp; nếu chạy vào giá trị không khớp nhánh nào sẽ ném `SwitchExpressionException`.

Pattern matching

Mẫu (pattern) mạnh hơn so sánh `==`:

Mẫu	Ví dụ	Ý nghĩa
Hằng	7	bằng 7
Quan hệ	<code>>= 4</code>	so sánh
Kết hợp	<code>>= 4 and <= 6, 2 or 3, not 0</code>	and / or / not
Kiểu	<code>string s</code>	là string, gán vào s
null	<code>null</code>	là null
when	<code>int n when n > 40</code>	thêm điều kiện phụ
Tuple	<code>(true, false)</code>	so khớp nhiều giá trị

```
object?[] doiTuong = [42, 7, "chao", 3.14, null];
foreach (var o in doiTuong)
{
    string mota = o switch
    {
        int n when n > 40 => $"số nguyên lớn: {n}",
        int n => $"số nguyên: {n}",
        string s => $"chuỗi dài {s.Length}",
        null => "null",
        _ => $"kiểu khác: {o.GetType().Name}",
    };
    Console.WriteLine(mota);
}
```

Pattern cũng dùng được với `is`: `if (o is string s && s.Length > 3) { ... }`, `if (x is >= 1 and <= 9)`, `if (o is not null)`. Chương 17 sẽ đi sâu hơn khi học đa hình.

Chọn `if` hay `switch`?

- `if`: điều kiện phức tạp, dùng `&&` / `||`, so sánh nhiều biến khác nhau.
- `switch`: một giá trị được phân loại thành nhiều nhánh; nhất là khi cần trả về một giá trị.
- Chuỗi `if / else if` quá dài và cùng so một biến → đổi sang `switch expression`.

Lỗi thường gặp

- CS8070: Control cannot fall out of switch — thiếu `break` ở `case`.
- Sắp xếp sai thứ tự điều kiện (`>= 50` trước `>= 90` khiến nhánh sau không bao giờ chạy).
- Switch expression thiếu nhánh `_` → cảnh báo CS8509, có thể lỗi lúc chạy.
- Dùng `=` thay `==` trong điều kiện.
- Lồng `if` quá sâu: đảo điều kiện và thoát sớm (`if (!hopLe) return;`) cho code phẳng và dễ đọc.

Bài tập

1. Nhập điểm 0–10, in xếp loại bằng `if`, rồi viết lại bằng `switch expression` với `>=` / `and`.
2. Nhập số tháng 1–12, in số ngày của tháng (bỏ qua năm nhuận) bằng `switch expression` dùng `or`.
3. Viết hàm nhận `object` và mô tả kiểu (số nguyên chẵn/lẻ, chuỗi rỗng/không rỗng, null...).
4. Trò "kéo–búa–bao": nhận hai lựa chọn, dùng `switch` trên tuple để quyết định người thắng.

Chương 8 — Vòng lặp

Mục tiêu học

Sau chương này, bạn sẽ:

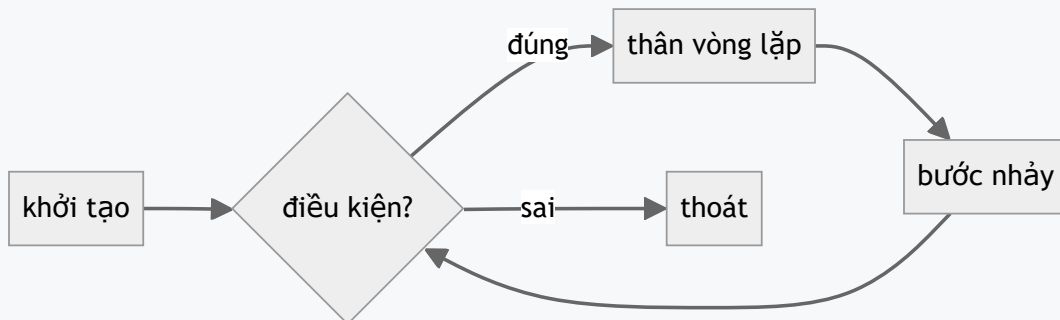
- Dùng `for`, `while`, `do-while`, `foreach` đúng chỗ.
- Điều khiển vòng lặp bằng `break`, `continue`.
- Viết vòng lặp lồng nhau và tránh **vòng lặp vô hạn**.
- Giải được các bài toán kinh điển: tổng chữ số, số nguyên tố, bảng cửu chương.

Code mẫu: [code/ch08-vong-lap/](#).

`for` — khi biết trước số lần lặp

```
for (int i = 1; i <= 5; i++)  
    Console.Write($"{i} ");  
// 1 2 3 4 5
```

Ba phần trong ngoặc: **khởi tạo** (chạy một lần) ; **điều kiện** (kiểm trước mỗi vòng) ; **bước nhảy** (chạy sau mỗi vòng). Biến `i` chỉ sống bên trong vòng lặp.



`while` — lặp khi điều kiện còn đúng

Khi không biết trước số lần lặp:

```
int n = 12345, tong = 0;  
while (n > 0)  
{  
    tong += n % 10;    // lấy chữ số cuối  
    n /= 10;          // bỏ chữ số cuối  
}
```

```
}  
Console.WriteLine(tong);    // 15
```

Điều kiện được kiểm **trước** — có thể thân vòng lặp không chạy lần nào.

do-while — chạy ít nhất một lần

```
int dem = 0;  
do  
{  
    dem++;  
} while (dem < 3);
```

Điều kiện kiểm **sau** thân vòng. Hợp với việc "nhập cho đến khi hợp lệ": luôn phải nhập ít nhất một lần.

foreach — duyệt qua tập hợp

```
string[] trai = ["tao", "cam", "xoai"];  
foreach (string t in trai)  
    Console.WriteLine(t);
```

Gọn nhất khi chỉ cần **đọc** từng phần tử. Không được gán lại biến lặp `t`, và không được thêm/xoá phần tử của tập đang duyệt (sẽ ném `InvalidOperationException` với `List<T>` — Chương 24). Cần chỉ số hoặc cần sửa mảng → dùng `for`.

break và continue

- `break` : **thoát hẳn** vòng lặp gần nhất.
- `continue` : **bỏ qua** phần còn lại của vòng hiện tại, sang vòng kế.

```
for (int i = 1; i <= 10; i++)  
{  
    if (i % 2 == 0) continue;    // bỏ số chẵn  
    if (i > 7) break;           // dừng khi vượt 7  
    Console.Write($"{i} ");      // 1 3 5 7  
}
```

Vòng lặp lồng nhau

```
for (int a = 2; a <= 4; a++)
{
    for (int b = 1; b <= 5; b++)
        Console.WriteLine($"{a}x{b}={a * b,-4}");
    Console.WriteLine();
}
```

Vòng ngoài chạy `m` lần, vòng trong `n` lần → thân chạy `m × n` lần. `break` chỉ thoát vòng **trong cùng**; muốn thoát nhiều tầng, gói vào một phương thức rồi `return`, hoặc dùng cờ `bool`.

Ví dụ điển hình — in số nguyên tố nhỏ hơn 30:

```
for (int so = 2; so < 30; so++)
{
    bool nguyênTo = true;
    for (int u = 2; u * u <= so; u++)
    {
        if (so % u == 0) { nguyênTo = false; break; }
    }
    if (nguyênTo) Console.WriteLine($"{so} ");
}
```

Điều kiện `u * u <= so` là tối ưu quen thuộc: chỉ cần thử ước đến căn bậc hai.

Chọn vòng lặp nào?

Tình huống	Dùng
Biết số lần / cần chỉ số	<code>for</code>
Duyệt toàn bộ tập hợp, chỉ đọc	<code>foreach</code>
Lặp đến khi điều kiện sai, số lần chưa biết	<code>while</code>
Phải chạy ít nhất một lần	<code>do-while</code>

Lỗi thường gặp

- **Vòng lặp vô hạn**: quên cập nhật biến (`while (n > 0)` mà không giảm `n`). Dừng bằng `Ctrl+C`.
- **Lệch một** (off-by-one): `i <= mang.Length` truy cập ngoài mảng — phải là `i < mang.Length`.
- Sửa tập hợp trong lúc `foreach` → `InvalidOperationException`.

- Khai báo lại biến trùng tên với biến ngoài vòng lặp (`CS0136`).
- Dùng `break` mong thoát cả hai vòng lồng nhau.

Bài tập

1. In tổng các số từ 1 đến 100; tổng các số chẵn trong đó.
2. Tìm số Fibonacci thứ `n` (không đệ quy).
3. In tam giác sao `*` gồm `n` dòng bằng hai vòng `for` lồng nhau.
4. Yêu cầu người dùng nhập số dương bằng `do-while` + `int.TryParse`, lặp đến khi hợp lệ.
5. Đoán số: máy chọn ngẫu nhiên 1–100 (`Random.Shared.Next(1, 101)`), người chơi đoán và nhận gợi ý cao/thấp.

Chương 9 — Mảng

Mục tiêu học

Sau chương này, bạn sẽ:

- Khai báo, khởi tạo, duyệt mảng một chiều; dùng chỉ số `^` và lát cắt `..`.
- Dùng các hàm tiện ích của `Array` (`Sort`, `Reverse`, `IndexOf` ...).
- Làm việc với mảng nhiều chiều và **jagged array**.
- Hiểu mảng là **kiểu tham chiếu** và hệ quả khi gán/sao chép.

Code mẫu: [code/ch09-mang/](#).

Mảng một chiều

Mảng lưu một dãy phần tử **cùng kiểu**, kích thước **cố định** sau khi tạo, truy cập bằng chỉ số bắt đầu từ **0**.

```
int[] diem = [8, 6, 9, 7, 10];           // collection expression (C# 12)
int[] rong = new int[3];                 // 3 phần tử, mặc định 0
string?[] ten = new string?[2];         // mặc định null
```

Giá trị mặc định: số → 0, bool → false, tham chiếu (string ...) → null.

```
Console.WriteLine(diem[0]);              // 8 – phần tử đầu
Console.WriteLine(diem[^1]);              // 10 – phần tử cuối (chỉ số ngược)
Console.WriteLine(diem.Length);          // 5
diem[1] = 7;                             // gán
```

Truy cập chỉ số ngoài phạm vi ném `IndexOutOfRangeException`.

Duyệt mảng

```
for (int i = 0; i < diem.Length; i++) Console.Write(diem[i] + " ");
foreach (int d in diem) Console.Write(d + " ");
```

Lát cắt (range)

```
int[] ba = diem[..3];                    // 3 phần tử đầu
int[] giữa = diem[1..^1];                 // bỏ phần tử đầu và cuối
```

`a..b` lấy từ chỉ số `a` đến **trước** `b`. Kết quả là **mảng mới** (bản sao).

Tiện ích của Array

```
Array.Sort(diem);           // sắp xếp tăng dần (thay đổi mảng gốc)
Array.Reverse(diem);        // đảo ngược
int vt = Array.IndexOf(diem, 9); // vị trí đầu tiên của 9, hoặc -1
Array.Copy(nguon, dich, soLuong); // sao chép
Array.Fill(rong, 7);        // điền giá trị
```

`string.Join(", ", diem)` ghép mảng thành chuỗi để in.

Mảng hai chiều (hình chữ nhật)

```
int[,] luoi = { { 1, 2, 3 }, { 4, 5, 6 } }; // 2 hàng x 3 cột
Console.WriteLine(luoi[1, 2]);              // 6
luoi.GetLength(0); // số hàng = 2
luoi.GetLength(1); // số cột = 3
```

Mọi hàng có cùng độ dài. Duyệt bằng hai vòng `for` lồng nhau.

Jagged array — mảng của các mảng

```
int[][] tamGiac = new int[4][];
for (int i = 0; i < tamGiac.Length; i++)
{
    tamGiac[i] = new int[i + 1]; // mỗi hàng dài khác nhau
    for (int j = 0; j <= i; j++) tamGiac[i][j] = j + 1;
}
```

Linh hoạt hơn khi các hàng dài ngắn khác nhau; truy cập bằng `a[i][j]`. Thường nhanh hơn mảng `[,]`.

Mảng là kiểu tham chiếu

```
int[] a = [1, 2, 3];
int[] b = a; // b và a trỏ CÙNG một mảng
b[0] = 99;
Console.WriteLine(a[0]); // 99 !
```

```
int[] c = (int[])a.Clone(); // bản sao độc lập (nông)
c[0] = 1;                  // không ảnh hưởng a
```

Gán mảng **không** sao chép dữ liệu, chỉ sao chép tham chiếu. Tương tự khi truyền mảng vào phương thức: phương thức sửa được nội dung mảng của người gọi (Chương 10). `Clone()` là sao chép **nông**: nếu phần tử là kiểu tham chiếu thì các đối tượng bên trong vẫn dùng chung.

Khi nào không dùng mảng?

Vì kích thước cố định, mảng bất tiện khi cần thêm/xoá phần tử. Khi đó dùng `List<T>` (Chương 24–25). Mảng vẫn hợp cho dữ liệu kích thước biết trước, tham số `params`, và cần hiệu năng cao.

Lỗi thường gặp

- `IndexOutOfRangeException`: dùng `<=` thay vì `<` khi duyệt, hoặc mảng rỗng.
- `NullReferenceException`: mảng chuỗi `new string[3]` chứa `null`, gọi `.Length` trên phần tử sẽ lỗi.
- Tưởng `b = a` là sao chép mảng.
- Thay đổi mảng gốc ngoài ý muốn khi `Array.Sort` (hãy `Clone()` trước nếu cần giữ bản gốc).
- Nhầm `int[,]` với `int[][]`.

Bài tập

1. Tìm số lớn nhất, nhỏ nhất và vị trí của chúng trong một mảng (không dùng LINQ).
2. Đảo ngược mảng tại chỗ bằng vòng lặp (không dùng `Array.Reverse`).
3. Cộng hai ma trận `2x3` bằng `int[,]`.
4. In tam giác Pascal 6 dòng bằng jagged array.
5. Đếm tần suất từng chữ số 0–9 trong một chuỗi số nhập vào (mảng đếm 10 phần tử).

Chương 10 — Phương thức

Mục tiêu học

Sau chương này, bạn sẽ:

- Viết phương thức có tham số và giá trị trả về; hiểu vì sao cần chia code thành phương thức.
- Dùng **overloading**, tham số tùy chọn, tham số đặt tên, `params`.
- Phân biệt truyền theo **giá trị** và theo **tham chiếu** (`ref`, `out`, `in`).
- Trả về nhiều giá trị bằng `out` và **tuple**; viết đệ quy; hiểu phạm vi (scope) biến.

Code mẫu: [code/ch10-phuong-thuc/](#).

Vì sao cần phương thức?

Phương thức (method) là một khối lệnh có tên, có thể gọi lại nhiều lần. Chia code thành phương thức giúp **tái sử dụng**, **dễ đọc** (mỗi phương thức làm một việc, tên nói lên việc đó) và **dễ kiểm thử**.

```
static int Binh(int x) => x * x;    // kiểu_trả_về Tên(tham_số) => biểu_thức;
```

Dạng đầy đủ:

```
public static int Binh(int x)
{
    return x * x;
}
```

- `int` (đứng trước tên): kiểu giá trị trả về. `void` nếu không trả về gì.
- `x` : **tham số** (parameter) trong định nghĩa; giá trị bạn truyền lúc gọi là **đối số** (argument).
- `return` trả giá trị và kết thúc phương thức.
- `=>` là cú pháp **expression-bodied** cho phương thức chỉ có một biểu thức.

Trong sách này, phương thức dùng chung đặt trong một `static class` (`Ham`): ở top-level statements, hàm cục bộ không overload được. Từ Chương 13 ta sẽ đặt phương thức trong class đúng nghĩa OOP.

Overloading — cùng tên, khác tham số

```
public static int Cong(int a, int b) => a + b;
public static double Cong(double a, double b) => a + b;
public static int Cong(int a, int b, int c) => a + b + c;
```

Trình biên dịch chọn phiên bản theo **số lượng và kiểu** đối số (không tính kiểu trả về). Hai phương thức chỉ khác nhau kiểu trả về thì không hợp lệ.

Tham số tùy chọn, tham số đặt tên, params

```
public static string ChaoHoi(string ten, string loiChao = "Xin chào", bool viet = false)
{
    return loiChao + " " + ten;
}

ChaoHoi("Tuan"); // dùng mặc định
ChaoHoi("Tuan", viet: true); // bỏ qua tham số giữa, đặt tên tham số cuối
ChaoHoi(loiChao: "Hi", ten: "An"); // đặt tên: thứ tự tùy ý
```

Tham số tùy chọn phải đứng **sau** các tham số bắt buộc. Đặt tên giúp lời gọi tự giải thích, đặc biệt với tham số `bool`.

```
public static int Tong(params int[] so) { ... }
Tong(1, 2, 3, 4); // số lượng đối số tùy ý
```

Giá trị hay tham chiếu?

Mặc định C# **truyền theo giá trị**: phương thức nhận một **bản sao**.

```
static void TangSai(int x) => x++; // chỉ tăng bản sao
int v = 10;
TangSai(v); // v vẫn là 10
```

Muốn phương thức sửa trực tiếp biến của người gọi, dùng `ref`:

```
static void TangDung(ref int x) => x++;
TangDung(ref v); // v = 11
static void HoanDoi(ref int a, ref int b) => (a, b) = (b, a);
```

Từ khóa	Người gọi phải gán trước?	Phương thức phải gán?	Mục đích
(không)	có	không	truyền bản sao
ref	có	không	đọc và ghi biến gốc
out	không	có	trả về thêm giá trị

Từ khoá	Người gọi phải gán trước?	Phương thức phải gán?	Mục đích
in	có	không (chỉ đọc)	tránh sao chép struct lớn

Với **kiểu tham chiếu** (mảng, class), giá trị được truyền là *bản sao của tham chiếu*: phương thức sửa được **nội dung** đối tượng, nhưng gán tham số sang đối tượng khác không ảnh hưởng người gọi (trừ khi `ref`).

out — mẫu TryXxx

```
public static bool ChiaCoDu(int a, int b, out int thuong, out int du)
{
    if (b == 0) { thuong = 0; du = 0; return false; }
    thuong = a / b;
    du = a % b;
    return true;
}

if (ChiaCoDu(17, 5, out int th, out int du)) Console.WriteLine($"{th} dư {du}");
```

Đây chính là mẫu của `int.TryParse`. Biến `out` có thể khai báo ngay tại chỗ gọi (`out int th`).

Tuple — trả về nhiều giá trị gọn hơn

```
public static (int Min, int Max) MinMax(int[] mang) => (mang.Min(), mang.Max());

var (min, max) = MinMax([4, 9, 1, 7]);
```

Đệ quy

Phương thức tự gọi lại chính nó, cần **điều kiện dừng**:

```
public static long GiaiThua(int n) => n <= 1 ? 1 : n * GiaiThua(n - 1);
```

Đệ quy rất hợp cho cấu trúc lồng nhau (cây, thư mục) nhưng mỗi lần gọi tốn bộ nhớ ngăn xếp: đệ quy quá sâu ném `StackOverflowException` (không bắt được, chương trình chết).

Phạm vi biến (scope)

Biến chỉ tồn tại trong khối `{ }` khai báo nó. Hàm cục bộ (local function) nhìn được biến của nơi bao quanh:

```
int dem = 0;
void Dem() => dem++;
Dem(); Dem();          // dem = 2
```

Không được khai báo biến cùng tên với biến ở khối bao ngoài (CS0136). Ưu tiên biến có phạm vi hẹp nhất có thể.

Đặt tên & thiết kế

- Tên phương thức là **động từ**, PascalCase : `TinhTong` , `LayDanhSach` .
- Mỗi phương thức làm **một việc**; nếu phải dùng chữ "và" để mô tả, hãy tách ra.
- Ít tham số ($\leq 3-4$); nhiều hơn thì gom vào một đối tượng (Chương 13).

Lỗi thường gặp

- CS0165 / CS0177 : dùng biến chưa gán, hoặc `out` chưa được gán trước khi return.
- CS0161: not all code paths return a value — có nhánh không `return` .
- Quên `ref` / `out` ở **lời gọi** (`TangDung(v)`) thay vì `TangDung(ref v)`).
- Đệ quy thiếu điều kiện dừng → `StackOverflowException` .
- Nhầm truyền theo giá trị với tham chiếu: tưởng sửa `x` trong phương thức là sửa biến gốc.

Bài tập

1. Viết `LaNguyenTo(int n)` trả `bool` , rồi in các số nguyên tố ≤ 100 bằng nó.
2. Viết `TinhTB(params double[] so)` ; thử gọi với 0 đối số và xử lý cho hợp lý.
3. Viết `TryDocSo(string s, out int kq)` giống `int.TryParse` (không dùng `TryParse`).
4. Viết `UCLN(a, b)` bằng đệ quy (thuật toán Euclid) và bằng vòng lặp.
5. Viết phương thức trả về (`tong` , `trungBinh` , `lonNhat`) của một mảng bằng tuple.

Chương 11 — Chuỗi (string)

Mục tiêu học

Sau chương này, bạn sẽ:

- Thao tác chuỗi thành thạo: cắt, tìm, thay, tách, ghép, so sánh.
- Định dạng đầu ra bằng **nội suy chuỗi** (`$"..."`) và các định dạng số/ngày.
- Hiểu `string` **bất biến** và dùng `StringBuilder` khi ghép nhiều lần.
- Dùng chuỗi nguyên văn (`@""`) và **raw string** (`"""`).

Code mẫu: [code/ch11-chuoi/](#) .

string là gì?

`string` là chuỗi các ký tự Unicode (mỗi `char` là một đơn vị UTF-16). Là kiểu **tham chiếu** nhưng hoạt động như giá trị vì nó **bất biến (immutable)**: một khi tạo ra, nội dung không đổi được. Mọi phương thức "sửa" chuỗi thực chất trả về **chuỗi mới**.

```
string a = "abc";
string b = a.ToUpper(); // "ABC" – chuỗi mới
Console.WriteLine(a);  // "abc" – a không đổi
```

Hệ quả thường gặp: viết `s.Trim()`; mà không gán kết quả là **vô tác dụng**.

Thao tác thường dùng

```
string s = " Xin chào, C# và .NET ";
s.Trim() // bỏ khoảng trắng hai đầu (TrimStart/TrimEnd)
s.Length // độ dài
s.ToUpper() / s.ToLower()
s.Contains("C#") // true
s.IndexOf("C#") // vị trí đầu tiên, -1 nếu không có
s.Replace("chào", "tạm biệt")
s.Trim().Substring(4, 4) // lấy 4 ký tự từ vị trí 4 → "chào"
s.StartsWith("Xin") / s.EndsWith("NET")
s[0] // ký tự tại vị trí 0 (kiểu char)
```

Tách và ghép

```
string[] ten = "an,binh,chi".Split(','); // ["an","binh","chi"]
string ghép = string.Join(" | ", ten); // "an | binh | chi"
```

Kiểm tra rỗng

```
string.IsNullOrEmpty(s) // null hoặc ""
string.IsNullOrWhiteSpace(s) // null, "" hoặc chỉ toàn khoảng trắng – thường dùng hc
```

Nội suy chuỗi và định dạng

```
string ten = "Tuan"; int tuoi = 25;
Console.WriteLine($"{ten} {tuoi} tuổi");
```

Trong `{ }` có thể viết biểu thức, kèm **căn lề** và **định dạng** sau dấu `,` và `:`:

Cú pháp	Ý nghĩa	Ví dụ
{x:N2}	số có phân cách nghìn, 2 số lẻ	1,234,567.89
{x:F1}	1 chữ số thập phân	1234567.9
{x:P1}	phần trăm	0.256 → 25.6% (tùy culture)
{ngay:dd/MM/yyyy}	định dạng ngày	09/03/2025
{s,6} / {s,-6}	căn phải / căn trái trong 6 ô	[ab] / [ab]

Kết quả của `N2`, `P1`, ngày tháng phụ thuộc **culture** (vùng ngôn ngữ) của máy: dấu thập phân có thể là `.` hoặc `,`. Chương trình mẫu đặt `CultureInfo.InvariantCulture` để cho kết quả giống nhau trên mọi máy. Khi ghi/đọc dữ liệu để trao đổi giữa các hệ thống (file, API), luôn dùng culture bất biến.

So sánh chuỗi

```
"abc" == "abc" // true – so sánh nội dung
string.Equals("ABC", "abc", StringComparison.OrdinalIgnoreCase) // true: bỏ qua hoa/v
```

Với `string`, `==` so sánh **nội dung** (không phải tham chiếu). Khi cần chính xác về hoa/thường và culture, chỉ rõ `StringComparison` (`Ordinal` cho so sánh kỹ thuật, `OrdinalIgnoreCase` cho không phân biệt hoa/thường). Tránh tự `ToLower()` rồi so sánh — tạo chuỗi thừa và dễ sai với một số ngôn ngữ.

StringBuilder — ghép chuỗi nhiều lần

Vì `string` bất biến, `s += x` trong vòng lặp tạo ra một chuỗi mới mỗi vòng → chậm và tốn bộ nhớ khi lặp hàng nghìn lần. Dùng `StringBuilder`, một bộ đệm có thể sửa:

```
var sb = new StringBuilder();
for (int i = 1; i <= 5; i++) sb.Append(i).Append(',');
sb.Length--; // bỏ dấu phẩy cuối
string kq = sb.ToString(); // "1,2,3,4,5"
```

Quy tắc: vài phép nối cố định → dùng `($"")` hoặc `+`; **lặp nhiều lần** → `StringBuilder`.

Chuỗi nguyên văn và raw string

```
string duongDan = @"C:\Users\Tuan\file.txt"; // @: không cần escape dấu \

string json = """
    { "ten": "Tuan", "tuoi": 25 }
    """; // raw string: nhiều dòng, dấu " tự do
```

Chuỗi thường phải escape: `"\n"` xuống dòng, `"\t"` tab, `"\\"`, `"\""`. Raw string (C# 11) rất hợp cho JSON, SQL, HTML nhưng; phần thụt lề trước dấu `"""` đóng được tự động cắt bỏ.

Duyệt ký tự

```
foreach (char c in "Lap trinh C#")
    if ("aeiouAEIOU".Contains(c)) nguyenAm++;
```

`char` có các hàm tiện ích: `char.IsDigit(c)`, `char.IsLetter(c)`, `char.ToUpper(c)`. Một số ký tự (emoji) chiếm hai `char` — khi xử lý văn bản đa ngôn ngữ chuyên sâu, cần dùng `System.Globalization.StringInfo`.

Lỗi thường gặp

- Quên gán kết quả: `s.Trim()`; (không đổi `s`).
- `ArgumentOutOfRangeException` từ `Substring` khi chỉ số/độ dài vượt quá.
- `NullReferenceException` khi gọi `.Length` trên chuỗi `null` → kiểm tra bằng `IsNullOrEmpty` hoặc `?.`
- Nối chuỗi bằng `+=` trong vòng lặp lớn.
- So sánh không phân biệt hoa/thường bằng `ToLower()` thay vì `StringComparison`.
- Số/ngày in ra sai dấu do khác culture.

Bài tập

1. Đếm số từ trong một câu (gợi ý: `Split(' ', StringSplitOptions.RemoveEmptyEntries)`).
2. Kiểm tra chuỗi có phải **palindrome** (đọc xuôi ngược giống nhau), bỏ qua hoa/thường và khoảng trắng.
3. Viết hàm viết hoa chữ cái đầu mỗi từ: `"le van tuan"` → `"Le Van Tuan"` .
4. Đếm số lần xuất hiện của mỗi ký tự trong chuỗi, in theo thứ tự chữ cái.
5. So sánh thời gian nối 100.000 số bằng `+=` và `StringBuilder` (dùng `System.Diagnostics.Stopwatch`).

Chương 12 — Vì sao cần OOP

Mục tiêu học

Sau chương này, bạn sẽ:

- Thấy được **vấn đề** mà lập trình thủ tục gặp khi chương trình lớn dần.
- Hiểu tư duy hướng đối tượng: **đối tượng = dữ liệu + hành vi**.
- Nắm tổng quan bốn trụ cột: **đóng gói, kế thừa, đa hình, trừu tượng**.
- Biết vài khái niệm nền: class, object, thành viên, thông điệp.

Code mẫu: [code/ch12-vi-sao-can-oop/](#) .

Bắt đầu bằng một vấn đề

Ta cần quản lý sinh viên. Cách tự nhiên nhất với những gì đã học:

```
string[] tenSv = ["An", "Binh"];  
double[] diemSv = [8.5, 6.0];
```

Chạy được với 2 dữ liệu. Nhưng thử để chương trình lớn lên:

- Thêm *ngày sinh, lớp, email* → thêm ba mảng, phải sửa **mọi nơi** đang dùng chúng.
- Sắp xếp theo điểm → phải hoán đổi phần tử ở **tất cả** các mảng song song; sót một mảng là dữ liệu lệch.
- Truyền một sinh viên vào phương thức → phải truyền cả đồng tham số rồi.
- Không có gì ngăn ai đó gán điểm `-5` hay `1000` .

Gốc rễ: **dữ liệu** nằm một nơi, **các hàm xử lý nó** nằm nơi khác, không có ràng buộc giữa chúng. Đây là *lập trình thủ tục*: chương trình là chuỗi hàm thao tác trên dữ liệu rời rạc.

Tư duy hướng đối tượng

Lập trình hướng đối tượng (OOP) gom **dữ liệu** và **hành vi** liên quan vào cùng một đơn vị gọi là **đối tượng**:

```
class SinhVien  
{  
    public string Ten { get; }  
    public double Diem { get; }  
    public string XepLoai() => Diem switch { >= 8 => "Gioi", >= 5 => "Trung binh", _ =  
}
```

Bây giờ *một sinh viên là một thứ*. Danh sách sinh viên là `List<SinhVien>` ; sắp xếp một danh sách duy nhất; thêm thuộc tính chỉ sửa trong class; và class tự bảo vệ dữ liệu của mình (Chương 14).

Class và Object

- **Class** (lớp) là *bản thiết kế/khuôn*: mô tả một loại đối tượng có dữ liệu gì, làm được gì.
- **Object** (đối tượng, còn gọi *instance*) là *sản phẩm cụ thể* tạo từ khuôn: sinh viên "An", sinh viên "Binh".

Khuôn bánh (class) và những chiếc bánh (object): một khuôn, nhiều bánh, mỗi bánh có nhân riêng (dữ liệu riêng) nhưng cùng hình dạng và cùng cách ăn (hành vi).

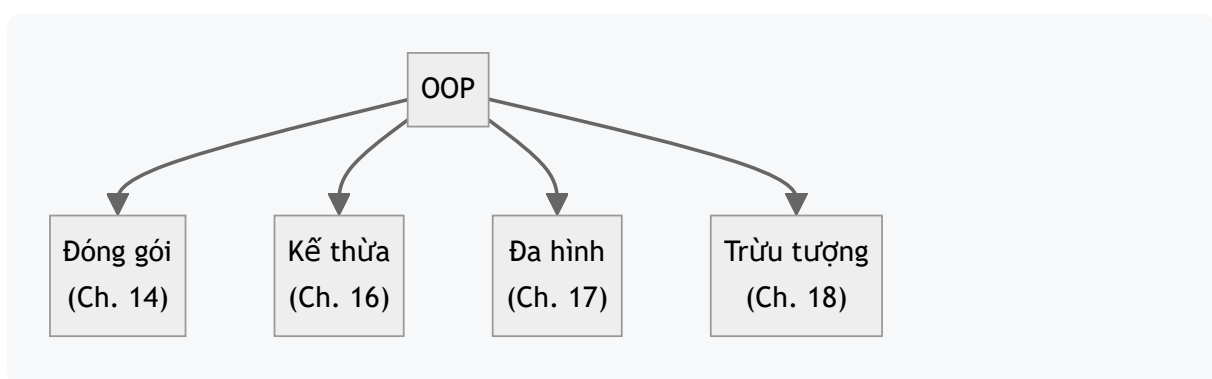
Thành viên của class

Thành viên	Vai trò	Ví dụ
Field / Property	dữ liệu (trạng thái)	Ten, Diem
Method	hành vi	XepLoai()
Constructor	khởi tạo đối tượng	new SinhVien("An", 8.5)

Chương trình OOP là tập các đối tượng **gửi thông điệp** cho nhau (gọi phương thức) để cùng hoàn thành công việc.

Bốn trụ cột của OOP

Sẽ học chi tiết ở các chương sau; đây là bức tranh tổng quan:



1. **Đóng gói (Encapsulation)** — giấu chi tiết bên trong, chỉ để lộ giao diện cần thiết; đối tượng tự bảo vệ tính hợp lệ của dữ liệu. *Ví dụ:* tài khoản ngân hàng không cho ai gán thẳng `SoDu = 1 tỷ`, chỉ cho `NapTien()`, `RutTien()` có kiểm tra.
2. **Kế thừa (Inheritance)** — class con nhận lại mọi thứ của class cha rồi bổ sung/chỉnh sửa; tái sử dụng code và mô hình quan hệ "là một" (`SinhVien` là một `Nguyen`).

3. **Đa hình (Polymorphism)** — cùng một lời gọi, đối tượng khác nhau phản ứng khác nhau.
`hinh.DienTich()` đúng cho cả hình tròn, hình chữ nhật, tam giác.
4. **Trừu tượng (Abstraction)** — tập trung vào *cái gì* thay vì *làm thế nào*: dùng `interface / abstract` để mô tả hợp đồng mà không quan tâm cài đặt.

OOP giúp ích gì? Và khi nào không?

Lợi ích: mô hình hoá thế giới thực tự nhiên; code dễ **bảo trì, tái sử dụng, mở rộng**; nhiều người cùng làm trên các class khác nhau; kiểm thử từng đối tượng riêng lẻ.

Lưu ý: OOP không phải viên đạn bạc. Thiết kế class dở (quá nhiều tầng kế thừa, class "thần thánh" ôm mọi việc) còn tệ hơn code thủ tục. C# hỗ trợ nhiều phong cách — thủ tục, hướng đối tượng, hàm (LINQ, lambda). Ta sẽ dùng chúng phối hợp: OOP để tổ chức chương trình, hàm để xử lý dữ liệu gọn gàng.

Cách nghĩ khi thiết kế class

1. Xác định **danh từ** trong bài toán → ứng viên class (sinh viên, tài khoản, đơn hàng).
2. Xác định **thuộc tính** của mỗi danh từ → field/property.
3. Xác định **động từ** → phương thức thuộc về đối tượng nào sở hữu dữ liệu tương ứng.
4. Xác định **quan hệ**: "có một" (đơn hàng có nhiều dòng hàng) hay "là một" (xe hơi là một phương tiện).

Lỗi tư duy thường gặp

- Coi class như "túi đựng hàm" — không có dữ liệu, chỉ có hàm tĩnh (đó vẫn là lập trình thủ tục).
- Public mọi thứ, để bên ngoài tùy ý sửa dữ liệu.
- Nhầm class và object ("tạo class" ≠ "tạo object").
- Lạm dụng kế thừa khi chỉ cần "có một" (composition) — Chương 16 sẽ nói kỹ.

Bài tập

1. Với bài toán *quản lý thư viện*, liệt kê ít nhất 4 class, mỗi class 3 thuộc tính và 2 hành vi.
2. Vẽ (giấy hoặc Mermaid) quan hệ giữa `DocGia`, `Sach`, `PhieuMuon`.
3. Mở code mẫu, thêm thuộc tính `Lop` cho `SinhVien`: đếm số chỗ phải sửa ở cách thủ tục và cách OOP.
4. Tìm ba vật quanh bạn (điện thoại, xe máy, cốc nước) và mô tả bằng class: dữ liệu + hành vi.

Chương 13 — Class và Object

Mục tiêu học

Sau chương này, bạn sẽ:

- Định nghĩa class với **field**, **constructor**, **phương thức** và tạo object bằng `new`.
- Hiểu `this`, thành viên `static`, và object initializer.
- Phân biệt **kiểu tham chiếu** (class) với **kiểu giá trị**; hiểu vì sao `b = a` không sao chép đối tượng.
- Hiểu `null`, và so sánh hai đối tượng.

Code mẫu: [code/ch13-class-object/](#).

Định nghĩa class

```
public class HìnhChuNhat
{
    public double Rong;      // field
    public double Cao;

    public HìnhChuNhat(double rong, double cao) // constructor
    {
        Rong = rong;
        Cao = cao;
    }

    public double DienTich() => Rong * Cao;      // phương thức
    public double ChuVi() => 2 * (Rong + Cao);
}
```

- **Field**: biến khai báo trong class, mỗi đối tượng có bản riêng.
- **Constructor**: phương thức đặc biệt — cùng tên class, không có kiểu trả về, chạy khi `new`.
- **Phương thức** thuộc class truy cập trực tiếp field của đối tượng đang gọi nó.

Quy ước: mỗi class một file `.cs` cùng tên (`HìnhChuNhat.cs`).

Tạo và dùng object

```
var h1 = new HìnhChuNhat(3, 4);
Console.WriteLine(h1.DienTich()); // 12
```

```
h1.Rong = 10; // truy cập field bằng dấu chấm
```

`new` cấp phát bộ nhớ, gọi constructor và trả về **tham chiếu** tới đối tượng. Mỗi `new` tạo một đối tượng mới, độc lập.

Nhiều constructor

```
public HìnhChuNhat() : this(1, 1) { } // gọi constructor kia, tránh lặp code
```

Nếu không viết constructor nào, C# cung cấp constructor mặc định (không tham số). **Khi bạn tự viết bất kỳ constructor nào**, constructor mặc định *không* còn được tạo tự động.

Object initializer

```
var h3 = new HìnhChuNhat { Rong = 5, Cao = 2 };
```

Gán field/property ngay sau `new` (yêu cầu class có constructor không tham số và field/property truy cập được).

Từ khoá this

`this` là tham chiếu tới **đối tượng hiện tại**. Dùng khi tên tham số trùng tên field:

```
public void DoiTên(string ten) => this.Ten = ten; // this.Ten là field, ten là tham
```

hoặc để truyền chính đối tượng đi nơi khác (`danhSach.Add(this)`), hoặc gọi constructor khác (`: this(...)`).

Thành viên static

Thành viên `static` thuộc về **class**, không thuộc đối tượng nào; gọi qua tên class.

```
public class SinhVien
{
    private static int demSo = 0; // chung cho mọi đối tượng
    public static int TongSoSinhVien => demSo;

    public int Ma;
    public SinhVien() { demSo++; Ma = demSo; }
}
```

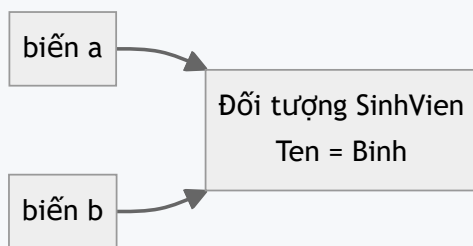
```
Console.WriteLine(SinhVien.TongSoSinhVien);
```

Phương thức `static` **không** có `this` nên không truy cập được thành viên thường. Dùng `static` cho tiện ích không phụ thuộc trạng thái (`Math.Max` , `int.Parse`) và bộ đếm dùng chung; đừng lạm dụng như biến toàn cục.

Class là kiểu tham chiếu

Đây là ý quan trọng nhất của chương:

```
var a = new SinhVien("An", 8);
var b = a; // b trỏ CÙNG đối tượng, không có bản sao nào
b.DoiTen("Binh");
Console.WriteLine(a.Ten); // "Binh"
```



Biến kiểu class chứa **địa chỉ** của đối tượng nằm trên heap. Gán `b = a` chỉ sao chép địa chỉ. Muốn bản sao thật phải tự tạo đối tượng mới (`new SinhVien(a.Ten, a.Diem)`).

Ngược lại `int` , `double` , `bool` , `struct` là **kiểu giá trị**: gán là sao chép dữ liệu.

Hệ quả khi truyền đối tượng vào phương thức: phương thức nhận bản sao của *tham chiếu* → sửa được **nội dung** đối tượng của người gọi.

null và NullReferenceException

Biến tham chiếu có thể chứa trỏ đâu cả: `null` .

```
SinhVien? khongCo = null;
Console.WriteLine(khongCo.Ten); // NullReferenceException!
Console.WriteLine(khongCo?.Ten); // an toàn: in rỗng
```

`NullReferenceException` là lỗi phổ biến nhất của người mới. Dấu `?` sau kiểu (`SinhVien?`) báo "có thể null" để trình biên dịch giúp bạn kiểm tra (Chương 23).

So sánh đối tượng

```
var d1 = new SinhVien("Dung", 7);  
var d2 = new SinhVien("Dung", 7);  
Console.WriteLine(d1 == d2);    // False: mặc định so sánh THAM CHIẾU (có phải cùng một
```

Muốn hai đối tượng "bằng nhau" khi cùng nội dung, phải tự định nghĩa (`Equals` , hoặc dùng `record` — Chương 20).

ToString()

Mọi đối tượng đều có `ToString()` . Mặc định trả về tên kiểu; override để in đẹp:

```
public override string ToString() => $"HCN {Rong} x {Cao}";  
Console.WriteLine(h1);    // WriteLine tự gọi ToString()
```

(`override` sẽ giải thích ở Chương 16.)

Lỗi thường gặp

- `NullReferenceException` : dùng đối tượng chưa `new` hoặc đang `null` .
- CS7036: There is no argument given that corresponds to the required parameter — gọi `new X()` nhưng class chỉ có constructor có tham số.
- Nhầm rằng `b = a` là sao chép.
- Gọi phương thức thường từ `static` mà không có đối tượng (CS0120).
- Quên `new` khi tạo mảng đối tượng: `new SinhVien[3]` tạo 3 ô `null` , chưa có đối tượng nào.
- So sánh hai đối tượng bằng `==` mong so sánh nội dung.

Bài tập

1. Viết class `HinhTron` (bán kính) với `DienTich()` , `ChuVi()` , và `ToString()` .
2. Viết class `PhanSo` (tử, mẫu) với `Cong(PhanSo other)` trả `PhanSo` mới (chưa cần rút gọn).
3. Thêm bộ đếm `static` vào `HinhTron` để biết đã tạo bao nhiêu hình.
4. Tạo mảng 3 `SinhVien` , sửa một phần tử qua tham chiếu thứ hai, quan sát mảng.
5. Viết phương thức `SinhVien Sao()` tạo bản sao độc lập của một sinh viên.

Chương 14 — Đóng gói và Property

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **đóng gói** và vì sao class phải tự bảo vệ dữ liệu của nó.
- Dùng đúng các **access modifier**: `public`, `private`, `protected`, `internal`.
- Viết **property** (auto-property, `get / set` có kiểm tra, `private set`, `init`, `required`).
- Thiết kế **đối tượng bất biến** và không để lộ collection nội bộ.

Code mẫu: [code/ch14-encapsulation-property/](#).

Đóng gói là gì?

Đóng gói (encapsulation) = giấu dữ liệu bên trong class và chỉ cho bên ngoài tác động qua các "cửa" do class kiểm soát. Lý do:

1. **Bảo vệ tính hợp lệ**: số dư không được âm, tuổi không được 500.
2. **Tự do thay đổi bên trong**: bên ngoài chỉ biết "cửa", nên đổi cách lưu trữ không làm vỡ code khác.
3. **Giảm phụ thuộc**: ít thứ lộ ra thì ít chỗ có thể dùng sai.

Ví dụ tệ: `public decimal SoDu;` — ai cũng gán được `tk.SoDu = 1_000_000_000`, hoặc `-5`.

Access modifier

Modifier	Ai truy cập được
<code>public</code>	mọi nơi
<code>private</code>	chỉ bên trong chính class đó (mặc định của thành viên)
<code>protected</code>	class đó và class con (Chương 16)
<code>internal</code>	mọi code trong cùng assembly (project) — mặc định của class cấp cao nhất
<code>protected internal / private protected</code>	kết hợp hai điều kiện

Nguyên tắc: để mọi thứ `private`, chỉ mở (`public`) khi thật cần. Để mở thêm về sau, rất khó thu hẹp lại.

Property — cửa vào có kiểm soát

Property trông như field khi dùng, nhưng thực chất là cặp phương thức `get / set` :

```
public class TaiKhoan
{
    private decimal _soDu;    // field ẩn (backing field)

    public decimal SoDu
    {
        get => _soDu;
        private set
        {
            if (value < 0) throw new InvalidOperationException("So du khong duoc am");
            _soDu = value;
        }
    }
}
```

- `get` trả giá trị; `set` nhận giá trị mới qua từ khoá `value` .
- `private set` : bên ngoài **đọc** được nhưng chỉ class mới **ghi** được.

Auto-property

Khi không cần logic, dùng dạng rút gọn (trình biên dịch tự sinh backing field):

```
public string ChuTaiKhoan { get; set; }
public string SoTaiKhoan { get; }      // chỉ gán được trong constructor
public bool ConTien => _soDu > 0;     // property tính toán, không lưu trữ
```

Vì sao dùng property thay vì field public ? Hôm nay `Ten` chỉ là auto-property; ngày mai cần kiểm tra hoặc ghi log thì thêm logic vào `get/set` mà **không đổi cú pháp** ở nơi gọi. Với field public không làm được. Ngoài ra nhiều framework (JSON, ORM, data binding) chỉ làm việc với property.

init và required

```
public class Sach
{
    public required string TieuDe { get; init; }    // bắt buộc gán, và chỉ gán lúc khởi tạo
    public int NamXuatBan { get; init; }
}
```

```
var s = new Sach { Tieude = "C#", NamXuatBan = 2025 };  
// s.NamXuatBan = 2026;    // LỖI: init-only
```

- `init` : gán được trong constructor hoặc object initializer, sau đó **chỉ đọc**.
- `required` (C# 11): trình biên dịch buộc phải gán khi tạo — không quên được.

Phương thức thay vì setter cho nghiệp vụ

Không phải mọi thay đổi đều nên là "gán thuộc tính". Với tài khoản, hành vi có ý nghĩa nghiệp vụ, nên cung cấp phương thức:

```
public void NapTien(decimal soTien)  
{  
    if (soTien <= 0) throw new ArgumentOutOfRangeException(nameof(soTien));  
    SoDu += soTien;  
    _lichSu.Add($"{soTien}");  
}  
  
public bool RutTien(decimal soTien)  
{  
    if (soTien <= 0 || soTien > SoDu) return false;  
    SoDu -= soTien;  
    _lichSu.Add($"-{soTien}");  
    return true;  
}
```

Hai phương thức này bảo đảm mọi thay đổi đều được kiểm tra **và** ghi lịch sử. Không có đường nào khác để thay đổi số dư → đối tượng luôn hợp lệ (đây gọi là giữ **bất biến của lớp** — class invariant).

Đừng để lộ collection nội bộ

```
public List<string> LichSu { get; } = [];           // TỆ: bên ngoài tự Add/  
private readonly List<string> _lichSu = [];  
public IReadOnlyList<string> LichSu => _lichSu;    // TỐT: chỉ đọc
```

Trả về `IReadOnlyList<T>` (hoặc `IEnumerable<T>`) để bên ngoài chỉ xem mà không phá được dữ liệu.

Đối tượng bất biến (immutable)

Đối tượng không đổi sau khi tạo: mọi thuộc tính `get / init`. Muốn "sửa" thì tạo bản mới:

```
public DiemDat DichChuyen(double dx, double dy) => new() { X = X + dx, Y = Y + dy };
```

Ưu điểm: an toàn khi chia sẻ giữa nhiều nơi/luồng, dễ suy luận, không có trạng thái "nửa vời".
Dữ liệu giá trị (toạ độ, tiền, ngày) rất hợp phong cách này; `record` (Chương 20) làm việc này gọn hơn nữa.

Quy ước đặt tên

- Property, phương thức, class: `PascalCase` (`SoDu` , `NapTien`).
- Field private: `_camelCase` (`_soDu`).
- Tham số, biến cục bộ: `camelCase` .

Lỗi thường gặp

- CS0122: 'X' is inaccessible due to its protection level — truy cập thành viên `private` .
- Stack overflow trong property tự tham chiếu: `public int X { get => X; }` (gọi lại chính nó) — phải dùng backing field.
- Để `public List<T>` cho phép bên ngoài phá dữ liệu.
- Kiểm tra hợp lệ trong `set` nhưng constructor lại gán thẳng vào field, bỏ qua kiểm tra.
- Lạm dụng getter/setter cho mọi field: class thành "túi dữ liệu", logic nằm rải rác nơi khác (giống lập trình thủ tục).

Bài tập

1. Viết class `NhanVien` có `Ten` (không rỗng) và `Luong` (không âm) với kiểm tra ném `ArgumentException` .
2. Thêm vào `TaiKhoan` phương thức `ChuyenKhoan(TaiKhoan den, decimal soTien)` , đảm bảo trừ và cộng cùng thành công hoặc cùng thất bại.
3. Viết class `NhiệtĐo` bất biến với `Celsius` và property tính toán `Fahrenheit` .
4. Cố tình thử các dòng bị comment trong `Program.cs` và đọc thông báo lỗi.
5. Giải thích vì sao `LichSu` trả `IReadOnlyList<string>` nhưng vẫn có thể bị "lách" bằng ép kiểu — và cách phòng (`AsReadOnly()`).

Chương 15 — Namespace, using và tổ chức project

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng **namespace** để nhóm class và tránh trùng tên.
- Hiểu `using` (directive, alias, `using static`, global using, implicit usings).
- Tổ chức project nhiều file/thư mục và hiểu **assembly** với `internal`.
- Đặt tên và cấu trúc thư mục theo quy ước chung của .NET.

Code mẫu: [code/ch15-namespace-using/](#).

Vấn đề: trùng tên

Dự án lớn có hàng trăm class, và thư viện bên ngoài cũng có class của chúng. Hai class cùng tên `Sach` hay `Logger` sẽ đụng nhau. **Namespace** là "họ" của class: `ThuVien.Models.Sach` khác `CuaHang.Sach`.

Khai báo namespace

Dạng **file-scoped** (C# 10) — khuyến nghị:

```
namespace ThuVien.Models;

public class Sach { ... }
```

Một dòng áp dụng cho cả file, đỡ thụt lề. Dạng khối cũ:

```
namespace ThuVien.Models
{
    public class Sach { ... }
}
```

Dấu `.` thể hiện cấp bậc: `ThuVien` → `Models` → `Sach`. Quy ước: namespace khớp với **thư mục** chứa file (`Models/Sach.cs` ↔ `namespace ThuVien.Models`), bắt đầu bằng tên dự án/công ty.

using — dùng tên ngắn

Không có `using`, phải viết tên đầy đủ:

```
var s = new ThuVien.Models.Sach { ... };
```

Có `using`, viết tên ngắn:

```
using ThuVien.Models;  
var s = new Sach { ... };
```

Các dạng khác:

```
using Console = System.Console;           // alias: đặt bí danh (dùng khi trùng tên)  
using static System.Math;                 // dùng thẳng Sqrt(x) thay vì Math.Sqrt(x)  
global using System.Text;                 // áp dụng cho MỌI file trong project
```

Implicit usings: với `<ImplicitUsings>enable</ImplicitUsings>` trong `.csproj`, SDK tự thêm `global using` cho các namespace phổ biến (`System`, `System.Collections.Generic`, `System.IO`, `System.Linq`, `System.Threading.Tasks` ...). Đó là lý do ta dùng `List<T>`, `Console` mà không cần `using`.

Khi hai namespace có class cùng tên và cả hai đều `using`, trình biên dịch báo `CS0104: ambiguous reference` — dùng alias hoặc tên đầy đủ để giải quyết.

Assembly và internal

- **Namespace** là cách nhóm về mặt *logic/tên gọi*.
- **Assembly** (`.dll` / `.exe`) là đơn vị *biên dịch/triển khai vật lý* = một project.

Một assembly có thể chứa nhiều namespace; một namespace có thể trải qua nhiều assembly. Hai khái niệm độc lập.

`internal` chỉ cho phép truy cập trong cùng **assembly**:

```
internal void XoaHet() => _sach.Clear();    // code trong project khác không thấy
```

Vì vậy khi tách một thư viện (`classlib`), chỉ đánh `public` cho phần muốn cho người khác dùng, phần còn lại để `internal` — bạn tự do thay đổi phần đó mà không lo làm vỡ người dùng thư viện.

Tổ chức project

Cấu trúc project trong code mẫu:

```
ch15-namespace-using/
├─ ch15-namespace-using.csproj
├─ Program.cs
├─ Models/
│   └─ Sach.cs          → namespace ThuVien.Models
└─ Services/
    └─ KhoSach.cs       → namespace ThuVien.Services
```

Quy ước phổ biến:

- **Một file một kiểu** (class/record/interface/enum), tên file = tên kiểu.
- Nhóm theo **vai trò**: `Models` (dữ liệu), `Services` (ng nghiệp vụ), `Repositories`, `Utils` ...
- Hoặc nhóm theo **tính năng** (`Orders/` , `Customers/`) — thường tốt hơn ở dự án lớn.
- Lớp phụ thuộc theo một chiều: `Services` dùng `Models` ; `Models` **không** dùng `Services` .

Khi dự án lớn dần, tách thành nhiều project trong một solution (như Chương 4): ví dụ `ThuVien.Core` (model, nghiệp vụ) và `ThuVien.App` (console). Ranh giới giữa các project cưỡng chế sự tách bạch mà namespace đơn thuần không làm được.

Quy ước đặt tên tổng hợp

Thành phần	Quy ước	Ví dụ
Namespace, class, method, property	PascalCase	<code>ThuVien.Models</code> , <code>NapTien</code>
Interface	I + PascalCase	<code>IKhoSach</code>
Field private	<code>_camelCase</code>	<code>_sach</code>
Tham số, biến cục bộ	<code>camelCase</code>	<code>soTien</code>
Hằng	PascalCase	<code>MaxRetry</code>

Lỗi thường gặp

- CS0246: The type or namespace name 'X' could not be found — thiếu `using` , thiếu tham chiếu project/package, hoặc gõ sai tên.
- CS0104: 'X' is an ambiguous reference — hai `using` cùng có kiểu `X` .
- Quên `public` : class mặc định là `internal` , project khác không thấy được.
- Namespace không khớp thư mục → khó tìm file; IDE cũng cảnh báo.
- Vòng phụ thuộc giữa các namespace (A dùng B, B dùng A) — dấu hiệu thiết kế cần xem lại; giữa các project thì không thể xảy ra (lỗi biên dịch).

Bài tập

1. Thêm class `NgnoiMuon` vào namespace `ThuVien.Models` và service `MuonSach` trong `ThuVien.Services` .
2. Thêm `global using ThuVien.Models;` vào một file `GlobalUsings.cs` rồi bỏ `using` khỏi các file khác.
3. Tách `Models` và `Services` thành hai project (`classlib`) và một `console` , thiết lập tham chiếu. Điều gì xảy ra với `internal` nay?
4. Tạo hai class cùng tên `Logger` ở hai namespace, dùng cả hai trong một file bằng alias.
5. Dùng `using static System.Math;` viết hàm tính khoảng cách hai điểm mà không gõ chữ `Math.` .

Chương 16 — Kế thừa

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng kế thừa (:) để tái sử dụng code và mô hình quan hệ "**là một**" (is-a).
- Gọi constructor lớp cha bằng `base`, hiểu **thứ tự khởi tạo**.
- Phân biệt `virtual` / `override`, `new` (che giấu), `sealed`, `protected`.
- Biết khi nào nên dùng **composition** ("có một") thay vì kế thừa.

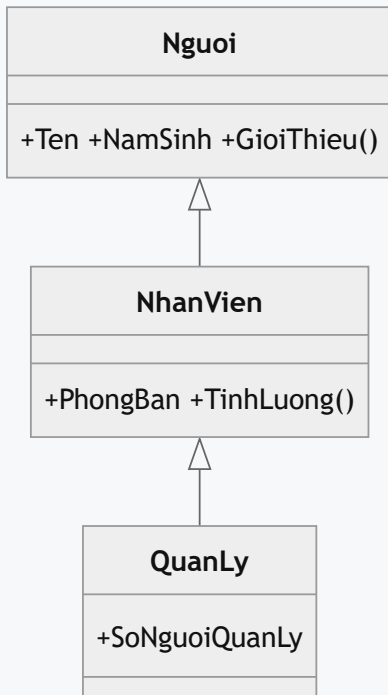
Code mẫu: [code/ch16-ke-thua/](#).

Kế thừa là gì?

Lớp con **nhận lại** mọi thành viên (không `private`) của lớp cha rồi thêm hoặc điều chỉnh. Cú pháp `class Con : Cha`.

```
class Ngươi
{
    public string Ten { get; }
    public int NamSinh { get; }
    public Ngươi(string ten, int namSinh) { Ten = ten; NamSinh = namSinh; }
    public virtual string GioiThieu() => $"Toi la {Ten}, sinh nam {NamSinh}";
}

class NhanVien : Ngươi
{
    public string PhongBan { get; }
    public NhanVien(string ten, int namSinh, string phongBan) : base(ten, namSinh)
        => PhongBan = phongBan;
}
```



NhanVien có sẵn `Ten`, `NamSinh`, `GioiThieu()` mà không viết lại. Mỗi lớp chỉ **một lớp cha trực tiếp** (C# không đa kế thừa lớp), nhưng chuỗi cha-con-cháu tùy ý. Mọi class cuối cùng đều kế thừa `object`.

Kiểm tra bằng câu "là một": "Nhân viên *là một* người" ✓ → kế thừa hợp lý. "Xe hơi *là một* động cơ" ✗ → sai.

base và thứ tự constructor

Constructor của lớp con **bắt buộc gọi** constructor lớp cha (trước tiên) để cha khởi tạo phần của nó:

```
public NhanVien(string ten, int namSinh, string phongBan) : base(ten, namSinh)
```

Thứ tự thực thi khi `new QuanLy(...)`: `Nguoi` → `NhanVien` → `QuanLy` (cha trước, con sau). Chạy code mẫu để thấy dòng in theo đúng thứ tự đó. Nếu lớp cha chỉ có constructor có tham số mà con không viết `: base(...)` → CS7036.

`base.TenPhuongThuc()` gọi phiên bản của cha từ trong con:

```
public override string GioiThieu() => $"{base.GioiThieu()}, phong {PhongBan}";
```

virtual và override

Để lớp con **thay đổi hành vi** của phương thức cha:

- Cha đánh dấu `virtual` (cho phép ghi đè).
- Con dùng `override` (ghi đè).

```
public virtual decimal TinhLuong() => LuongCoBan; // NhanVi
public override decimal TinhLuong() => LuongCoBan + SoNguoiQuanLy * 2_000_000m; // C
```

Phương thức không `virtual` thì không ghi đè được. Property cũng có thể `virtual/override`. `abstract` (Chương 18) là dạng `virtual` bắt buộc con phải ghi đè.

Override khác overload: *overload* = cùng tên khác tham số trong một class (Chương 10);
override = con viết lại phương thức cùng chữ ký của cha.

new — che giấu, không phải ghi đè

```
class Cha { public virtual string Ma() => "Cha.Ma"; public string Ten() => "Cha.Ten";
class Con : Cha
{
    public override string Ma() => "Con.Ma";
    public new string Ten() => "Con.Ten";
}

Cha cha = new Con();
cha.Ma(); // "Con.Ma" - override: quyết định theo đối tượng THẬT
cha.Ten(); // "Cha.Ten" - new: quyết định theo KIỂU KHAI BÁO
```

`new` tạo ra một phương thức mới *che* phương thức cha; gọi qua biến kiểu cha vẫn ra bản cha. Đây hầu như luôn là bẫy — nếu muốn thay đổi hành vi, hãy dùng `virtual/override`. Nếu quên `new`, trình biên dịch cảnh báo `CS0108`.

protected và sealed

- `protected`: thành viên lớp con truy cập được nhưng bên ngoài thì không. Ví dụ `LuongCoBan` trong `NhanVien`. Đừng lạm dụng — mọi `protected` là một phần giao diện với lớp con, khó đổi về sau.
- `sealed class X` cấm kế thừa `X`; `sealed override` cấm con của con ghi đè tiếp. Nên `sealed` các class không được thiết kế để kế thừa (giúp an toàn và tối ưu).

Composition thay vì kế thừa

Kế thừa gắn chặt con với cha: đổi cha là ảnh hưởng mọi con; cây kế thừa sâu rất khó hiểu. Lời khuyên nổi tiếng: **"ưu tiên composition hơn kế thừa"** — quan hệ **"có một"**:

```
class XeHoi
{
    private readonly DongCo _dongCo = new();
    public void Chay() { _dongCo.Khoi(); Console.WriteLine("Xe đang chạy"); }
}
```

`XeHoi` có một `DongCo` và giao việc cho nó; đổi loại động cơ không ảnh hưởng cây kế thừa của xe. Dùng kế thừa khi thật sự là quan hệ "là một" và muốn đa hình; còn để tái sử dụng code thì composition (và interface, Chương 18) thường linh hoạt hơn.

Lỗi thường gặp

- `CS7036` — quên gọi `: base(...)` khi lớp cha không có constructor mặc định.
- Quên `virtual` ở cha rồi thắc mắc vì sao `override` báo lỗi `CS0506`.
- Dùng `new` (che giấu) mà tưởng đã `override`: kết quả khác nhau tùy kiểu khai báo.
- Cây kế thừa quá sâu (> 3 tầng) hoặc kế thừa chỉ để lấy vài hàm tiện ích.
- Kế thừa sai quan hệ (Hình vuông : HìnhChuNhat có vấn đề khi lớp con cho phép sửa riêng chiều rộng — vi phạm nguyên lý Liskov, Chương 42).

Bài tập

1. Tạo `PhuongTien` → `XeMay`, `OTo`; mỗi loại có `TinhPhiDangKy()` khác nhau (`virtual/override`).
2. Trong code mẫu, thêm `class GiamDoc : QuanLy`. Đoán thứ tự dòng in khi tạo `GiamDoc` rồi chạy kiểm tra.
3. Đổi `Ma()` thành không `virtual` (bỏ `override`, dùng `new`), quan sát kết quả `cha.Ma()`.
4. Đánh dấu `QuanLy` là `sealed` rồi thử tạo lớp kế thừa nó — đọc thông báo lỗi.
5. Viết lại ví dụ `XeHoi` bằng kế thừa sai (`XeHoi : DongCo`) và nêu vì sao thiết kế đó tệ.

Chương 17 — Đa hình

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **đa hình (polymorphism)**: một lời gọi, nhiều hành vi tùy đối tượng thật.
- Dùng **upcasting/downcasting**, `is`, `as` và pattern matching để làm việc với kiểu con.
- Hiểu cơ chế **liên kết động** (dynamic dispatch) ở mức khái niệm.
- Viết code "mở" cho mở rộng: thêm kiểu mới không phải sửa code cũ.

Code mẫu: [code/ch17-da-hinh/](#).

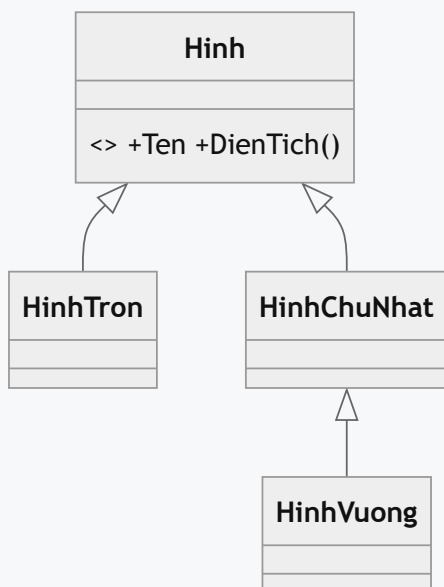
Đa hình là gì?

Đa hình = "nhiều hình dạng". Khi đã có kế thừa, ta có thể dùng biến **kiểu cha** để trỏ tới đối tượng **kiểu con**, và phương thức `virtual` sẽ chạy phiên bản **của đối tượng thật**:

```
Hinh[] cacHinh = [new HinhTron(2), new HinhChuNhat(3, 4), new HinhVuong(5)];

foreach (Hinh h in cacHinh)
    Console.WriteLine($"{h.Ten}: {h.DienTich():F2}");
```

Vòng lặp chỉ biết `Hinh`, không biết hình cụ thể là gì, nhưng `h.DienTich()` vẫn tính đúng theo từng loại.



Vì sao đây là sức mạnh lớn?

Giả sử thêm `HìnhTamGiac` : chỉ cần viết class mới kế thừa `Hình` . Vòng lặp, hàm `Sum` , mọi nơi nhận `Hình` **không phải sửa dòng nào**. Nếu dùng chuỗi `if (h is HìnhTron) ... else if ...` ở khắp nơi, mỗi lần thêm hình phải sửa hết mọi chỗ. Đây là nguyên lý **mở để mở rộng, đóng để sửa đổi** (Open/Closed, Chương 42).

Upcasting và downcasting

```
Hình hình = new HìnhVuong(2);    // upcasting: con -> cha. Luôn an toàn, ngầm định
```

- **Upcasting** (con → cha): tự động, an toàn (mọi hình vuông đều là hình).
- **Downcasting** (cha → con): phải tường minh và **có thể thất bại** (một `Hình` chưa chắc là `HìnhTron`).

```
HìnhTron t = (HìnhTron)hình;    // InvalidCastException nếu sai!
```

Ba cách kiểm tra an toàn:

```
// 1. as: trả về null nếu không ép được
HìnhTron? tron = hình as HìnhTron;
if (tron != null) { ... }

// 2. is + biến mẫu: kiểm tra và ép trong một bước (nên dùng)
if (hình is HìnhChuNhat hcn)
    Console.WriteLine($"{hcn.Rong} x {hcn.Cao}");

// 3. switch pattern
string mota = h switch
{
    HìnhVuong v => $"vuong canh {v.Rong}",
    HìnhChuNhat r => $"chu nhat {r.Rong}x{r.Cao}",
    HìnhTron t => $"tron ban kinh {t.BanKinh}",
    _ => "khac",
};
```

Thứ tự nhánh quan trọng: `HìnhVuong` kế thừa `HìnhChuNhat` , nên phải đặt nhánh `HìnhVuong` **trước**; nếu không, nhánh `HìnhChuNhat` sẽ "nuốt" luôn hình vuông (trình biên dịch thường báo lỗi vì nhánh sau không bao giờ đạt tới).

`hình.GetType()` cho kiểu **thật** lúc chạy; kiểu **khai báo** là kiểu của biến (`Hình`).

Liên kết động — chuyện gì xảy ra bên dưới?

Khi gọi phương thức `virtual` qua biến kiểu cha, CLR không quyết định lúc biên dịch mà **lúc chạy**, dựa vào kiểu thật của đối tượng (thông qua bảng phương thức ảo — *vtable* — mỗi kiểu giữ). Đó là **liên kết động** (dynamic dispatch / late binding). Ngược lại phương thức không `virtual` (và `new`) được quyết định theo **kiểu khai báo** (Chương 16).

Chi phí của `virtual` rất nhỏ; đừng tránh nó vì lý do hiệu năng trừ khi đã đo được vấn đề.

Đa hình không chỉ có kế thừa

Các dạng thường gặp trong C#:

1. **Đa hình lớp con** (chương này): kế thừa + `virtual/override`.
2. **Đa hình qua interface** (Chương 18): nhiều class không liên quan cùng cài đặt một interface.
3. **Đa hình tham số** (generics, Chương 22): cùng một code cho nhiều kiểu.
4. (Overloading là "đa hình lúc biên dịch" — quyết định theo kiểu khai báo.)

Cách viết code đa hình tốt

- Nhận tham số và trả về bằng kiểu **trừu tượng** (`Hinh` , `IThanhToan`) thay vì kiểu cụ thể.
- Tránh chuỗi `if (x is A) ... else if (x is B)` lặp lại ở nhiều nơi — dấu hiệu nên chuyển hành vi vào phương thức ảo của chính lớp đó.
- Dùng pattern matching khi *không sở hữu* các lớp (thư viện ngoài) hoặc hành vi thuộc về nơi xử lý, không thuộc lớp.

Lỗi thường gặp

- `InvalidCastException` do downcasting sai — dùng `is` / `as`.
- `NullReferenceException` khi dùng kết quả `as` mà quên kiểm tra `null`.
- Nhánh `switch` / `is` tổng quát đặt trước nhánh cụ thể hơn.
- Quên `virtual` ở lớp cha nên `override` không có tác dụng như mong đợi (hoặc dùng `new` nhầm).
- Gọi phương thức chỉ có ở lớp con qua biến kiểu cha mà không ép kiểu (`CS1061`).

Bài tập

1. Thêm `HinhTamGiac` (đáy, cao) vào code mẫu. Có phải sửa vòng lặp `foreach` không?
2. Tạo `DongVat` (abstract, `KeuLen()`), `Cho`, `Meo`, `Vit`; đặt vào `List<DongVat>` và gọi `KeuLen()` cho tất cả.
3. Với danh sách `object` chứa số, chuỗi, mảng, null — dùng `switch` pattern để mô tả từng phần tử.

4. Giải thích vì sao đoạn sau ném lỗi, sửa bằng `is : Hình h = new HìnhVuong(1); var t = (HìnhTron)h; .`
5. Đổi vị trí hai nhánh `HìnhVuong` và `HìnhChuNhat` trong `switch` và đọc thông báo của trình biên dịch.

Chương 18 — Abstract class và Interface

Mục tiêu học

Sau chương này, bạn sẽ:

- Viết và dùng **abstract class**: khung chung, thành viên `abstract`, **template method**.
- Viết và cài đặt **interface**: hợp đồng, nhiều interface, **default interface method**, **explicit implementation**.
- Chọn đúng giữa abstract class và interface.
- Hiểu vì sao lập trình theo interface làm code linh hoạt, dễ kiểm thử.

Code mẫu: [code/ch18-abstract-interface/](#) .

Abstract class

Abstract class là lớp **không tạo được đối tượng trực tiếp** (`new BaoCao()` là lỗi), chỉ để kế thừa. Nó có thể chứa cả phần **đã cài đặt** lẫn phần **chỉ khai báo** (`abstract`) buộc lớp con phải hoàn thành.

```
abstract class BaoCao(string tieuDe)
{
    protected string TieuDe { get; } = tieuDe;

    // Template method: khung cố định, các bước con thay đổi
    public string Xuat() => $"{DauTrang()}\n{TieuDe}\n{ChanTrang()}";

    protected abstract string DauTrang();                // BẮT BUỘC con cài đặt
    protected virtual string ChanTrang() => "-- het --";  // mặc định, con có thể g

}

class BaoCaoText(string tieuDe) : BaoCao(tieuDe)
{
    protected override string DauTrang() => "=== BAO CAO ===";
}
```

`Xuat()` là **Template Method**: quy trình định sẵn ở lớp cha, chỗ thay đổi do lớp con quyết định. Đây là một design pattern quen thuộc (Chương 41).

Dùng abstract class khi các lớp con **thực sự chia sẻ trạng thái/code** và có quan hệ "là một" rõ ràng (`Hinh`, `BaoCao`).

Interface

Interface là một **hợp đồng**: liệt kê "làm được gì" mà không nói "làm thế nào". Class nào cài đặt interface cam kết cung cấp đủ các thành viên đó.

```
interface IThanhToan
{
    void ThanhToan(decimal soTien);
    decimal TinhPhi(decimal soTien) => 0m;    // default interface method
}

class TheTinDung(string soThe) : IThanhToan
{
    public void ThanhToan(decimal soTien) => Console.WriteLine($"The {soThe}: tra {soTien}m");
    public decimal TinhPhi(decimal soTien) => soTien * 0.02m;
}
```

Quy ước đặt tên: bắt đầu bằng chữ **I** (`IThanhToan`). Thành viên interface mặc định là `public` . Interface không chứa field, không có constructor.

Nhiều interface

Một class chỉ kế thừa **một** lớp cha nhưng cài đặt **nhiều** interface:

```
class ViDienTu : IThanhToan, ILuuVet { ... }
```

Đây là cách C# đặt "đa kế thừa hành vi" một cách an toàn.

Default interface method (C# 8)

Interface có thể cung cấp cài đặt mặc định cho một thành viên (`TinhPhi` ở trên). Lớp cài đặt có thể dùng luôn hoặc ghi đè. Hữu ích để **thêm phương thức mới vào interface mà không làm vỡ** các class đã cài đặt cũ. Lưu ý phương thức mặc định chỉ gọi được **qua biến kiểu interface** (`IThanhToan c = ...; c.TinhPhi(...)`).

Explicit implementation

Khi hai interface có phương thức trùng tên nhưng nghĩa khác nhau:

```
interface IBay { string DiChuyen(); }
interface IBoi { string DiChuyen(); }

class ChimCanhCut : IBay, IBoi
{
    // ...
}
```

```

    string IBay.DiChuyen() => "Bay";
    string IBoi.DiChuyen() => "Boi";
}

IBay may = new ChimCanhCut();
may.DiChuyen();    // "Bay"

```

Thành viên khai báo tường minh **chỉ gọi được qua kiểu interface**, không qua kiểu class.

Abstract class hay Interface?

	Abstract class	Interface
Quan hệ	"là một" (cùng họ)	"có khả năng" (cùng hợp đồng)
Kế thừa/cài đặt	1 lớp cha	nhiều interface
Trạng thái (field)	có	không
Constructor	có	không
Code dùng chung	có	chỉ mặc định (hạn chế)
Dùng khi	chia sẻ code & trạng thái, có khung xử lý	định nghĩa hợp đồng, tách rời phụ thuộc

Quy tắc thực dụng: bắt đầu bằng interface. Chỉ dùng abstract class khi có phần code/trạng thái dùng chung đáng kể. Hai thứ dùng chung được: interface mô tả hợp đồng, abstract class cung cấp cài đặt gốc để tiện kế thừa.

Lập trình theo interface

```

class DonHang(IThanhToan thanhToan)
{
    public void ThanhToan(decimal soTien) => thanhToan.ThanhToan(soTien);
}

```

`DonHang` chỉ biết `IThanhToan`, không biết thẻ, ví hay tiền mặt. Hệ quả:

- **Linh hoạt:** thêm hình thức thanh toán mới không sửa `DonHang`.
- **Để kiểm thử:** khi test, truyền vào một đối tượng giả (mock) thay cho cổng thanh toán thật (Chương 38).
- **Nền của Dependency Injection** ([ASP.NET](#) Core, Tập 2): framework cấp phát đối tượng cài đặt interface cho bạn.

Đây là ý tưởng cốt lõi của kiến trúc hiện đại: *phụ thuộc vào trừu tượng, không phụ thuộc vào chi tiết* (Chương 42).

Lỗi thường gặp

- CS0534: 'X' does not implement inherited abstract member 'Y' — chưa override đủ thành viên abstract .
- CS0144: Cannot create an instance of the abstract type — cố new một abstract class hoặc interface.
- CS0737: does not implement interface member ... not public — quên public khi cài đặt (implicit implementation phải public).
- Interface quá to ("fat interface") ép class phải cài đặt những thứ không cần — nên tách nhỏ.
- Gọi default interface method qua biến kiểu class: `new TienMat().TinhPhi(1)` báo lỗi vì class không "thừa hưởng" nó.

Bài tập

1. Tạo interface `IDocDuLieu<T>` -đơn-giản: `string Doc()` , cài đặt `DocFile` và `DocMang` (trả dữ liệu cố định).
2. Viết abstract class `NhacCu` có abstract string `ChoiNhac()` và template method `BieuDien()` (chào + chơi + cảm ơn).
3. Thêm hình thức thanh toán `ChuyenKhoan` vào code mẫu mà **không sửa** vòng lặp gọi `ThanhToan` .
4. Cho interface `IHình { double DienTich(); }` — cài đặt bằng `record` và bằng `class` ; so sánh.
5. Thêm một phương thức mới vào `ILuuVet` bằng default method và xác nhận `ViDienTu` không cần sửa.

Chương 19 — Static, partial, extension method, enum

Mục tiêu học

Sau chương này, bạn sẽ:

- Viết **static class** và hiểu khi nào phù hợp.
- Viết **extension method** để "mở rộng" kiểu có sẵn (nền tảng của LINQ).
- Dùng **enum** và `[Flags]` để biểu diễn tập giá trị có tên.
- Biết `partial class` và **lớp lồng nhau** (nested class).

Code mẫu: [code/ch19-static-extension-enum/](#).

Static class

Static class chỉ chứa thành viên `static`, không tạo đối tượng, không kế thừa. Hợp cho tiện ích không có trạng thái:

```
static class ToanHoc
{
    public static int BinhPhuong(int x) => x * x;
}

Console.WriteLine(ToanHoc.BinhPhuong(9));
```

`Math`, `Console`, `File` đều là static class. **Đừng lạm dụng:** nhồi mọi thứ vào static class biến chương trình thành lập trình thủ tục, khó kiểm thử vì không thay thế được (không có interface). Nghiệp vụ nên nằm trong đối tượng; static chỉ dành cho hàm thuần túy và tiện ích.

Extension method

Cho phép "thêm" phương thức cho một kiểu **mà không sửa hay kế thừa** kiểu đó (kể cả `string`, `int`, kiểu của thư viện):

```
static class MoRong
{
    public static bool LaSoChan(this int n) => n % 2 == 0;
    public static string VietHoaChuDau(this string s) { ... }
}
```

```
Console.WriteLine(5.LaSoChan()); // false
Console.WriteLine("hello world".VietHoaChuDau()); // "Hello World"
```

Quy tắc: khai báo trong **static class** không lồng nhau; tham số đầu có từ khoá **this** chỉ ra kiểu được mở rộng; namespace chứa nó phải được **using** để "thấy" phương thức. Thực chất chỉ là đường cú pháp cho lời gọi phương thức static (`MoRong.LaSoChan(5)`).

Extension method là cách toàn bộ **LINQ** (`Where` , `Select` , `OrderBy` ... — Chương 30) được viết, và cách cấu hình `builder.Services.AddXxx()` trong [ASP.NET Core](#). Nguyên tắc: dùng khi phương thức thật sự "thuộc về" kiểu đó về mặt khái niệm và bạn không sở hữu mã nguồn; đừng tạo hàng loạt extension trên `object` / `string` cho mọi việc vặt.

Lưu ý: nếu kiểu đã có phương thức cùng chữ ký, phương thức của kiểu **luôn thắng** extension.

Enum

Enum là tập hợp các hằng số có tên, thay cho "số ma thuật":

```
enum TrangThaiDon
{
    ChoXacNhan = 1,
    DangGiao = 2,
    DaGiao = 3,
}

var tt = TrangThaiDon.DangGiao;
Console.WriteLine($"{tt} = {(int)tt}"); // DangGiao = 2
```

So với `int trangThai = 2;` thì `TrangThaiDon.DangGiao` tự mô tả, và trình biên dịch chặn gán giá trị lung tung (`switch` có thể cảnh báo thiếu nhánh).

Thao tác hữu ích:

```
Enum.Parse<TrangThaiDon>("DaGiao"); // ném lỗi nếu sai
Enum.TryParse("KhongCo", out TrangThaiDon kq); // false
foreach (var v in Enum.GetValues<TrangThaiDon>()) ... // liệt kê
```

Enum mặc định là `int` , bắt đầu từ `0` , tăng dần nếu không gán. Ép số bất kỳ sang enum (`(TrangThaiDon)99`) **không báo lỗi** — dùng `Enum.IsDefined` để kiểm tra. Gắn thêm hành vi cho enum bằng **extension method** (như `MoTa()` trong mã mẫu).

[Flags] — tổ hợp nhiều giá trị

Khi các giá trị có thể **kết hợp** (quyền hạn, tùy chọn), dùng lũy thừa của 2 và thuộc tính [Flags] :

```
[Flags]
enum Quyen { Khong = 0, Doc = 1, Ghi = 2, Xoa = 4, TatCa = Doc | Ghi | Xoa }

var q = Quyen.Doc | Quyen.Ghi;           // bật 2 cờ → "Doc, Ghi"
q.HasFlag(Quyen.Ghi);                    // true
(q & Quyen.Xoa) != 0;                     // false
q |= Quyen.Xoa;                           // bật thêm
q &= ~Quyen.Doc;                          // tắt Doc
```

Đây chính là toán tử bit đã giới thiệu ở Chương 6: `|` để bật, `&` để kiểm tra, `~` để đảo mặt nạ.

Partial class

`partial` cho phép chia **một** class thành nhiều đoạn/file, gộp lại lúc biên dịch:

```
partial class SinhVien { public string Ten { get; set; } = ""; }
partial class SinhVien { public override string ToString() => Ten; }
```

Ít dùng trong code bình thường. Nó chủ yếu để **tách code do công cụ sinh ra** khỏi code bạn viết tay (ví dụ source generator, một số tệp do IDE/framework sinh). Điều quan trọng là nhận ra `partial` khi gặp.

Lớp lồng nhau (nested class)

Class khai báo bên trong class khác. Nếu để `private`, chỉ class chứa nó thấy được — hợp cho chi tiết cài đặt:

```
class DanhSachLienKet
{
    private Nut? _dau;
    private class Nut(int giaTri)           // chỉ DanhSachLienKet dùng được
    {
        public int GiaTri { get; } = giaTri;
        public Nut? Tiep { get; set; }
    }
}
```

Dùng khi lớp nhỏ chỉ có nghĩa trong ngữ cảnh lớp cha (nút của danh sách, bộ duyệt...). Gọi từ ngoài (nếu `public`): `DanhSachLienKet.Nut`. Lớp lồng truy cập được cả thành viên `private` của lớp bao ngoài.

Lỗi thường gặp

- Extension method không xuất hiện: thiếu `using` namespace, hoặc không đặt trong static class.
- Ép số vào enum không hợp lệ mà không kiểm tra (`Enum.IsDefined`).
- Quên gán giá trị lũy thừa của 2 cho enum `[Flags]` (`Doc=1, Ghi=2, Xoa=3` sai → `3 = Doc|Ghi`).
- Trộn tiện ích tĩnh và trạng thái toàn cục (`static` field thay đổi được) → lỗi khó truy vết, nhất là đa luồng.
- Đặt `partial` cho class mà chưa cần — chỉ làm code khó tìm.

Bài tập

1. Viết extension `DemTu(this string s)` trả số từ trong chuỗi và `Chia<T>(this IEnumerable<T>, int n)` (tùy chọn).
2. Định nghĩa enum `NgayTrongTuan` và extension `LaCuoiTuan()` .
3. Tạo `[Flags] enum TuyChonIn { None = 0, DamNet = 1, Nghieng = 2, GachChan = 4 }` , in ra tổ hợp và kiểm tra từng cờ.
4. Viết `static class DinhDang` có `Tien(decimal)` và `Ngay(DateTime)` ; cân nhắc: nên là static class hay class thường có interface? Vì sao?
5. Viết thêm nested class `DuyetVien` bên trong `DanhSachLienKet` để duyệt danh sách.

Chương 20 — Lớp object, struct và record

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu `object` là gốc của mọi kiểu và các phương thức nó cung cấp (`ToString` , `Equals` , `GetHashCode`).
- Định nghĩa **bằng nhau** đúng cách: `Equals` + `GetHashCode` , `IEquatable<T>` .
- Phân biệt **struct** (kiểu giá trị) và **class** (kiểu tham chiếu).
- Dùng **record** cho dữ liệu bất biến: so sánh theo giá trị, `with` , `deconstruct`.
- Sắp xếp bằng `IComparable<T>` và `IComparer<T>` .

Code mẫu: [code/ch20-object-struct-record/](#) .

object — gốc của mọi thứ

Mọi kiểu trong C# (kể cả `int`) đều kế thừa trực tiếp/gián tiếp từ `System.Object` . Ba phương thức bạn hay ghi đè:

Phương thức	Mặc định	Ghi đề để
<code>ToString()</code>	tên kiểu	in dạng dễ đọc
<code>Equals(object?)</code>	so sánh tham chiếu (class)	so sánh theo nội dung
<code>GetHashCode()</code>	dựa trên tham chiếu	bấm theo nội dung, nhất quán với <code>Equals</code>

Ngoài ra `GetType()` cho kiểu thật lúc chạy.

Bằng nhau: tham chiếu hay giá trị?

```
var s1 = new SinhVienClass("An", 1);
var s2 = new SinhVienClass("An", 1);
s1.Equals(s2); // false: hai đối tượng khác nhau (mặc định so sánh tham chiếu)
```

Để hai đối tượng "cùng nội dung thì bằng nhau", phải tự định nghĩa:

```
class DiemTuDinhNghia(int x, int y) : IEquatable<DiemTuDinhNghia>
{
    public int X { get; } = x;
    public int Y { get; } = y;
```

```

public bool Equals(DiemTuDinhNghia? other) => other is not null && X == other.X &&
public override bool Equals(object? obj) => Equals(obj as DiemTuDinhNghia);
public override int GetHashCode() => GetHashCode.Combine(X, Y);
}

```

Quy tắc vàng: Equals và GetHashCode đi đôi

Nếu `a.Equals(b)` là `true` thì **bắt buộc** `a.GetHashCode() == b.GetHashCode()`. Vi phạm sẽ làm `HashSet` và `Dictionary` hoạt động sai (không tìm thấy phần tử vốn đã có) — Chương 26 giải thích cơ chế. Vì vậy **luôn ghi đề cả hai**. `HashCode.Combine(...)` là cách viết mã băm gọn và đúng. Chỉ dùng các trường **bất biến** để tính mã băm; nếu trường đổi giá trị sau khi đối tượng nằm trong `HashSet`, nó sẽ "lạc" mất.

Viết tay như vậy rất dài dòng — đó là lý do `record` ra đời (bên dưới).

struct — kiểu giá trị

```

struct ToaDo { public int X; public int Y; }

var p1 = new ToaDo { X = 1, Y = 2 };
var p2 = p1;      // SAO CHÉP toàn bộ dữ liệu
p2.X = 99;        // p1.X vẫn là 1

```

	class	struct
Loại	tham chiếu	giá trị
Gán / truyền	sao chép địa chỉ	sao chép dữ liệu
Có thể null	có	không (trừ <code>Nullable<T>, ?</code>)
Kế thừa	có	không (chỉ cài interface)
Bộ nhớ	heap (có GC)	thường trên stack / nằm trong đối tượng chứa nó

Dùng `struct` cho kiểu **nhỏ, bất biến, hành xử như một giá trị** (toạ độ, tiền, màu, `DateTime`, `Guid`). Kích thước nên $\lesssim 16$ byte. Dùng `readonly struct` để bảo đảm bất biến. Không dùng `struct` cho đối tượng lớn hoặc cần danh tính riêng — sao chép sẽ tốn kém và gây nhầm lẫn.

record — kiểu dữ liệu hiện đại

`record` là class (hoặc struct) với **so sánh**, `ToString`, **sao chép, deconstruct tự sinh**, dùng cho dữ liệu bất biến:

```
record NguoRecord(string Ten, int Tuoi);    // 1 dòng thay cho cả chục dòng

var a = new NguoRecord("An", 20);
var b = new NguoRecord("An", 20);
Console.WriteLine(a == b);    // True – so sánh THEO GIÁ TRỊ
Console.WriteLine(a);        // NguoRecord { Ten = An, Tuoi = 20 }

var c = a with { Tuoi = 21 };    // sao chép, đổi một phần → bản mới, a không đổi
var (ten, tuoi) = a;            // deconstruct
```

Trình biên dịch tự sinh: property `init`, `Equals` / `GetHashCode` / `==`, `ToString`, `Deconstruct`, và hỗ trợ `with`.

- `record` (= `record class`): kiểu tham chiếu, bất biến theo mặc định, hợp DTO, message, value object.
- `record struct` / `readonly record struct`: kiểu giá trị (`readonly record struct` `DiemRS(int X, int Y);`).

Trong [ASP.NET Core](#) (Tập 2) `record` là lựa chọn quen thuộc cho DTO/request/response. Khi cần đối tượng có **danh tính và trạng thái thay đổi** (entity, `TaiKhoan`), vẫn dùng `class`.

Sắp xếp: `IComparable<T>` và `IComparer<T>`

- `IComparable<T>`: đối tượng biết "so với chính mình" — **thứ tự tự nhiên** (một kiểu chỉ có một):

```
record SanPham(string Ten, decimal Gia) : IComparable<SanPham>
{
    public int CompareTo(SanPham? other) => other is null ? 1 : Gia.CompareTo(other.Gia)
}

ds.Sort();    // theo giá
```

- `IComparer<T>`: lớp so sánh **bên ngoài**, cho phép nhiều cách sắp xếp khác nhau:

```
class SoSanhTheoTen : IComparer<SanPham> { ... }
ds.Sort(new SoSanhTheoTen());
ds.Sort((x, y) => y.Gia.CompareTo(x.Gia));    // hoặc dùng lambda
```

Quy ước giá trị trả về: âm (nhỏ hơn), 0 (bằng), dương (lớn hơn). Chương 27 sẽ kết hợp với LINQ (`OrderBy`).

Lỗi thường gặp

- Ghi đè `Equals` mà quên `GetHashCode` (cảnh báo CS0659) → `HashSet` / `Dictionary` lỗi ngầm.
- Mã băm dựa trên trường **thay đổi được**.
- Dùng `==` cho `class` thường và tưởng so sánh nội dung.
- Struct lớn có thể thay đổi (`mutable struct`): sao chép ngầm gây bug rất khó thấy. Ưu tiên `readonly struct`.
- Boxing: gán struct cho biến `object` hay interface sẽ bọc vào heap — tốn hiệu năng nếu lặp nhiều.
- Nhầm `record` với "class bất biến hoàn toàn": các thuộc tính kiểu tham chiếu bên trong (`List`, class khác) vẫn có thể đổi.

Bài tập

1. Viết `record TienTe(decimal SoTien, string Loai)` và kiểm tra `==`, `with`, `ToString`.
2. Tạo `HashSet<DiemTuDinhNghia>` rồi cố tình bỏ `GetHashCode` và quan sát `Contains` — vì sao sai?
3. So sánh `struct` và `class` cho `ToaDo`: mảng 1 triệu phần tử, thử đo bộ nhớ/thời gian tạo (gợi ý `Stopwatch`, `GC.GetTotalMemory`).
4. Cài `IComparable<SinhVien>` sắp xếp theo điểm giảm dần, ưu tiên tên khi điểm bằng nhau.
5. Viết `record NguoIDung(string Ten, string Email)` rồi giải thích khi nào nên đổi thành `class`.

Chương 21 — Xử lý ngoại lệ

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **exception** là gì và vì sao dùng nó thay cho mã lỗi trả về.
- Dùng `try / catch / finally`, nhiều `catch`, **exception filter** (`when`).
- Biết `throw` và `throw;` khác nhau thế nào; tạo **custom exception**.
- Dùng `using` để giải phóng tài nguyên; kiểm tra tham số bằng `ThrowIfNull ...`
- Nắm nguyên tắc: khi nào bắt, khi nào để lỗi nổi lên.

Code mẫu: [code/ch21-exception/](#) .

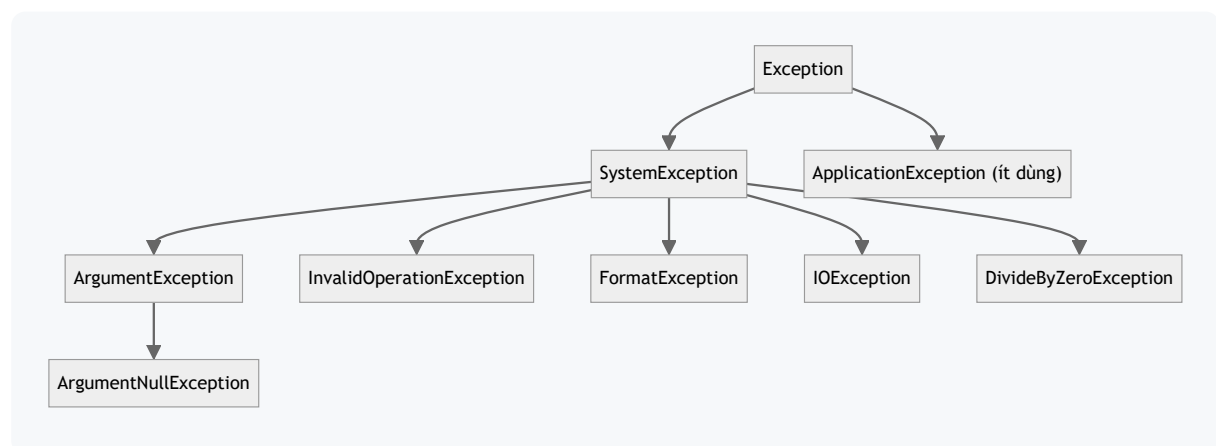
Exception là gì?

Khi có sự cố lúc chạy (chia cho 0, file không tồn tại, mạng đứt), .NET **ném (throw)** một đối tượng ngoại lệ. Nếu không ai bắt, ngoại lệ "nổi" ngược lên chuỗi lời gọi (call stack) và cuối cùng **làm chương trình sập**.

```
int so = int.Parse("abc"); // FormatException – chương trình dừng nếu không xử lý
```

Ưu điểm so với trả mã lỗi: lỗi không thể bị **lặng lẽ bỏ qua**, và code xử lý lỗi tách khỏi luồng chạy bình thường.

Mọi ngoại lệ kế thừa `System.Exception`. Các thuộc tính quan trọng: `Message`, `StackTrace`, `InnerException`.



`try / catch / finally`

```
try
{
    // code có thể ném lỗi
}
catch (FormatException e)
{
    // xử lý riêng cho FormatException
}
finally
{
    // luôn chạy: dù có lỗi hay không, kể cả khi return
}
```

- Có thể có **nhều** `catch` ; CLR chọn cái **đầu tiên khớp**, nên đặt kiểu **cụ thể trước, tổng quát sau** (`catch (Exception)` phải đứng cuối; sai thứ tự là lỗi biên dịch).
- `finally` dùng để dọn dẹp (đóng file, giải phóng kết nối). Nhưng thường `using` (bên dưới) gọn hơn.
- `try` phải đi kèm ít nhất một `catch` hoặc `finally` .

Ném ngoại lệ và custom exception

```
if (soLuong > tonKho) throw new KhongDuHangException(soLuong, tonKho);
```

Tự định nghĩa ngoại lệ khi cần **mang thêm thông tin nghiệp vụ** và cho phép bên gọi bắt chính xác:

```
class KhongDuHangException(int soCan, int soCon)
    : Exception($"Khong du hang: can {soCan}, con {soCon}")
{
    public int SoCan { get; } = soCan;
    public int SoCon { get; } = soCon;
}
```

Quy ước: tên kết thúc bằng `Exception` , kế thừa `Exception` (không phải `ApplicationException`).

Exception filter — when

```
catch (HttpLoi e) when (e.MaLoi is >= 400 and < 500)
{
```

```
Console.WriteLine($"Loi client {e.MaLoi}");
}
```

Chỉ bắt khi điều kiện đúng; nếu sai, ngoại lệ đi tiếp như chưa từng gặp `catch` này (và **giữ nguyên stack trace**). Tốt hơn việc bắt rồi `if ... else throw`.

Ném lại: `throw;` và `throw e;`

```
catch (Exception)
{
    GhiLog(...);
    throw;           // ĐÚNG: giữ nguyên stack trace gốc
    // throw e;      // SAI: reset stack trace, mất dấu nơi lỗi thật sự xảy ra
}
```

Muốn thêm ngữ cảnh, bọc ngoại lệ gốc vào `InnerException`:

```
catch (FormatException ex)
{
    throw new ApplicationException("Doc cau hinh that bai", ex); // giữ nguyên nhân
}
```

using — giải phóng tài nguyên

Đối tượng giữ tài nguyên ngoài (file, kết nối, socket) cài đặt `IDisposable`. `using` bảo đảm gọi `Dispose()` ngay cả khi có lỗi:

```
using var tn = new TaiNguyen("ket noi DB"); // Dispose() chạy khi ra khỏi phạm vi
// hoặc dạng khối:
using (var tn2 = new TaiNguyen("x")) { ... }
```

`using` chính là `try/finally` viết gọn. Với tài nguyên bất đồng bộ dùng `await using` (Chương 33). Tự viết class quản lý tài nguyên thì cài `IDisposable`.

Kiểm tra tham số đầu vào (guard clause)

```
static void ChaoHoi(string ten)
{
    ArgumentNullException.ThrowIfNull(ten);
    ArgumentException.ThrowIfNullOrWhiteSpace(ten);
    ArgumentOutOfRangeException.ThrowIfNegative(tuoi);
}
```

```
...  
}
```

Kiểm tra ở **đầu** phương thức, ném lỗi ngay khi tham số sai — lỗi lộ ra sớm, gần nguyên nhân.

Nguyên tắc dùng exception

1. **Chỉ dùng cho tình huống bất thường**, không dùng để điều khiển luồng thường: kiểm tra trước (`TryParse` , `ContainsKey`) hơn là bắt `FormatException` / `KeyNotFoundException` .
Ném/bắt exception chậm hơn nhiều.
2. **Chỉ bắt khi biết cách xử lý**. Nếu không xử lý được, để nó nổi lên tới nơi có thể (ở web: middleware xử lý lỗi toàn cục).
3. **Không bắt rồi nuốt lặng lẽ** (`catch { }`): lỗi biến mất mà không ai biết.
4. **Bắt kiểu cụ thể**, tránh `catch (Exception)` trừ ở "ranh giới" chương trình (ghi log, trả lỗi 500).
5. Thông báo lỗi nói **cái gì sai và dữ liệu liên quan**, không chỉ "Lỗi".
6. Dùng đúng kiểu có sẵn: `ArgumentException` (tham số sai), `InvalidOperationException` (trạng thái không cho phép), `NotSupportedException` , `NotSupportedException` (chưa viết)...

Lỗi thường gặp

- CS0160: A previous catch clause already catches all exceptions — `catch` tổng quát đặt trước `catch` cụ thể.
- `throw e;` làm mất stack trace.
- `catch (Exception) { }` nuốt lỗi.
- Dùng exception cho luồng bình thường (vòng lặp bắt `FormatException` thay vì `TryParse`).
- `return` trong `finally` làm nuốt ngoại lệ — tránh.
- Quên `Dispose` (rò rỉ kết nối/file) — dùng `using` .
- `StackOverflowException` / `OutOfMemoryException` không nên (và thường không thể) bắt.

Bài tập

1. Viết chương trình đọc số từ bàn phím, chia 100 cho số đó, xử lý riêng `FormatException` và `DivideByZeroException` .
2. Tạo `TaiKhoanKhongDuTienException` mang theo số dư hiện tại và số tiền cần rút; dùng trong `TaiKhoan.RutTien` (Chương 14).
3. Viết class `KetNoi` : `IDisposable` in ra khi mở/đóng; dùng `using` và cố tình ném lỗi giữa chừng — `Dispose` có chạy?
4. Thay `throw;` thành `throw e;` trong ví dụ và so sánh stack trace bằng `e.StackTrace` .

5. Viết phương thức `TryChia(int a, int b, out int kq)` trả `bool` thay vì ném lỗi; khi nào nên chọn kiểu này?

Chương 22 — Generics

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu vấn đề mà **generics** giải quyết: tái sử dụng code mà vẫn **an toàn kiểu**.
- Viết **generic class**, **generic method**, **generic interface/record**.
- Dùng **constraint** (`where`) để giới hạn kiểu.
- Hiểu ở mức khái niệm **covariance** / **contravariance** (`out` / `in`).

Code mẫu: [code/ch22-generics/](#) .

Vấn đề: một ngăn xếp cho nhiều kiểu

Muốn viết ngăn xếp (stack) lưu số nguyên, rồi lại cần ngăn xếp lưu chuỗi... Có hai cách "cũ" đều tệ:

1. Viết `NganXepInt` , `NganXepString` , ... — lặp code.
2. Dùng `object` cho mọi thứ — mất an toàn kiểu:

```
var stack = new NganXepObject();
stack.Day(1);
stack.Day("chao");           // vẫn chạy!
int x = (int)stack.Lay();     // InvalidCastException lúc chạy
```

Ngoài ra số nguyên bị **boxing** (bọc vào heap) khi chuyển thành `object` → chậm.

Generics giải quyết cả hai: viết **một lần** với tham số kiểu `T` , trình biên dịch kiểm tra kiểu **lúc biên dịch**, không boxing.

Generic class

```
class NganXep<T>
{
    private readonly List<T> _ds = [];
    public int SoLuong => _ds.Count;
    public void Day(T item) => _ds.Add(item);

    public T Lay()
    {
        if (_ds.Count == 0) throw new InvalidOperationException("Ngan xep rong");
    }
}
```

```

        T item = _ds[^1];
        _ds.RemoveAt(_ds.Count - 1);
        return item;
    }
}

var soNguyen = new NganXep<int>();
soNguyen.Day(1);
// soNguyen.Day("a");    // LỖI BIÊN DỊCH – an toàn kiểu

```

T là **tham số kiểu** (type parameter), được thay bằng kiểu cụ thể khi dùng (`NganXep<int>` , `NganXep<string>`). Quy ước tên: `T` , hoặc `TKey` , `TValue` , `TResult` . Bạn đã dùng generics từ lâu: `List<T>` , `Dictionary<TKey, TValue>` .

Generic method

Phương thức có tham số kiểu riêng; trình biên dịch thường **suy luận** được `T` :

```

public static void HoanDoi<T>(ref T a, ref T b) => (a, b) = (b, a);

int x = 1, y = 2;
Tien.HoanDoi(ref x, ref y);    // T = int (tự suy ra)
Tien.HoanDoi<string>(ref s1, ref s2);    // chỉ rõ khi cần

```

Constraint — giới hạn kiểu với where

Với `T` bất kỳ, bạn chỉ dùng được những gì `object` có. Muốn so sánh hai `T` thì phải cho biết `T` **so sánh được**:

```

public static T LonNhat<T>(params T[] ds) where T : IComparable<T>
{
    T max = ds[0];
    foreach (var x in ds)
        if (x.CompareTo(max) > 0) max = x;
    return max;
}

Tien.LonNhat(3, 9, 4);    // 9
Tien.LonNhat("cam", "tao", "xoai");    // "xoai"
// Tien.LonNhat(new object(), ...);    // LỖI: object không IComparable<object>

```

Các constraint thường gặp:

Constraint	Ý nghĩa
where T : class	T là kiểu tham chiếu
where T : struct	T là kiểu giá trị
where T : new()	có constructor không tham số (cho phép new T())
where T : BaseClass	T kế thừa BaseClass
where T : IInterface	T cài đặt interface
where T : notnull	T không nullable
Kết hợp	where T : class, ICoMa, new() (thứ tự: class/struct trước, new() cuối)

Constraint cho phép **dùng** thành viên của kiểu ràng buộc bên trong code generic (ví dụ `item.Ma` khi `T : ICoMa`).

```
class KhoTheoMa<T> where T : class, ICoMa
{
    private readonly Dictionary<string, T> _ds = [];
    public void Them(T item) => _ds[item.Ma] = item;
    public T? Tim(string ma) => _ds.GetValueOrDefault(ma);
}
```

Generic interface và record

```
interface IKetQua<out T> { bool ThanhCong { get; } T? GiaTri { get; } string? Loi { get; } }

record KetQua<T>(bool ThanhCong, T? GiaTri, string? Loi) : IKetQua<T>
{
    public static KetQua<T> ThatBai(string loi) => new(false, default, loi);
}
```

`default` cho giá trị mặc định của `T` (0 với số, `null` với tham chiếu). Kiểu **Result** như trên là mẫu quen thuộc để trả về "thành công hoặc lỗi" mà không ném exception (sẽ gặp lại ở Tập 3).

Covariance và contravariance (cơ bản)

Câu hỏi: `string` là `object`, vậy `IEnumerable<string>` có là `IEnumerable<object>` không?

- **Covariance (out T)**: giữ nguyên chiều kế thừa; an toàn khi `T` chỉ xuất hiện ở vị trí **đầu ra** (trả về).

```
IEnumerable<string> tenChuoai = ["a", "b"];
IEnumerable<object> tenObj = tenChuoai; // OK vì IEnumerable<out T>
```

- **Contravariance (in T)**: đảo chiều; an toàn khi T chỉ ở vị trí **đầu vào** (tham số).

```
Action<object> inRa = o => Console.WriteLine(o);
Action<string> inChuoai = inRa; // OK vì Action<in T>: nơi nhận string có thể dùng hà
```

Còn `List<T>` vừa đọc vừa ghi nên **không** biến thiên: `List<object> ds = new List<string>();` là lỗi — nếu cho phép, bạn sẽ `Add(5)` được vào một danh sách chuỗi. Bạn hiếm khi tự khai báo `in / out`, nhưng hiểu để đọc hiểu lỗi và API.

Khi nào nên viết generic?

- Có cấu trúc dữ liệu/thuật toán dùng lại cho nhiều kiểu (kho lưu trữ, cache, kết quả, phân trang).
- Đừng generic hoá quá sớm: nếu chỉ có một kiểu dùng, cứ viết cụ thể; khi thứ hai xuất hiện hãy tổng quát hoá.
- Nếu constraint dài dằng dặc và lộn xộn, có thể thiết kế đang phức tạp hơn cần thiết.

Lỗi thường gặp

- CS0314/CS0311 : kiểu truyền vào không thoả constraint.
- CS0304: Cannot create an instance of the variable type 'T' — thiếu `where T : new()`.
- CS0019: Operator '>' cannot be applied to operands of type 'T' — dùng `IComparable<T>` thay vì toán tử.
- Nhầm `default(T)` : với kiểu tham chiếu là `null` — cân nhắc `T? / nullable`.
- Kỳ vọng `List<Con>` gán được cho `List<Cha>` (không được, xem covariance).
- Khai báo tham số kiểu trùng tên kiểu có sẵn (`class Kho<string>` là sai — `string` không phải tên tham số).

Bài tập

1. Viết `HangDoi<T>` (queue) với `Them`, `Lay`, `SoLuong` bằng `LinkedList<T>` bên trong.
2. Viết `static T NhoNhat<T>(IEnumerable<T> ds) where T : IComparable<T>`, thử với `int`, `string`, record `SanPham`.
3. Viết `Cap<TA, TB>` (record generic hai tham số) và phương thức `DaoNguoc()` trả `Cap<TB, TA>`.
4. Viết `Kho<T> where T : class, ICoMa` hỗ trợ `Xoa(string ma)`; thử với `Sach` và với `int` — quan sát lỗi.

5. Giải thích vì sao `ReadOnlyList<T>` là covariant còn `IList<T>` thì không.

Chương 23 — Nullable value type và nullable reference type

Mục tiêu học

Sau chương này, bạn sẽ:

- Biểu diễn "không có giá trị" cho kiểu giá trị bằng `int?`, `DateTime?` ...
- Hiểu **nullable reference types**: `string` vs `string?` và cảnh báo của trình biên dịch.
- Dùng thành thạo `?.`, `??`, `??=`, `is null`, `is not null`, `!`.
- Thiết kế API biểu đạt rõ ràng khi nào giá trị có thể vắng mặt.

Code mẫu: [code/ch23-nullable/](#).

"Tỷ đô": lỗi null

`null` nghĩa là "không tham chiếu tới gì". Gọi phương thức trên `null` → `NullReferenceException`, lỗi phổ biến nhất của người viết C#. Người phát minh ra `null` gọi nó là "sai lầm tỷ đô". C# hiện đại có hai công cụ để chế ngự nó.

Nullable value type: T?

Kiểu giá trị (`int`, `bool`, `DateTime`, `struct`) mặc định **không thể** null. Thêm `?` để cho phép:

```
int? tuoi = null;           // thực chất là Nullable<int>
tuoi.HasValue;             // false
tuoi.GetValueOrDefault(-1); // -1
tuoi = 30;
tuoi.Value;                 // 30 (nếu null thì ném InvalidOperationException)
```

Ví dụ thực tế: cột CSDL có thể trống (ngày sinh chưa nhập), tham số tùy chọn, kết quả "không tìm thấy" của phép tính.

Lan truyền null: phép toán trên `int?` cho kết quả `null` nếu một vế là null:

```
int? a = 5, b = null;
var tong = a + b;    // null
```

Nullable reference type (C# 8+)

Kiểu tham chiếu ngày trước luôn có thể null mà trình biên dịch không cảnh báo. Với `<Nullable>enable</Nullable>` trong `.csproj` (mặc định trong template mới), quy ước đổi

thành:

- `string` — **không** được null (cam kết).
- `string?` — **có thể** null (phải kiểm tra trước khi dùng).

```
string ten = "An";           // OK
string ten2 = null;         // CẢNH BÁO CS8600: gán null cho kiểu không nullable
string? tuyChon = null;     // OK
int len = tuyChon.Length;   // CẢNH BÁO CS8602: dereference có thể null
```

Chỉ là **cảnh báo** lúc biên dịch (không đổi hành vi lúc chạy), nhưng nó phát hiện hàng loạt bug tiềm ẩn trước khi chạy. Thói quen tốt: **coi cảnh báo nullable là lỗi**, hoặc bật

`<TreatWarningsAsErrors>` cho nullable.

Quy tắc thiết kế:

- Property/tham số **bắt buộc** → kiểu không nullable (`string Ten`), đảm bảo gán trong constructor hoặc dùng `required`.
- Có thể vắng mặt → `string?` `GhiChu`, và người dùng buộc phải xử lý.
- Phương thức có thể "không tìm thấy" → trả `T?` (`NguoaiDung? TimNguoai(...)`).

Toán tử làm việc với null

Toán tử	Tên	Ý nghĩa
<code>x?.Y</code>	null-conditional	nếu x null trả null, ngược lại x.Y
<code>x?[i]</code>	null-conditional index	như trên cho chỉ mục
<code>x ?? d</code>	null-coalescing	x nếu khác null, ngược lại d
<code>x ??= d</code>	null-coalescing assignment	gán d cho x nếu x đang null
<code>x is null / x is not null</code>	kiểm tra bằng pattern	(nên dùng thay <code>== null</code>)
<code>x!</code>	null-forgiving	báo trình biên dịch "tin tôi, không null"

[illegible]

```
s ??= "moi gan";  
if (s is not null) Console.WriteLine(s.Length); // sau kiểm tra, trình biên dịch biế
```

?. cắt cả chuỗi phía sau: a?.B.C.D — nếu a null, toàn biểu thức là null (không đánh giá B.C.D). Kết quả của ?. luôn là kiểu nullable (int?), nên thường đi cùng ??.

Toán tử ! — dùng thận trọng

```
string? chuaChac = LayChuoi();  
Console.WriteLine(chuaChac!.Length); // tắt cảnh báo, nhưng nếu thật sự null vẫn Nul
```

! chỉ **tắt cảnh báo**, không bảo vệ gì cả. Chỉ dùng khi bạn thật sự biết hơn trình biên dịch (ví dụ giá trị chắc chắn được khởi tạo bởi framework). Nếu thấy mình rải ! khắp nơi, thiết kế đang có vấn đề.

Bảo vệ ở "ranh giới"

Với dữ liệu từ bên ngoài (người dùng, file, API), không tin vào kiểu không nullable — kiểm tra thật:

```
static void XuLy(string dauVao)  
{  
    ArgumentNullException.ThrowIfNull(dauVao);  
    ...  
}
```

Trong nội bộ code đã bật nullable, các hàm private có thể tin vào kiểu.

Thiết kế thay thế cho null

Đôi khi "trả về null" mơ hồ. Các lựa chọn:

- **Trả T?** : đơn giản, cho "có hoặc không" (TimNguoi).
- **TryXxx(out T)** : if (dict.TryGetValue(k, out var v)) — mẫu quen thuộc của .NET.
- **Trả collection rỗng** thay vì null ([], Array.Empty<T>(), Enumerable.Empty<T>()) — bên gọi khỏi kiểm tra null.
- **Giá trị mặc định có nghĩa** (Null Object pattern): một đối tượng "không làm gì".
- **Result/Option** kết quả có kiểu (Tập 3).

Đừng trả null cho collection; và đừng dùng null mang nhiều ý nghĩa ("chưa tải", "lỗi", "rỗng").

Lỗi thường gặp

- `NullReferenceException` — nguyên nhân số 1; đọc `StackTrace` để biết biến nào null.
- `InvalidOperationException: Nullable object must have a value` — `.Value` trên `int?` đang null.
- Tắt nullable toàn cục để hết cảnh báo thay vì sửa.
- Rải `!` để "im lặng" trình biên dịch.
- Gán `null!` cho property để hết cảnh báo: dùng `required` hoặc khởi tạo mặc định tốt hơn.
- Quên rằng chuỗi `?.` cho ra kiểu nullable: `int len = s?.Length;` báo lỗi, phải `?? 0`.

Bài tập

1. Viết hàm `int? TimViTri(int[] mang, int giaTri)` trả `null` nếu không có; dùng `??` để hiển thị "không thấy".
2. Cho record `NguoDung(string Ten, DiaChi? DiaChi)` — in thành phố hoặc "N/A" bằng một biểu thức duy nhất.
3. Bật `<Nullable>enable</Nullable>`, viết class có `string Ten` không khởi tạo, đọc cảnh báo và sửa bằng 3 cách (`required`, constructor, giá trị mặc định).
4. Viết phương thức `TryDocSo(string? s, out int so)` xử lý cả `null`.
5. So sánh: phương thức trả `List<string>?` với trả `List<string>` rỗng — khác gì với người gọi?

Chương 24 — Tổng quan collection

Mục tiêu học

Sau chương này, bạn sẽ:

- Biết các collection chính của .NET và **khi nào chọn cái nào**.
- Nắm khái niệm **độ phức tạp** (Big-O) ở mức đủ dùng để so sánh collection.
- Hiểu hệ thống interface: `IEnumerable<T>`, `ICollection<T>`, `IList<T>`, `IDictionary<K,V>`, `ReadOnly...`.
- Dùng `List<T>`, `Dictionary<K,V>`, `HashSet<T>`, `Queue<T>`, `Stack<T>` cơ bản.

Code mẫu: <code/ch24-tong-quan-collection/>.

Vì sao cần collection?

Mảng có kích thước cố định và ít thao tác. Chương trình thật cần cấu trúc **co giãn, tra cứu nhanh, loại trùng, xếp hàng**... Không gian tên `System.Collections.Generic` cung cấp sẵn các cấu trúc đó dưới dạng **generic** (Chương 22) — an toàn kiểu, không boxing. (Các collection không generic cũ như `ArrayList`, `Hashtable` đã lỗi thời: đừng dùng.)

Bản đồ nhanh: chọn collection nào?

Cần	Dùng	Ghi chú
Danh sách có thứ tự, truy cập theo chỉ số	<code>List<T></code>	lựa chọn mặc định
Tra cứu giá trị theo khoá	<code>Dictionary<K,V></code>	$O(1)$ trung bình
Tập hợp không trùng, kiểm tra "có chưa?"	<code>HashSet<T></code>	$O(1)$ trung bình
Hàng đợi vào trước ra trước (FIFO)	<code>Queue<T></code>	Enqueue/Dequeue
Ngăn xếp vào sau ra trước (LIFO)	<code>Stack<T></code>	Push/Pop
Tự động sắp xếp theo khoá	<code>SortedDictionary</code> , <code>SortedSet</code> , <code>SortedList</code>	$O(\log n)$
Chèn/xoá giữa nhiều, đã có node	<code>LinkedList<T></code>	hiếm dùng
Chỉ đọc, không cho ai sửa	<code>ReadOnlyList<T></code> , <code>ImmutableList<T></code>	Chương 14

Cần	Dùng	Ghi chú
Đa luồng	ConcurrentDictionary, ConcurrentQueue...	Chương 34

Quy tắc thực dụng: dùng `List<T>` nếu không có lý do khác; cần "tìm theo khoá" → `Dictionary` ; cần "có/không" trên tập lớn → `HashSet` .

Độ phức tạp thời gian (Big-O) — đủ dùng

Big-O mô tả thời gian chạy tăng thế nào khi số phần tử `n` tăng:

Ký hiệu	Tên	<code>n = 1.000.000</code> cỡ	Ví dụ
$O(1)$	hằng số	1 bước	<code>list[i]</code> , <code>dict[key]</code>
$O(\log n)$	logarit	~20 bước	tìm nhị phân, <code>SortedSet</code>
$O(n)$	tuyến tính	1 triệu bước	duyệt <code>List</code> , <code>list.Contains</code>
$O(n \log n)$		~20 triệu	sắp xếp
$O(n^2)$	bình phương	1 nghìn tỷ	hai vòng lặp lồng nhau

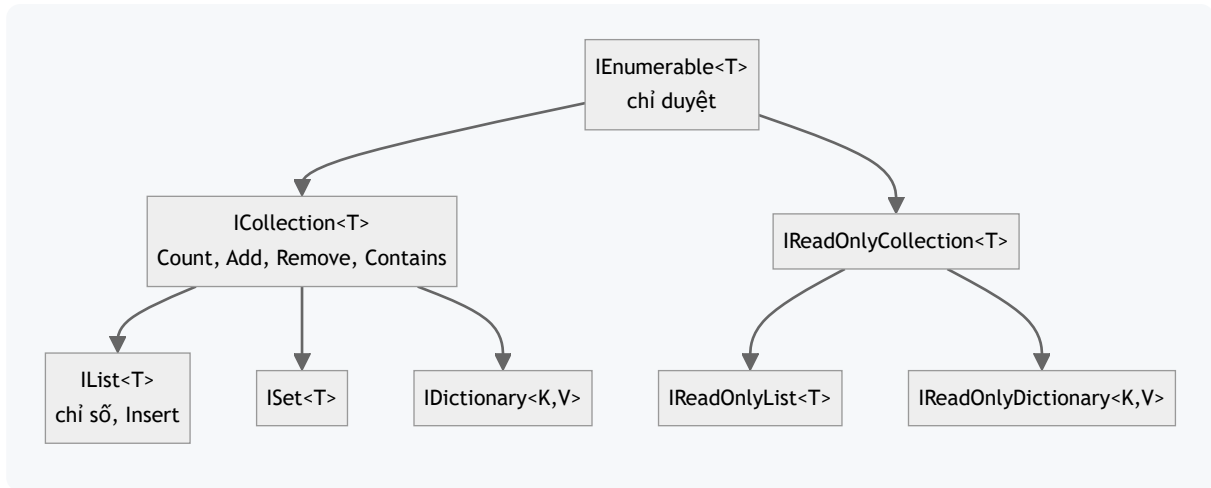
Bảng thao tác điển hình:

	<code>List<T></code>	<code>LinkedList<T></code>	<code>Dictionary</code>	<code>HashSet</code>	<code>SortedSet</code>
Truy cập theo chỉ số	$O(1)$	$O(n)$	—	—	—
Tìm giá trị / <code>Contains</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(\log n)$
Thêm cuối	$O(1)^*$	$O(1)$	$O(1)^*$	$O(1)^*$	$O(\log n)$
Chèn/xoá giữa	$O(n)$	$O(1)^{**}$	—	—	—

* trung bình (đôi khi phải cấp phát lại — Chương 25). ** khi đã có sẵn node.

Chương trình mẫu đo trực tiếp: 1000 lần `Contains` trên 100.000 phần tử — `List` chậm hơn `HashSet` hàng trăm lần. Chọn sai collection là lý do phổ biến nhất khiến chương trình "chậm đột ngột khi dữ liệu lớn".

Hệ thống interface



Nguyên tắc: **tham số nhận interface tối thiểu cần thiết, kiểu trả về đủ cụ thể.**

- Chỉ cần duyệt → `IEnumerable<T>` .
- Cần `Count` /chỉ số nhưng không sửa → `IReadOnlyList<T>` .
- Bên ngoài không được sửa → trả `IReadOnlyList<T>` (Chương 14).

Các collection cốt lõi — thao tác chính

```

var ds = new List<string> { "An", "Binh" };
ds.Add("Chi"); ds.Insert(0, "Dung"); ds.Remove("Binh");

var tuoi = new Dictionary<string, int> { ["An"] = 20 };
if (tuoi.TryGetValue("An", out int t)) { ... }

var so = new HashSet<int> { 1, 2, 3, 3 }; // chỉ còn 1, 2, 3

var q = new Queue<string>(); q.Enqueue("a"); q.Dequeue();
var st = new Stack<string>(); st.Push("a"); st.Pop();
  
```

Khởi tạo bằng **collection expression** (C# 12): `List<int> ds = [1, 2, 3];` , `int[] mang = [1, 2, 3];` , `IReadOnlyList<int> r = [1, 2, 3];` .

Chọn kích thước và bảo vệ dữ liệu

- Biết trước số lượng → `new List<T>(capacity)` tránh cấp phát lại.
- Đừng trả `List<T>` nội bộ ra ngoài; trả `IReadOnlyList<T>` (Chương 14) hoặc bản sao.
- Không sửa (thêm/xoá) collection **khi đang foreach** trên nó → `InvalidOperationException` (Chương 25).

Lỗi thường gặp

- Dùng `List.Contains` trong vòng lặp lớn ($O(n^2)$ ẩn) — chuyển sang `HashSet / Dictionary` .
- `KeyNotFoundException` khi `dict[key]` với khoá chưa có — dùng `TryGetValue` .
- `InvalidOperationException: Collection was modified` khi sửa lúc duyệt.
- `InvalidOperationException: Queue/Stack empty` — kiểm tra `Count` hoặc dùng `TryDequeue / TryPop` .
- Lưu số lượng lớn vào `List<int>` rồi tra cứu theo giá trị (nên dùng `HashSet<int>`).
- Dùng `ArrayList / Hashtable` cũ.

Bài tập

1. Đọc danh sách từ một chuỗi, in số từ **khác nhau** (dùng `HashSet`).
2. Kiểm tra dấu ngoặc cân bằng trong biểu thức `"(a + [b * {c}])"` bằng `Stack<char>` .
3. Mô phỏng hàng đợi in ấn bằng `Queue<string>` : thêm 5 tài liệu, in từng cái theo thứ tự.
4. Đo thời gian `List.Contains` và `HashSet.Contains` với $n = 10^4, 10^5, 10^6$ và vẽ nhận xét về tốc độ tăng.
5. Cho `List<SinhVien>` , dựng `Dictionary<string, SinhVien>` theo mã để tra cứu nhanh.

Chương 25 — List<T> và LinkedList<T>

Mục tiêu học

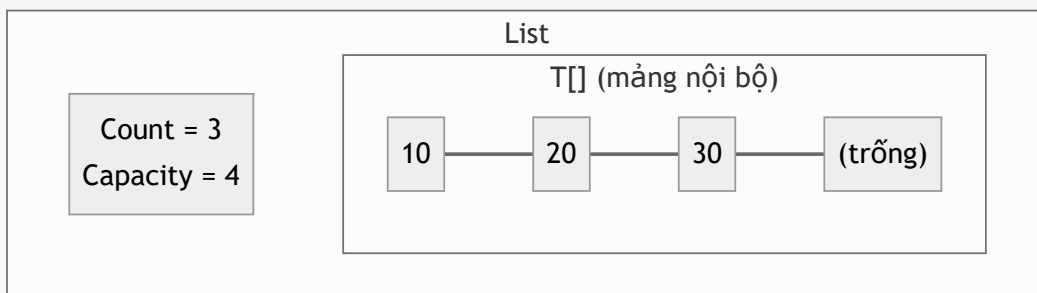
Sau chương này, bạn sẽ:

- Hiểu List<T> hoạt động bên trong ra sao (mảng động, Capacity, nhân đôi).
- Dùng thành thạo các thao tác của List<T> và hiểu chi phí từng thao tác.
- Biết LinkedList<T> khác gì và **vì sao hiếm khi nên dùng**.
- Tránh lỗi sửa danh sách trong lúc foreach.

Code mẫu: [code/ch25-list-linkedlist/](#).

List<T> bên trong: mảng động

List<T> bọc một **mảng** T[] cùng một số đếm Count. Mảng nội bộ dài Capacity ≥ Count.



Khi Add mà Count == Capacity, List tạo mảng mới **gấp đôi**, chép phần tử sang rồi bỏ mảng cũ:

```
public void Them(T item)
{
    if (Count == _data.Length)
        Array.Resize(ref _data, _data.Length * 2); // tốn O(n) nhưng hiếm
    _data[Count++] = item;
}
```

Chạy chương trình mẫu để thấy Capacity nhảy 4 → 8 → 16 ... Đây là **chi phí khấu hao O(1)**: thỉnh thoảng một lần chép tốn O(n), nhưng chia đều thì mỗi Add chỉ tốn O(1). Vì thế:

- Truy cập list[i] là **O(1)** (tính địa chỉ trực tiếp).
- Add cuối là O(1) khấu hao; Insert / RemoveAt **đầu/giữa** là O(n) (phải dịch các phần tử).
- Contains, IndexOf, Remove(giaTri) là O(n) (duyệt).

Biết trước cỡ → `new List<T>(1000)` để khởi cấp phát lại nhiều lần.

Thao tác thường dùng

```
var so = new List<int> { 5, 3, 8, 1, 9, 2 };
so.Add(7); // thêm cuối
so.AddRange([4, 6]); // thêm nhiều
so.Insert(0, 100); // chèn tại vị trí
so.Remove(8); // xoá phần tử đầu tiên bằng 8
so.RemoveAt(0); // xoá theo chỉ số
so.RemoveAll(x => x % 2 == 0); // xoá theo điều kiện
so.Sort(); // sắp xếp tại chỗ
int vt = so.BinarySearch(8); // nhị phân – chỉ đúng khi ĐÃ sắp xếp
int? dau = so.Find(x => x > 3); // phần tử đầu thoả điều kiện (default nếu không có)
so.Contains(9); so.IndexOf(9); so.Count; so.Clear();
so.GetRange(1, 3); so.Reverse();
```

`List<T>` cho phép phần tử trùng và `null` (nếu `T` là kiểu tham chiếu).

Sửa danh sách khi đang duyệt

```
foreach (var x in t)
    if (x == 2) t.Remove(x); // InvalidOperationException: Collection was modified
```

`foreach` dùng bộ liệt kê (enumerator) theo dõi một "số phiên bản" của danh sách; sửa cấu trúc giữa chừng làm số phiên bản đổi → lần `MoveNext` kế tiếp ném lỗi. Cách làm đúng:

```
t.RemoveAll(x => x == 2); // gọn nhất
for (int i = t.Count - 1; i >= 0; i--) // duyệt ngược rồi xoá
    if (t[i] == 2) t.RemoveAt(i);
var ketQua = t.Where(x => x != 2).ToList(); // hoặc tạo danh sách mới
```

(Gán lại giá trị phần tử `t[i] = ...` trong `for` thì được; chỉ thay đổi *cấu trúc* — thêm/xoá — mới bị cấm khi `foreach`.)

LinkedList<T> — danh sách liên kết đôi

Mỗi phần tử là một **node** chứa giá trị và con trỏ tới node trước/sau:



```
var ll = new LinkedList<string>();
ll.AddLast("B"); ll.AddFirst("A");
var nodeB = ll.Find("B")!;
ll.AddAfter(nodeB, "C");
ll.Remove(nodeB);           // O(1) vì đã có node
```

	List<T>	LinkedList<T>
Truy cập chỉ số	O(1)	O(n) (không có chỉ số!)
Thêm/xoá đầu	O(n)	O(1)
Thêm/xoá giữa khi có node	O(n)	O(1)
Bộ nhớ	gọn, liền kề	mỗi node tốn thêm 2 con trỏ + đối tượng riêng
Cache CPU	rất tốt (liền kề)	kém (rải rác trên heap)

Thực tế: List<T> gần như luôn nhanh hơn LinkedList<T> kể cả ở những thao tác "lý thuyết" thuộc về danh sách liên kết, nhờ dữ liệu liền kề hợp với bộ nhớ đệm CPU. Chỉ cần nhắc LinkedList khi thật sự cần chèn/xoá liên tục ở giữa với các node đã nắm giữ (ví dụ cache LRU). Nếu chỉ cần thêm/xoá ở hai đầu, dùng Queue<T> / Stack<T> / Deque dạng PriorityQueue ... phù hợp hơn.

Mảng hay List<T>?

- Số lượng cố định, hiệu năng tối đa, tham số params → **mảng**.
- Số lượng thay đổi → List<T>.
- Chỉ duyệt một lần, sinh dần dữ liệu → IEnumerable<T> với yield (Chương 27).

Lỗi thường gặp

- InvalidOperationException: Collection was modified khi thêm/xoá trong foreach.
- ArgumentOutOfRangeException — chỉ số ngoài [0, Count). Lưu ý Count (không phải Length) với List.
- BinarySearch trên danh sách **chưa sắp xếp** cho kết quả sai (không báo lỗi).
- Dùng Insert(0, x) / RemoveAt(0) trong vòng lặp lớn → O(n²).
- Nhầm Remove(x) (theo giá trị, xoá phần tử đầu tiên khớp, trả bool) với RemoveAt(i).
- Trả List<T> nội bộ ra ngoài cho người khác sửa tùy ý.

Bài tập

- Mở rộng MangDong<T> với Xoa(int index) và Chen(int index, T item); kiểm tra độ phức tạp.

2. Xóa mọi số âm khỏi `List<int>` theo ba cách khác nhau (`RemoveAll` , duyệt ngược, `Where`).
3. Đo thời gian thêm 100.000 phần tử vào **đầu** `List<int>` bằng `Insert(0, x)` so với `LinkedList.AddFirst` ; giải thích kết quả.
4. Cài đặt lịch sử trình duyệt: `List<string>` với con trỏ "trang hiện tại" và các thao tác Back/Forward.
5. Viết hàm loại phần tử trùng khỏi `List<int>` giữ nguyên thứ tự lần xuất hiện đầu (gợi ý `HashSet`).

Chương 26 — Dictionary, HashSet và họ hàng

Mục tiêu học

Sau chương này, bạn sẽ:

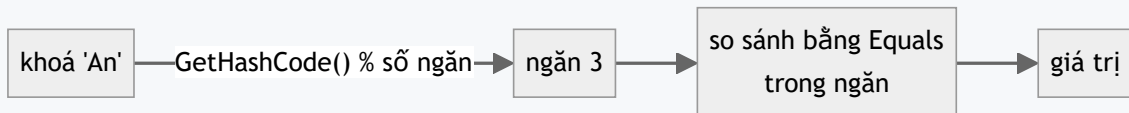
- Hiểu **bảng băm** (hash table) hoạt động thế nào và vì sao tra cứu $O(1)$.
- Dùng `Dictionary<K,V>` và `HashSet<T>` đúng cách (`TryGetValue`, phép toán tập hợp).
- Hiểu vai trò then chốt của `Equals` / `GetHashCode` với khoá.
- Biết khi nào dùng `SortedDictionary`, `SortedSet`.

Code mẫu: [code/ch26-dictionary-hashset/](#).

Bảng băm hoạt động ra sao?

`Dictionary<TKey, TValue>` và `HashSet<T>` dùng **bảng băm**. Ý tưởng:

1. Từ khoá `k`, gọi `k.GetHashCode()` được một số nguyên (**mã băm**).
2. Lấy mã băm chia dư cho số "ngăn" (bucket) → biết cần nhìn vào ngăn nào.
3. Trong ngăn chỉ có vài phần tử; so sánh bằng `Equals` để tìm đúng khoá.



Không phải duyệt cả tập, nên tra cứu/thêm/xoá là **$O(1)$ trung bình** — bất kể có 10 hay 10 triệu phần tử. Đổi lại: thứ tự duyệt **không được đảm bảo**, và tốn bộ nhớ hơn `List`.

`Dictionary<TKey, TValue>`

```
var diem = new Dictionary<string, double> { ["An"] = 8.5, ["Binh"] = 7.0 };
diem["Chi"] = 9.0;           // thêm, hoặc ghi đè nếu đã có
diem.Add("Dung", 6.5);       // thêm, NÉM lỗi nếu khoá đã tồn tại
diem.TryAdd("An", 1.0);      // thêm nếu chưa có, trả bool

double d = diem["An"];        // NÉM KeyNotFoundException nếu không có
if (diem.TryGetValue("Zzz", out var v)) { ... } // an toàn
diem.GetValueOrDefault("Zzz", -1); // giá trị mặc định nếu không có

diem.ContainsKey("An"); diem.Remove("An"); diem.Count;
```

```
foreach (var (ten, d2) in diem) { ... }           // duyệt cặp key/value
diem.Keys; diem.Values;
```

Mẫu đếm tần suất (rất hay gặp):

```
var tanSuat = new Dictionary<string, int>();
foreach (var t in tu)
    tanSuat[t] = tanSuat.GetValueOrDefault(t) + 1;
```

Quy tắc: khoá là duy nhất và không được `null` ; giá trị thì tùy. Luôn ưu tiên `TryGetValue` khi không chắc khoá có mặt.

HashSet<T>

Tập hợp **không trùng lặp**, kiểm tra "có chứa?" $O(1)$:

```
var so = new HashSet<int> { 1, 2, 3, 3, 3 };    // {1, 2, 3}
so.Add(2);           // false: đã có
so.Contains(2);      // true,  $O(1)$ 
```

Có sẵn các phép toán tập hợp (thay đổi tập gọi phương thức):

```
var hop = new HashSet<int>(a); hop.UnionWith(b);    // hợp
var giao = new HashSet<int>(a); giao.IntersectWith(b); // giao
var hieu = new HashSet<int>(a); hieu.ExceptWith(b); // hiệu (a \ b)
a.IsSubsetOf(b); a.Overlaps(b);
```

Mẹo: loại trùng nhanh — `var duyNhat = new HashSet<int>(danhSach);` hoặc `danhSach.Distinct()` (LINQ).

Vai trò của Equals và GetHashCode

Bảng băm chỉ hoạt động đúng nếu **khóa đồng nhất có cùng mã băm và Equals đúng** (Chương 20). Ví dụ:

```
class DiemSai(int x, int y) { public int X { get; } = x; public int Y { get; } = y; }

var sai = new HashSet<DiemSai> { new(1, 2) };
sai.Contains(new DiemSai(1, 2));    // FALSE! mặc định so sánh tham chiếu, mã băm khác

record DiemDung(int X, int Y);      // record tự sinh Equals/GetHashCode theo giá trị
new HashSet<DiemDung> { new(1, 2) }.Contains(new DiemDung(1, 2));    // TRUE
```

Vì vậy dùng làm khoá: ưu tiên `string`, số, `Guid`, `enum`, `record`, hoặc tự viết đúng cả hai phương thức. Khoá phải **bất biến**: nếu đổi giá trị khoá sau khi đã bỏ vào tập, mã băm đổi và phần tử "lạc" mất.

Tuỳ chỉnh cách so sánh khoá bằng `IEqualityComparer<T>`:

```
var d = new Dictionary<string, int>(StringComparer.OrdinalIgnoreCase) { ["Abc"] = 1 };
d.ContainsKey("ABC"); // true
```

Họ "sorted": SortedDictionary, SortedSet, SortedList

Giữ phần tử **luôn theo thứ tự khoá**, cài bằng cây cân bằng nên các thao tác đều **$O(\log n)$** :

```
var sd = new SortedDictionary<string, int> { ["chuoi"] = 3, ["tao"] = 1, ["cam"] = 2 }
// duyệt ra: cam, chuoi, táo
var ss = new SortedSet<int> { 5, 1, 3 }; // Min = 1, Max = 5; GetViewBetween(2, 4)
```

Dùng khi cần **duyệt theo thứ tự** hoặc truy vấn khoảng (nhỏ nhất/lớn nhất, "các khoá giữa a và b"). Nếu chỉ tra cứu theo khoá thì `Dictionary` nhanh hơn.

Sửa dictionary khi đang duyệt

Thêm/xoá khoá trong lúc `foreach` gây `InvalidOperationException`. Sao chép danh sách khoá trước:

```
foreach (var k in diem.Keys.ToList()) // ToList() cho phép sửa an toàn
    diem[k + "_moi"] = 0;
```

Chọn gì?

Bài toán	Dùng
Tra cứu theo mã/khoá	<code>Dictionary</code>
Đếm/nhóm theo khoá	<code>Dictionary<K, int></code> hoặc <code>GroupBy</code> (Chương 30)
Loại trùng, kiểm tra tồn tại	<code>HashSet</code>
Cần thứ tự khoá	<code>SortedDictionary/SortedSet</code>
Tra cứu theo hai chiều	hai <code>Dictionary</code>

Lỗi thường gặp

- `KeyNotFoundException` — dùng `dict[key]` thay vì `TryGetValue` .
- `ArgumentException`: An item with the same key has already been added — Add khoá trùng (dùng `dict[key] = v`).
- Dùng class **không override** `Equals/GetHashCode` làm khoá.
- Khoá thay đổi sau khi đã thêm.
- Khoá `null` → `ArgumentNullException` .
- Giả định thứ tự duyệt của `Dictionary` (không đảm bảo).

Bài tập

1. Đếm số lần xuất hiện của từng từ trong một đoạn văn; in 3 từ nhiều nhất (`OrderByDescending`).
2. Tìm hai số trong mảng có tổng bằng `k` trong $O(n)$ bằng `HashSet / Dictionary` .
3. Cho hai danh sách email, tìm email chỉ có ở danh sách thứ nhất, và email có ở cả hai (phép toán tập hợp).
4. Sửa `DiemSai` cho đúng (override `Equals` và `GetHashCode`) và kiểm tra `Contains` .
5. Dựng từ điển Anh–Việt không phân biệt hoa/thường bằng `StringComparer.OrdinalIgnoreCase` .

Chương 27 — IEnumerable, yield và sắp xếp

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu `foreach` hoạt động dựa trên `IEnumerable<T>` / `IEnumerator<T>`.
- Viết bộ sinh dữ liệu bằng `yield return` và hiểu **thực thi trì hoãn** (deferred execution).
- Tự cài đặt `IEnumerable<T>` cho lớp của mình.
- Sắp xếp bằng `Sort`, `OrderBy/ThenBy`, `IComparer` / `Comparer.Create`, biết sự khác nhau.

Code mẫu: [code/ch27-ienumerable-yield/](#).

foreach thực sự làm gì?

```
foreach (var x in ds) { ... }
```

được trình biên dịch biến thành:

```
using var it = ds.GetEnumerator();
while (it.MoveNext())
{
    var x = it.Current;
    ...
}
```

Tức là **bất kỳ kiểu nào có** `GetEnumerator()` (thường là cài `IEnumerable<T>`) đều dùng được với `foreach`.

```
interface IEnumerable<T> { IEnumerator<T> GetEnumerator(); }
interface IEnumerator<T> { bool MoveNext(); T Current { get; } void Reset(); }
```

`IEnumerable<T>` là interface "gốc" của mọi collection và là kiểu nền của **LINQ** (Chương 30): nó chỉ nói "tôi có thể đưa bạn từng phần tử một", **không** nói có bao nhiêu hay cho truy cập theo chỉ số.

yield return — sinh dữ liệu theo yêu cầu

Viết bộ liệt kê bằng tay khá dài. `yield return` để trình biên dịch tự sinh:

```
static IEnumerable<long> Fibonacci()
{
    long a = 0, b = 1;
    while (true)           // vô hạn, nhưng an toàn!
    {
        yield return a;     // "trả" một giá trị và tạm dừng tại đây
        (a, b) = (b, a + b);
    }
}

foreach (var f in Fibonacci().Take(10)) Console.Write($"{f} ");
```

Mỗi lần `MoveNext()`, phương thức chạy tiếp từ chỗ dừng cho tới `yield return` kế tiếp. Nhờ vậy:

- **Tiết kiệm bộ nhớ:** không dựng toàn bộ danh sách, kể cả dãy vô hạn hay rất lớn (đọc từng dòng file khổng lồ).
- **Lười (lazy):** chỉ tính khi được yêu cầu. `Take(10)` chỉ chạy 10 vòng.
- `yield break;` kết thúc dãy sớm.

Thực thi trì hoãn (deferred execution)

```
static IEnumerable<int> DemSo(int n)
{
    for (int i = 1; i <= n; i++) { Console.WriteLine($" sinh {i}"); yield return i; }
}

var seq = DemSo(3);           // CHƯA chạy dòng nào của DemSo
Console.WriteLine("Da tao seq");
foreach (var x in seq) Console.WriteLine($"nhân {x}"); // giờ mới chạy: sinh 1, nhân
```

Kết quả xen kẽ "sinh" và "nhân" — chứng tỏ giá trị được sinh **từng cái một** theo nhịp `foreach`. Hệ quả cần nhớ:

1. Phương thức `yield` **không chạy** cho tới khi bắt đầu duyệt — cả kiểm tra tham số cũng bị hoãn (nếu cần kiểm tra ngay, tách thành phương thức bọc bên ngoài rồi gọi phương thức `yield` bên trong).
2. Duyệt **hai lần** thì chạy lại từ đầu hai lần; nếu tốn kém, hãy `.ToList()` để "chốt" kết quả.
3. Dữ liệu nguồn thay đổi giữa lúc tạo và lúc duyệt sẽ ảnh hưởng kết quả (Chương 30).

Tự cài `IEnumerable<T>`

```

class DanhSachTen(string[] ten) : IEnumerable<string>
{
    public IEnumerator<string> GetEnumerator()
    {
        foreach (var t in ten) yield return t;
    }

    System.Collections.IEnumerator System.Collections.IEnumerable.GetEnumerator() => GetEnumerator()
}

foreach (var t in new DanhSachTen(["An", "Binh"])) ...

```

Cài đặt interface không generic `IEnumerable` (dòng cuối) chỉ là thủ tục kế thừa; chuyển thẳng tới bản generic. Sau khi cài đặt, lớp của bạn dùng được với `foreach` **và mọi phương thức LINQ** (`Where`, `Select` ...).

Sắp xếp

`List<T>.Sort` — tại chỗ

```
sv.Sort((x, y) => y.Diem.CompareTo(x.Diem)); // giảm dần theo điểm
```

- Thay đổi **danh sách gốc**, trả về `void`.
- **Không ổn định**: hai phần tử "bằng nhau" có thể đổi thứ tự tương đối. Dùng `IComparable<T>` / `IComparer<T>` / lambda (Chương 20).

`OrderBy / ThenBy (LINQ)` — tạo dãy mới

```
var xep = sv.OrderByDescending(s => s.Diem).ThenBy(s => s.Ten);
```

- **Ổn định** (giữ thứ tự gốc của các phần tử bằng nhau), **không** sửa nguồn, trả về `IOrderedEnumerable<T>`, thực thi trì hoãn.
- `ThenBy / ThenByDescending` thêm khoá phụ. Rất dễ đọc — chọn cách này cho hầu hết trường hợp.

`Comparer<T>.Create` và `IComparer<T>`

```

var theoTen = Comparer<SinhVien>.Create((x, y) => string.Compare(x.Ten, y.Ten, StringComparison.Ordinal));
sv.Sort(theoTen);

```

Dùng khi cần truyền một bộ so sánh có thể tái sử dụng (cho `Sort` , `SortedSet` , `SortedDictionary` , `Array.Sort` , `OrderBy(x => x, comparer)`).

Sắp xếp chuỗi

Luôn chỉ rõ `StringComparison` / `StringComparer` — sắp xếp theo culture khác nhau cho kết quả khác nhau và chậm hơn `Ordinal` .

Khi nào trả `IEnumerable<T>`?

- Bạn muốn trả một dãy chỉ-đọc, có thể lười, và người dùng chỉ cần duyệt.
- Nhưng đừng để lộ nguồn thay đổi được, và nếu người gọi sẽ duyệt nhiều lần hoặc cần `Count` , hãy trả `IReadOnlyList<T>` .
- Cảnh giác **duyệt nhiều lần**: mỗi lần lại chạy lại toàn bộ truy vấn/truy cập CSDL phía sau.

Lỗi thường gặp

- Tưởng phương thức `yield` chạy ngay khi gọi.
- Duyệt cùng `IEnumerable` nhiều lần ngầm gây tính lại tốn kém (hoặc gọi CSDL nhiều lần).
- `yield return` trong `try/catch` (không được phép với `catch`) hoặc trong phương thức có tham số `ref` / `out` .
- Dùng `Sort` khi cần sắp xếp **ổn định**.
- `NullReferenceException` do so sánh giá trị `null` trong bộ so sánh tự viết.
- Dùng `Count()` (LINQ) trên `IEnumerable` khổng lồ chỉ để hỏi "có phần tử nào không?" — dùng `Any()` .

Bài tập

1. Viết `IEnumerable<int> SoChan(int toiDa)` bằng `yield return` ; in ra và dùng LINQ `Take(5)` .
2. Viết `IEnumerable<string> DocDong(string path)` đọc file theo từng dòng bằng `yield` (hoặc gọi `File.ReadLines`).
3. Tự cài `PhamVi` (`IEnumerable<int>`) để `foreach (var i in new PhamVi(1, 5))` chạy được.
4. So sánh `Sort` và `OrderBy` trên danh sách có nhiều phần tử điểm bằng nhau — thứ tự có khác không? Vì sao?
5. Sắp xếp danh sách tên tiếng Việt theo `StringComparer.Create(new CultureInfo("vi-VN"), true)` và so với `Ordinal` .

Chương 28 — Thuật toán tìm kiếm

Mục tiêu học

Sau chương này, bạn sẽ:

- Cài đặt và so sánh bốn thuật toán tìm kiếm: **Linear, Binary, Jump, Interpolation**.
- Đọc hiểu độ phức tạp thời gian của mỗi thuật toán và điều kiện áp dụng.
- Biết dùng lại hàm có sẵn của .NET (`Array.BinarySearch` , `List.BinarySearch` , `Contains` , `HashSet`).
- Chọn thuật toán/cấu trúc phù hợp với dữ liệu và nhu cầu.

Code mẫu: [code/ch28-thuat-toan-tim-kiem/](#) .

Bài toán

Cho một tập giá trị, tìm vị trí của giá trị `x` (hoặc `-1` nếu không có). Câu hỏi thực tế: **nhANH đến đâu, và dữ liệu cần đáp ứng điều kiện gì?** Code mẫu đếm số "bước" (số lần so sánh) mỗi thuật toán dùng để bạn thấy sự khác biệt.

1. Tìm tuần tự (Linear Search) — $O(n)$

Duyệt từ đầu, so từng phần tử:

```
public static int Tim(int[] a, int x)
{
    for (int i = 0; i < a.Length; i++)
        if (a[i] == x) return i;
    return -1;
}
```

- **Không yêu cầu sắp xếp.** Đơn giản, đúng với mọi loại dữ liệu.
- Trường hợp xấu nhất: n phép so sánh (không thấy hoặc ở cuối).
- Tốt cho dữ liệu nhỏ, chỉ tìm vài lần, hoặc chưa sắp xếp.

2. Tìm nhị phân (Binary Search) — $O(\log n)$

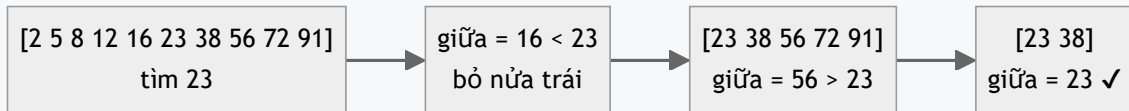
Điều kiện: mảng **đã sắp xếp**. Mỗi bước so với phần tử **giữa**, loại bỏ một nửa:

```
int lo = 0, hi = a.Length - 1;
while (lo <= hi)
```

```

{
    int mid = lo + (hi - lo) / 2;    // không dùng (lo + hi) / 2 vì có thể tràn số
    if (a[mid] == x) return mid;
    if (a[mid] < x) lo = mid + 1;
    else hi = mid - 1;
}
return -1;

```



Với 1.000.000 phần tử chỉ cần khoảng **20** phép so sánh. Đây là thuật toán quan trọng nhất của chương. Trong .NET có sẵn: `Array.BinarySearch(a, x)`, `list.BinarySearch(x)` (trả số âm — phần bù của vị trí chèn — nếu không thấy).

Lỗi kinh điển: `(lo + hi) / 2` có thể tràn `int` khi mảng rất lớn; `lo + (hi - lo) / 2` an toàn.

3. Tìm nhảy (Jump Search) — $O(\sqrt{n})$

Điều kiện: mảng đã sắp xếp. **Nhảy** từng khối kích thước $\sqrt{n}$ cho tới khi vượt quá `x`, rồi tìm **tuần tự** trong khối đó:

```

int buocNhay = (int)Math.Sqrt(n);
int truoc = 0, cur = buocNhay;
while (cur < n && a[cur - 1] < x) { truoc = cur; cur += buocNhay; }
for (int i = truoc; i < Math.Min(cur, n); i++)
{
    if (a[i] == x) return i;
    if (a[i] > x) break;
}
return -1;

```

Chậm hơn nhị phân nhưng chỉ **duyệt tiến** (không nhảy lùi) — hợp với dữ liệu mà "quay lại" tốn kém (băng từ, luồng đọc tuần tự). Trong thực tế ít dùng hơn nhị phân, nhưng đáng biết như một giải pháp trung gian.

4. Tìm nội suy (Interpolation Search) — trung bình $O(\log \log n)$

Điều kiện: đã sắp xếp và giá trị **phân bố khá đều**. Thay vì luôn chọn giữa, **ước lượng** vị trí theo tỷ lệ giá trị (giống tìm tên trong danh bạ: tìm "Zoe" mở gần cuối):

```
int pos = lo + (int)((long)(x - a[lo]) * (hi - lo) / (a[hi] - a[lo]));
```

Với dữ liệu phân bố đều, nhảy gần như thẳng tới đích: cực nhanh (0, 3, 6, 9, ...). Nhưng với dữ liệu lệch nhiều (ví dụ 1, 2, 4, 8, 16, ... 2^{30}) có thể suy biến thành **$O(n)$** . Luôn kiểm tra `a[hi] == a[lo]` để tránh chia cho 0 và giữ `x` trong khoảng `[a[lo], a[hi]]`.

So sánh tổng hợp

Thuật toán	Yêu cầu sắp xếp	Thời gian (tệ nhất)	Trung bình / ghi chú
Linear	Không	$O(n)$	đơn giản, mọi kiểu dữ liệu
Binary	Có	$O(\log n)$	mặc định cho mảng đã sắp xếp
Jump	Có	$O(\sqrt{n})$	chỉ duyệt tiến
Interpolation	Có + phân bố đều	$O(n)$	$O(\log \log n)$ khi dữ liệu đều

Chạy chương trình mẫu (mảng 0, 3, 6, ..., 2997, tìm 2001): tuyến tính cần ~668 bước, nhị phân chỉ ~10, nhảy ~40, nội suy chỉ 1 bước. Con số chính xác in ra khi bạn chạy.

Đừng quên: cấu trúc dữ liệu cũng là "thuật toán tìm kiếm"

Nếu tìm kiếm là thao tác thường xuyên, thường **chọn đúng cấu trúc** tốt hơn tối ưu thuật toán trên mảng:

Tình huống	Chọn
Tìm theo khoá, dữ liệu đổi liên tục	Dictionary<K,V> / HashSet<T> — $O(1)$
Cần thứ tự + tìm theo khoảng	SortedSet<T> / SortedDictionary — $O(\log n)$
Mảng tĩnh đã sắp xếp, tìm nhiều lần	Array.BinarySearch
Dữ liệu ít, tìm vài lần	Contains / Array.IndexOf (tuần tự)
Dữ liệu chưa sắp xếp, tìm một lần	tuần tự (sắp xếp $O(n \log n)$ sẽ không đáng)

Sắp xếp một lần rồi tìm nhiều lần thì đáng; sắp xếp để tìm đúng một lần thì lãng phí.

Lỗi thường gặp

- Tìm nhị phân/nhảy/nội suy trên mảng **chưa sắp xếp** — kết quả sai mà không báo lỗi.
- Lệch một ở biên: `hi = a.Length` thay vì `a.Length - 1`; `while (lo < hi)` thay vì `<=`.

- Tràn số khi tính `mid` bằng `(lo + hi) / 2`.
- Nội suy chia cho 0 khi `a[hi] == a[lo]`; quên kiểm tra `x` nằm trong khoảng.
- Quên rằng `BinarySearch` trả **số âm** khi không thấy (dùng `~kq` để lấy vị trí chèn).
- Nhầm `Array.BinarySearch` với dữ liệu trùng: không đảm bảo trả về vị trí đầu tiên.

Bài tập

1. Sửa `TimNhiPhan` để trả về vị trí **đầu tiên** của `x` khi có nhiều giá trị trùng.
2. Cài `TimNhiPhan` **đệ quy** và so sánh với bản lặp.
3. Tạo mảng phân bố lệch (1, 2, 4, 8, ...) và đếm số bước của nội suy so với nhị phân.
4. Dùng `Array.BinarySearch` để chèn một phần tử vào `List<int>` đã sắp xếp mà vẫn giữ thứ tự (dùng `~kq`).
5. Đo (`Stopwatch`) tìm 1000 giá trị ngẫu nhiên trong 1.000.000 phần tử: `Contains` trên `List`, `BinarySearch`, `HashSet`.

Chương 29 — Duyệt đồ thị và cây: BFS và DFS

Mục tiêu học

Sau chương này, bạn sẽ:

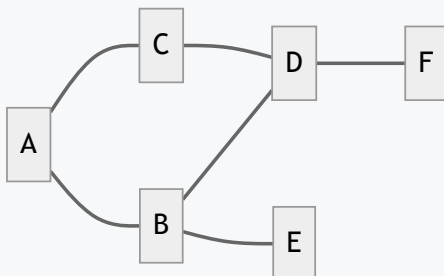
- Biểu diễn **đồ thị** bằng danh sách kề và **cây** bằng đối tượng lồng nhau.
- Cài đặt **BFS** (dùng `Queue`) và **DFS** (dùng `Stack` hoặc đệ quy).
- Dùng BFS tìm **đường đi ngắn nhất** trên đồ thị không trọng số.
- Duyệt cây thư mục / cấu trúc phân cấp và biết khi nào chọn BFS hay DFS.

Code mẫu: [code/ch29-bfs-dfs/](#).

Đồ thị và cây

Đồ thị = tập **đỉnh** (node) và tập **cạnh** nối các đỉnh. Ví dụ: bản đồ đường phố, mạng bạn bè, các trang web liên kết. **Cây** là đồ thị đặc biệt: liên thông, không có chu trình, có một **gốc** (thư mục, cây menu, cấu trúc công ty).

Ví dụ đồ thị dùng trong chương:



Biểu diễn: danh sách kề

Mỗi đỉnh giữ danh sách các đỉnh kề của nó:

```
class DoThi
{
    private readonly Dictionary<string, List<string>> _ke = [];

    public void ThemCanh(string a, string b)
    {
        if (!_ke.ContainsKey(a)) _ke[a] = [];
        if (!_ke.ContainsKey(b)) _ke[b] = [];
        _ke[a].Add(b);
    }
}
```

```

        _ke[b].Add(a);        // đồ thị vô hướng: thêm cả hai chiều
    }
}

```

Tiết kiệm bộ nhớ ($O(V+E)$) và duyệt các đỉnh kề nhanh — phù hợp đa số bài toán. (Ma trận kề `bool[,]` tốn $O(V^2)$ — chỉ hợp đồ thị dày, nhỏ.)

BFS — Duyệt theo chiều rộng

Duyệt **theo từng "lớp"** từ gần đến xa: đỉnh cách nguồn 1 cạnh, rồi 2 cạnh, rồi 3 cạnh... Dùng **hàng đợi (Queue)**:

```

public List<string> BFS(string batDau)
{
    var kq = new List<string>();
    var daTham = new HashSet<string> { batDau };
    var hangDoi = new Queue<string>();
    hangDoi.Enqueue(batDau);
    while (hangDoi.Count > 0)
    {
        var u = hangDoi.Dequeue();
        kq.Add(u);
        foreach (var v in _ke[u])
            if (daTham.Add(v)) hangDoi.Enqueue(v);    // Add trả false nếu đã thăm
    }
    return kq;
}

```

Từ `A`: `A → B → C → D → E → F` (A trước, rồi các kề của A là B, C; rồi kề của B, C...).

- `daTham` là bắt buộc: đồ thị có chu trình (A–B–D–C–A), thiếu nó sẽ lặp mãi.
- Độ phức tạp **$O(V + E)$** : mỗi đỉnh vào hàng đợi một lần, mỗi cạnh xét một lần.

Đường đi ngắn nhất (không trọng số)

Vì BFS đi theo lớp, lần đầu chạm tới một đỉnh chính là bằng **số cạnh ít nhất**. Lưu "đỉnh cha" để dựng lại đường đi:

```

var cha = new Dictionary<string, string?> { [tu] = null };
// ... BFS, mỗi khi phát hiện v từ u: cha[v] = u
// dựng đường: đi ngược từ đích về nguồn qua cha rồi đảo lại

```

`DuongDiNganNhat("A", "F") → A → B → D → F`. Đây là cách giải các bài toán "ít bước nhất": mê cung, chuyển đổi từ này sang từ khác, số lần bấm ít nhất, bậc quan hệ ("bạn của bạn").

Đồ thị **có trọng số** (mỗi cạnh có chi phí) cần thuật toán khác: **Dijkstra**, dùng `PriorityQueue` — vượt ngoài phạm vi chương này.

DFS — Duyệt theo chiều sâu

Đi **sâu nhất có thể** theo một nhánh, hết đường thì **quay lui**. Dùng **ngăn xếp (Stack)** hoặc **đệ quy** (thực chất dùng ngăn xếp gọi hàm):

```
// Đệ quy: gọn nhất
public List<string> DFSDeQuy(string batDau)
{
    var kq = new List<string>();
    var daTham = new HashSet<string>();
    void Tham(string u)
    {
        if (!daTham.Add(u)) return;
        kq.Add(u);
        foreach (var v in _ke[u]) Tham(v);
    }
    Tham(batDau);
    return kq;
}
```

Từ A: A → B → D → C → F → E (đi sâu qua B, D... rồi mới quay lại). Bản khử đệ quy dùng `Stack<string>` cho kết quả tương tự (đảo thứ tự đẩy vào để giữ thứ tự duyệt). Bản dùng `Stack` tránh được `StackOverflowException` khi đồ thị rất sâu. Cũng **$O(V + E)$** .

BFS hay DFS?

	BFS (Queue)	DFS (Stack/đệ quy)
Thứ tự	theo lớp, gần → xa	sâu nhất trước
Đường ngắn nhất (không trọng số)	Có	Không
Bộ nhớ	có thể lớn (cả một "lớp")	nhỏ (chiều sâu)
Ứng dụng tiêu biểu	ngắn nhất, "cách nhau bao xa"	phát hiện chu trình, sắp xếp topo, thoát mê cung, đếm thành phần liên thông

Duyệt cây (thư mục, menu, tổ chức)

Cây không có chu trình nên **không cần** tập `daTham` :

```
class Nut(string ten, List<Nut> con)
{
    public string Ten { get; } = ten;
    public List<Nut> Con { get; } = con;

    public void In(int thut = 0)
    {
        Console.WriteLine($"{new string(' ', thut * 2)}{Ten}");
        foreach (var c in Con) c.In(thut + 1);        // DFS đệ quy
    }

    public int DemNut() => 1 + Con.Sum(c => c.DemNut());
    public int DoSau() => 1 + (Con.Count == 0 ? 0 : Con.Max(c => c.DoSau()));
}
```

Kết quả in ra dạng cây thụt lề — chính là cách các công cụ hiển thị cây thư mục. Với hệ thống file thật, dùng `Directory.EnumerateFileSystemEntries(path, "*", SearchOption.AllDirectories)` hoặc đệ quy trên `DirectoryInfo`. Duyệt cây theo từng tầng (in tất cả nút cấp 1, rồi cấp 2...) chính là BFS.

Lỗi thường gặp

- **Quên `daTham`** → vòng lặp vô hạn trên đồ thị có chu trình.
- Đánh dấu "đã thăm" **khi lấy ra** thay vì khi **đưa vào** hàng đợi (BFS) → một đỉnh bị đưa vào nhiều lần.
- Dùng DFS để tìm đường ngắn nhất (sai — có thể ra đường dài hơn).
- Đệ quy quá sâu → `StackOverflowException`: chuyển sang bản dùng `Stack<T>`.
- Quên đồ thị **có hướng/vô hướng**: vô hướng phải thêm cạnh hai chiều.
- Không xử lý đồ thị **không liên thông**: duyệt từ một đỉnh chỉ tới được thành phần của nó.

Bài tập

1. Đếm số **thành phần liên thông** của một đồ thị (lặp qua mọi đỉnh chưa thăm và chạy BFS/DFS).
2. Giải mê cung `char[,]` ("S" bắt đầu, "E" kết thúc, "#" tường) bằng BFS, in độ dài đường đi ngắn nhất.
3. Phát hiện chu trình trong đồ thị vô hướng bằng DFS (nhớ đỉnh cha).
4. Duyệt cây **theo tầng** (BFS) cho `Nut`, in mỗi tầng trên một dòng.
5. Viết hàm liệt kê mọi file `.cs` dưới một thư mục bằng DFS đệ quy trên `DirectoryInfo`.

Chương 30 — LINQ

Mục tiêu học

Sau chương này, bạn sẽ:

- Truy vấn dữ liệu bằng **LINQ** dạng method syntax và query syntax.
- Dùng thành thạo `Where`, `Select`, `OrderBy`, `GroupBy`, `Join`, `Any/All`, `Sum/Average/Max`, `Skip/Take`.
- Hiểu **thực thi trì hoãn** và khi nào phải `ToList()` / `ToArray()`.
- Tránh các bẫy hiệu năng và logic thường gặp.

Code mẫu: [code/ch30-linq/](#).

LINQ là gì?

LINQ (Language Integrated Query) là bộ phương thức mở rộng (Chương 19) trên `IEnumerable<T>` để **truy vấn** tập dữ liệu theo kiểu khai báo: nói *muốn gì* thay vì viết vòng lặp *làm thế nào*.

```
// Kiểu vòng lặp
var ketQua = new List<string>();
foreach (var s in sanPham)
    if (s.Gia < 1_000_000m) ketQua.Add(s.Ten);

// Kiểu LINQ
var ketQua = sanPham.Where(s => s.Gia < 1_000_000m).Select(s => s.Ten);
```

Cùng cú pháp dùng cho mảng, `List`, `Dictionary`, file, XML, và — quan trọng nhất — **CSDL qua Entity Framework Core** (Tập 2): truy vấn C# được dịch sang SQL. Cần `using System.Linq;` (đã có sẵn nhờ implicit usings).

Các toán tử cốt lõi

Dữ liệu mẫu: `record SanPham(string Ten, string NhomHang, decimal Gia, int TonKho)`.

Lọc và chiếu

```
sanPham.Where(s => s.Gia < 1_000_000m)           // lọc
sanPham.Select(s => s.Ten)                       // chiếu: lấy/biến đổi từng phầ
```

```
sanPham.Select(s => new { s.Ten, ThanhTien = s.Gia * s.TonKho }) // kiểu ẩn danh
sanPham.SelectMany(dh => dh.DongHang) // "làm phẳng" danh sách lồng r
```

Sắp xếp

```
sanPham.OrderBy(s => s.Gia)
sanPham.OrderByDescending(s => s.Gia).ThenBy(s => s.Ten)
```

Truy xuất phần tử và kiểm tra

Phương thức	Ý nghĩa
First() / FirstOrDefault()	phần tử đầu (ném lỗi / trả default nếu rỗng)
Single() / SingleOrDefault()	đúng một phần tử (lỗi nếu 0 hoặc nhiều hơn 1)
Last(), ElementAt(i)	cuối / theo chỉ số
Any() / Any(đk)	có phần tử (thoả điều kiện) nào không?
All(đk)	mọi phần tử thoả điều kiện?
Contains(x)	có chứa x?

Dùng `Any()` thay cho `Count() > 0` — dừng ngay khi thấy phần tử đầu tiên.

Tổng hợp

```
sanPham.Count(s => s.NhomHang == "Phu kien")
sanPham.Sum(s => s.Gia * s.TonKho)
sanPham.Average(s => s.Gia)
sanPham.Max(s => s.Gia); sanPham.MinBy(s => s.Gia) // MaxBy/MinBy trả về CẢ phần tử
new[] { "a", "b", "c" }.Aggregate((acc, x) => acc + "-" + x) // gộp tùy ý
```

Nhóm: GroupBy

```
foreach (var nhom in sanPham.GroupBy(s => s.NhomHang))
    Console.WriteLine($"{nhom.Key}: {nhom.Count()} sp, tổng tiền {nhom.Sum(s => s.TonKho)}
```

Mỗi `nhom` là một `IGrouping<TKey, T>`: có `.Key` và chính nó là dãy các phần tử thuộc nhóm.

Ghép: Join

```
sanPham.Join(nhomInfo, s => s.NhomHang, n => n.Ten, (s, n) => $"{s.Ten} ({n.MoTa})")
```

Ghép hai nguồn theo khoá chung (giống `INNER JOIN` của SQL). `Zip` ghép theo vị trí;
`Concat` / `Union` / `Intersect` / `Except` là phép toán tập hợp.

Phân trang và loại trùng

```
sanPham.OrderBy(s => s.Ten).Skip((trang - 1) * kíchThuoc).Take(kíchThuoc)
sanPham.Select(s => s.NhomHang).Distinct()
sanPham.Chunk(3) // chia thành các mảng 3 phần tử
```

Chuyển thành cấu trúc cụ thể

```
.ToList() .ToArray() .ToDictionary(s => s.Ten, s => s.Gia) .ToHashSet() .ToLoc
```

Query syntax

Cú pháp giống SQL, biên dịch ra đúng các lời gọi method ở trên:

```
var re = from s in sanPham
        where s.Gia < 1_000_000m
        orderby s.Gia descending
        select s.Ten;
```

Method syntax phổ biến hơn (chuỗi `.` linh hoạt, đủ toán tử); query syntax nổi bật khi có nhiều `join` / `let`. Bạn có thể trộn cả hai.

Thực thi trì hoãn — nhắc lại và mở rộng

Hầu hết toán tử LINQ (`Where`, `Select`, `OrderBy` ...) **không chạy ngay**; chúng chỉ tạo "công thức" và chạy khi bạn **duyet** (`foreach`, `ToList`, `Count`, `First` ...):

```
var danhSach = new List<int> { 1, 2, 3 };
var chan = danhSach.Where(x => x % 2 == 0); // chưa chạy
danhSach.Add(4);
Console.WriteLine(string.Join(",", chan)); // 2,4 – thấy cả số 4 thêm sau!

var daChot = danhSach.Where(x => x % 2 == 0).ToList(); // chạy NGAY, lưu kết quả
danhSach.Add(6);
Console.WriteLine(string.Join(",", daChot)); // 2,4 – không có 6
```

Hệ quả:

- Truy vấn duyệt **hiều lần** thì **chạy lại nhiều lần** (và với EF Core là **gọi CSDL nhiều lần**). Cần dùng nhiều lần hoặc cần "chụp ảnh" dữ liệu → `.ToList()` một lần.
- Biến bị "đóng" (closure) trong lambda được đọc **lúc chạy**, không phải lúc tạo truy vấn.
- Lỗi trong lambda chỉ hiện khi duyệt, không phải lúc gọi `Where`.

Bẫy và mẹo hiệu năng

- **Đừng `ToList()` quá sớm** khi còn muốn lọc thêm (đặc biệt với EF Core: lọc phải diễn ra ở CSDL, không phải trong bộ nhớ).
- `Count()` duyệt hết nếu nguồn là `IEnumerable` thuần — nếu chỉ hỏi "có không?" dùng `Any()`.
- `OrderBy` rồi `First()` là $O(n \log n)$; dùng `MinBy` / `MaxBy` là $O(n)$.
- `list.Where(...).Count()` → `list.Count(...)`.
- Với danh sách lớn, chuyển `Contains` trên `List` thành `HashSet` (Chương 26) khi lọc lặp.
- LINQ trên tập nhỏ rất tiện; đường ống nóng (hot path) hàng triệu phần tử hãy đo (`Stopwatch` / `BenchmarkDotNet`) trước khi tối ưu.
- Tránh tác dụng phụ (sửa biến ngoài, ghi log) bên trong `Where` / `Select` : vì chạy trì hoãn và có thể chạy nhiều lần.

Lỗi thường gặp

- `InvalidOperationException: Sequence contains no elements` — `First()` / `Max()` trên dãy rỗng (dùng `FirstOrDefault`, `DefaultIfEmpty`).
- `Single()` khi có nhiều hơn một kết quả.
- Duyệt cùng truy vấn nhiều lần rồi ngạc nhiên vì chạy lại.
- Sửa collection nguồn khi đang duyệt truy vấn lười.
- Nhầm `Select` (biến đổi) với `Where` (lọc).
- `NullReferenceException` với `FirstOrDefault()` không kiểm tra `null` (`?.`).

Bài tập

Với `List<SanPham>` của chương:

1. Liệt kê tên các sản phẩm còn hàng, giá giảm dần.
2. Tính tổng giá trị tồn kho từng nhóm hàng; tìm nhóm có giá trị cao nhất.
3. Tìm sản phẩm đắt thứ hai (`OrderByDescending` + `Skip(1)` + `First`).
4. Viết một truy vấn thống kê: mỗi nhóm có bao nhiêu sản phẩm hết hàng.
5. Viết lại đoạn code dùng vòng lặp `foreach` + `if` của bạn (từ chương trước) thành LINQ; so sánh độ dễ đọc.
6. Chứng minh thực thi trì hoãn bằng một truy vấn có `Console.WriteLine` bên trong `Select` — in ra khi nào?

Chương 31 — Delegate, event và lambda

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **delegate** là "kiểu của phương thức" và dùng `Func`, `Action`, `Predicate` có sẵn.
- Viết **lambda** và hiểu **closure**.
- Dùng **event** theo mẫu publisher/subscriber (`EventHandler<T>`).
- Truyền hành vi như một tham số — nền tảng của LINQ, async và [ASP.NET Core](#).

Code mẫu: [code/ch31-delegate-event-lambda/](#).

Delegate — biến chứa phương thức

Với `int x = 5`; bạn lưu một giá trị. **Delegate** cho phép lưu **một phương thức** vào biến, truyền đi và gọi sau:

```
delegate int PhepTinh(int a, int b);           // định nghĩa "hình dạng" phương thức

PhepTinh cong = (a, b) => a + b;
Console.WriteLine(cong(3, 4));                // 7

static int ApDung(int a, int b, PhepTinh f) => f(a, b);
ApDung(10, 5, (x, y) => x - y);               // 5 – truyền hành vi như tham số
```

Delegate là kiểu **an toàn**: chỉ nhận phương thức khớp chữ ký (số tham số, kiểu, kiểu trả về). Nó cho phép **tách "làm gì" khỏi "khi nào/ở đâu làm"** — ví dụ `Sort` không biết bạn so sánh theo tiêu chí nào, bạn truyền cho nó.

Func, Action, Predicate — delegate dựng sẵn

Hiếm khi cần tự khai báo `delegate`, vì .NET có sẵn dạng generic:

Kiểu	Chữ ký	Ví dụ
<code>Action / Action<T1,...></code>	không trả về (void)	<code>Action<string> inRa = s => Console.WriteLine(s);</code>
<code>Func<T1,...,TResult></code>	trả về <code>TResult</code> (tham số cuối là kiểu trả về)	<code>Func<int,int,int> cong = (a, b) => a + b;</code>

Kiểu	Chữ ký	Ví dụ
Predicate<T>	nhận T trả bool	Predicate<int> laChan = x => x % 2 == 0;

```
Func<int, int> binh = x => x * x;
Func<int, int, string> ghep = (a, b) => $"{a}-{b}";
```

Lambda expression

Lambda là cách viết **hàm vô danh** ngắn gọn: (tham số) => biểu_thức hoặc (tham số) => { khối_lệnh }.

```
x => x * x // một tham số, không cần ngoặc
(a, b) => a + b // nhiều tham số
() => Console.WriteLine("hi") // không tham số
(int a, int b) => { var t = a + b; return t * 2; } // khối lệnh
static x => x * 2 // static lambda: không cho phép capture, an toàn & r
```

Kiểu tham số thường được suy luận từ delegate đích. **Method group** cho phép truyền thẳng tên phương thức: `so.Select(binh)`, `list.ForEach(Console.WriteLine)`.

Truyền hành vi vào hàm

```
static IEnumerable<int> Loc(IEnumerable<int> ds, Predicate<int> dieuKien)
{
    foreach (var x in ds)
        if (dieuKien(x)) yield return x;
}

Loc(so, x => x % 2 == 0); // lọc chẵn
Loc(so, x => x > 3); // lọc > 3
```

Đây chính xác là cách `where` của LINQ được viết (Chương 30).

Closure — lambda "nhớ" biến bên ngoài

```
int heSo = 3;
Func<int, int> nhanHeSo = x => x * heSo;
nhanHeSo(5); // 15
heSo = 10;
nhanHeSo(5); // 50 – closure giữ THAM CHIẾU tới biến, không sao chép giá trị lúc tạ
```

Lambda **bắt (capture)** biến của phạm vi bao quanh và đọc/ghi giá trị *hiện tại lúc gọi*. Hàm trả về hàm:

```
Func<int, int> TaoBoCong(int n) => x => x + n;
var cong5 = TaoBoCong(5);
cong5(1);    // 6 - n = 5 vẫn "sống" sau khi TaoBoCong kết thúc
```

Cẩn thận: bắt biến **vòng lặp** (`for`) trong lambda chạy muộn có thể cho kết quả bất ngờ (mọi lambda cùng thấy giá trị cuối); `foreach` an toàn hơn vì mỗi vòng có biến riêng. Và closure giữ đối tượng "sống" lâu hơn dự kiến (rò rỉ bộ nhớ nếu lưu lambda mãi).

Multicast delegate

Một delegate có thể chứa **hiều** phương thức; gọi một lần chạy tất cả theo thứ tự đăng ký:

```
Action thongBao = () => Console.WriteLine("ghi log");
thongBao += () => Console.WriteLine("gui email");
thongBao();    // ghi log, gui email
```

`+=` thêm, `-=` bỏ. Với delegate có kiểu trả về, chỉ giá trị của phương thức **cuối** được trả về. Đây là nền của **event**.

Event — mẫu Publisher / Subscriber

Event là delegate được "bọc" để: bên ngoài chỉ được **đăng ký** (`+=`) / **huỷ** (`-=`); chỉ bản thân class phát mới được **kích hoạt** (`Invoke`). Nhờ đó bên phát không phụ thuộc vào bên nghe (giảm ràng buộc — Observer pattern, Chương 41).

```
class DoUongEventArgs(string ten) : EventArgs
{
    public string TenDoUong { get; } = ten;
}

class MayPha
{
    public event EventHandler<DoUongEventArgs>? SanSang;    // publisher

    public void Pha(string ten)
    {
        Console.WriteLine($"Dang pha {ten}...");
        SanSang?.Invoke(this, new DoUongEventArgs(ten));    // ?. : an toàn khi chưa ai
    }
}
```

```
var may = new MayPha();
may.SanSang += (sender, e) => Console.WriteLine($"[Khach A] nhan: {e.TenDoUong}"); /
may.SanSang += DocThongBao;
may.Pha("ca phe sua");
may.SanSang -= DocThongBao; // huỷ đăng ký
```

Quy ước chuẩn: `EventHandler<TEventArgs>` (`object? sender, TEventArgs e`); tên sự kiện là động từ (`SanSang` , `Clicked`); phương thức kích hoạt `OnXxx` .

Điểm cần nhớ: **hãy huỷ đăng ký (-=)** khi subscriber không còn dùng — nếu không publisher vẫn giữ tham chiếu, ngăn GC dọn subscriber ("memory leak do event" rất phổ biến). Trong [ASP.NET](#) Core bạn ít tự viết event, nhưng sẽ gặp mẫu tương tự trong thông báo, `IObservable` , `MediatR` (Tập 3).

Khi nào dùng cái nào?

Tình huống	Chọn
Truyền một hành vi vào hàm (lọc, sắp xếp, callback)	Func/Action/Predicate + lambda
Nhiều bên cần được báo khi có chuyện xảy ra	event
Hợp đồng nhiều phương thức có liên quan	interface (Chương 18)
Một hành vi đơn lẻ, cần tên rõ nghĩa	delegate đặt tên (PhepTinh) hoặc interface nhỏ

Lỗi thường gặp

- `NullReferenceException` khi gọi `SanSang(this, e)` mà chưa ai đăng ký — dùng `SanSang?.Invoke(...)` .
- Quên huỷ đăng ký event → rò rỉ bộ nhớ, handler chạy cả khi đối tượng đã "chết" logic.
- Mong đợi closure sao chép giá trị lúc tạo (thực ra đọc lúc gọi).
- Nhầm `Func<int, string>` (nhận int, trả string) với `Func<string, int>` .
- Lambda quá dài, phức tạp — tách thành phương thức có tên.
- Dùng `async void` cho event handler mà không bắt lỗi (Chương 33).

Bài tập

1. Viết `ApDung<T>(IEnumerable<T> ds, Action<T> hanhDong)` tự cài đặt `ForEach` .
2. Viết `Func<double, double> TaoHamNhan(double heSo)` trả về hàm nhân với hệ số; kiểm chứng closure.

3. Tạo class `NutBam` có event `DuocBam` và hai subscriber; huỷ một subscriber rồi bấm lại.
4. Cài `SapXep<T>(List<T> ds, Func<T, T, int> soSanh)` bằng thuật toán sắp xếp chèn đơn giản.
5. Giải thích vì sao đoạn sau in `3 3 3` rồi sửa cho in `0 1 2`:

```
var fs = new List<Action>();  
for (int i = 0; i < 3; i++) fs.Add(() => Console.Write(i + " "));
```

Chương 32 — Record, pattern matching và cú pháp C# hiện đại

Mục tiêu học

Sau chương này, bạn sẽ:

- Tổng hợp và dùng thành thạo các tính năng C# hiện đại: `record`, `init` / `required`, **primary constructor**, **collection expression**, **raw string literal**.
- Dùng **pattern matching** nâng cao: `property`, `relational`, `logical`, **list pattern**.
- Viết code ngắn hơn mà vẫn rõ nghĩa, và biết khi nào **không** nên dùng.

Code mẫu: [code/ch32-csharp-hien-dai/](#).

Nhiều tính năng đã xuất hiện rải rác ở các chương trước; chương này gom lại thành "bộ đồ nghề" để đọc và viết code kiểu .NET hiện đại (thứ bạn sẽ gặp khắp nơi trong [ASP.NET Core](#)).

Record và with

```
record SinhVien(string Ten, int Tuoi, string Lop);

var a = new SinhVien("An", 20, "CNTT");
var b = a with { Tuoi = 21 };           // sao chép, đổi một phần → bản mới
a == b;                                // false: so sánh theo giá trị các thuộc tính
var (ten, tuoi, _) = a;                 // deconstruct
```

Ôn (Chương 20): `record` tự sinh `Equals`, `GetHashCode`, `ToString`, `Deconstruct` và thuộc tính `init`. Dùng cho dữ liệu bất biến: DTO, thông điệp, giá trị nghiệp vụ.

init và required

```
class SanPham
{
    public required string Ten { get; init; }
    public required decimal Gia { get; init; }
}

var sp = new SanPham { Ten = "Chuot", Gia = 150_000m };
// new SanPham { Ten = "x" }; // LỖI BIÊN DỊCH: thiếu Gia
```

`required` ép người dùng class gán giá trị lúc khởi tạo; `init` cho phép gán chỉ một lần. Kết hợp với nullable (Chương 23) loại bỏ hàng loạt lỗi "quên gán".

Primary constructor (C# 12)

Thêm số constructor đặt ngay sau tên class, dùng được trong toàn thân class:

```
class KhachHang(string ten, string email)
{
    public string Ten { get; } = ten; // gán vào property
    public string ChaoHoi() => $"Xin chào {Ten} <{email}>"; // dùng trực tiếp trong ph
}
```

Rất hợp cho **dependency injection** trong [ASP.NET Core](#): `class DonHangService(IRepository repo, ILogger<DonHangService> log) { ... }`.

Lưu ý: thêm số primary constructor của `class / struct` là **biến bị bắt (captured)**, không tự thành property/field public (khác `record`). Muốn chỉ đọc, nên gán vào property/ `readonly` field.

Collection expression (C# 12)

```
int[] x = [1, 2, 3];
List<int> gop = [0, .. x, .. [4, 5], 99]; // ".." trải (spread) một dãy vào
IEnumerable<string> r = ["a", "b"];
List<int> rong = [];
```

Một cú pháp thống nhất cho mảng, `List`, `HashSet`, `Span` ... Kiểu đích quyết định kết quả.

Raw string literal (C# 11)

```
string json = $$$"""
{
    "ten": "{{ten2}}",
    "tuoi": 20
}
""";
```

Đặt trong `"""..."""`, thoải mái dùng dấu `"` và `\` mà không escape; nhiều dòng; thật lè của dấu đóng `"""` được tự cắt. `$$$"""` nghĩa là dấu chèn giá trị là **hai** ngoặc `{{ }}` (để một ngoặc `{ }` đơn của JSON không bị hiểu nhầm).

Pattern matching nâng cao

Bạn đã biết mẫu hằng, kiểu, `when` (Chương 7). Đây đủ hơn:

```
static string MoTa(object? o) => o switch
{
    null => "null",
    int n and > 0 => $"so duong {n}",           // kiểu + quan hệ + log
    int n => $"so khong duong {n}",
    string { Length: > 3 } s => $"chuoi dai '{s}'",       // property pattern
    string s => $"chuoi ngan '{s}'",
    int[] [var dau, .., var cuoi] => $"mang tu {dau} den {cuoi}", // list pattern
    SinhVien { Tuoi: < 20, Lop: var lop } => $"SV tre lop {lop}", // property patter
    SinhVien sv => $"SV {sv.Ten}",
    _ => $"khac: {o.GetType().Name}",
};
```

Mẫu	Ví dụ	Ý nghĩa
Relational	< 0, >= 15	so sánh với hằng
Logical	and, or, not	kết hợp mẫu
Property	{ Length: > 3 }	kiểm tra thuộc tính
Positional	(var x, var y)	dùng Deconstruct
List	[1, .., var cuoi], [_, _]	khớp cấu trúc dãy (.. = "phần còn lại")
var	var x	luôn khớp, gán vào biến

```
static string PhanLoaiNhietDo(double t) => t switch
{
    < 0 => "dong bang",
    >= 0 and < 15 => "lanh",
    >= 15 and < 30 => "de chiu",
    >= 30 => "nong",
    double.NaN => "loi",
};
```

Trình biên dịch kiểm tra **tính đầy đủ**: quên trường hợp sẽ cảnh báo. Pattern matching hợp khi dữ liệu có nhiều dạng (kiểu, cấu trúc, khoảng giá trị); nếu hành vi nên thuộc về chính đối tượng, đa hình (Chương 17) vẫn tốt hơn.

Các cú pháp ngắn gọn khác

```
int[] mang = [10, 20, 30, 40, 50];
mang[^1];           // 50 – chỉ số từ cuối
mang[1..3];         // [20, 30] – lát cắt
s ??= "mac dinh";    // gán nếu null
if (s is not null and { Length: > 3 }) ...
(int min, int max) = (list.Min(), list.Max()); // tuple deconstruct
```

- **Expression-bodied members:** `public string Ten => ten; .`
- **Target-typed new:** `List<int> ds = new(); , Point p = new(1, 2); .`
- **static lambda** (`static n => n * 2`): cấm bắt biến ngoài — an toàn và không cấp phát closure.
- **Local functions:** hàm lồng trong hàm, hợp cho trợ giúp cục bộ.
- **File-scoped namespace** (`namespace X;`), **global usings**, **top-level statements** (Chương 15, 3).

Còn gì mới?

Mỗi bản .NET đi cùng một bản C# mới, thường thêm vài đường cú pháp. Từ C# 13/14 (đi cùng .NET 9/10) có thêm các tính năng như `params` cho collection, từ khoá `field` trong property (thân `get/set` dùng backing field tự sinh), **extension members** (khai báo cả property/toán tử mở rộng), null-conditional assignment (`a?.B = 1`)... Xem mục "What's new in C#" trên learn.microsoft.com khi cập nhật lên phiên bản mới — luôn kiểm tra phiên bản ngôn ngữ mà project của bạn đang dùng (`<LangVersion>`).

Nguyên tắc dùng "cú pháp mới"

1. **Ưu tiên độ rõ ràng**, không phải độ ngắn. Chuỗi pattern lồng ba tầng khó đọc hơn hai `if`.
2. Đồng đội/CI phải dùng cùng phiên bản SDK, nếu không code sẽ không biên dịch.
3. Dùng công cụ: IDE gợi ý "convert to pattern matching/collection expression" và `.editorconfig` thống nhất phong cách.
4. Đừng viết lại hàng loạt code cũ chỉ để "cho mới" — thay khi sửa tới.

Lỗi thường gặp

- Nhầm tham số primary constructor với property (không tự sinh property cho `class`).
- Dùng `record` cho entity có danh tính và trạng thái thay đổi (dễ gây nhầm khi `==` so sánh theo giá trị).
- Nhánh `switch` tổng quát đặt trước nhánh cụ thể → lỗi biên dịch "nhánh không bao giờ đạt".
- `with` chỉ sao chép **nông**: thuộc tính kiểu tham chiếu (`List`, `class`) vẫn dùng chung.

- Quên `$$""` khi chuỗi JSON có `{ }` nhưng vẫn muốn nội suy.
- Dùng collection expression với kiểu không hỗ trợ.

Bài tập

1. Viết lại một vòng `foreach` + `if/else if` đã có thành switch expression với property pattern.
2. Dùng list pattern viết hàm phân tích lệnh: `["go", var huong]`, `["take", var vat]`, `["quit"]`, còn lại là "không hiểu".
3. Viết `record TienTe(decimal SoTien, string Loai)` với phép cộng (`operator +`) chỉ cho cùng loại tiền.
4. Tạo chuỗi HTML nhiều dòng có nội suy bằng raw string.
5. Viết `class DonHangService(IRepo repo, ILogger log)` (interface tự định nghĩa) dùng primary constructor.

Chương 33 — Lập trình bất đồng bộ async/await

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu vì sao cần **bất đồng bộ** và nó khác **đa luồng** thế nào.
- Viết và gọi phương thức `async Task / Task<T>` đúng cách.
- Chạy nhiều việc **song song** với `Task.WhenAll / WhenAny`, xử lý lỗi, **hủy** bằng `CancellationToken`.
- Dùng `IAsyncEnumerable<T>` và biết cách tránh các lỗi kinh điển (`async void`, `.Result`, `deadlock`).

Code mẫu: [code/ch33-async-await/](#).

Vấn đề: chờ đợi mà không lãng phí

Nhiều thao tác phần lớn thời gian là **chờ**: đọc file, gọi API, truy vấn CSDL. Nếu thread đứng chờ (**block**), nó không làm được việc gì khác:

- Ứng dụng giao diện: màn hình **đơ**.
- Web server: mỗi request chiếm một thread trong lúc chờ → hết thread, không phục vụ thêm được ai.

Bất đồng bộ (asynchronous) cho phép thread **rảnh tay** trong lúc chờ, và tiếp tục xử lý khi kết quả sẵn sàng. Trong [ASP.NET Core](#) (Tập 2), `async` là mặc định cho mọi thao tác I/O, nên chương này rất quan trọng.

Async ≠ đa luồng. Với I/O (chờ mạng, đĩa), **không có thread nào bị chiếm** trong lúc chờ. Đa luồng (Chương 34) là chạy *tính toán* song song trên nhiều thread/core. Hai công cụ cho hai bài toán khác nhau.

Task, async và await

- `Task` đại diện một công việc sẽ hoàn thành trong tương lai (không kết quả); `Task<T>` có kết quả kiểu `T`.
- `async` đánh dấu phương thức được phép dùng `await`.
- `await` "chờ mà không chặn": nếu Task chưa xong, phương thức **trả quyền điều khiển** về nơi gọi; khi xong, phần còn lại của phương thức **chạy tiếp** từ chỗ dừng.

```
static async Task<string> TaiDuLieuAsync(string ten, int ms)
{
    await Task.Delay(ms);    // giả lập chờ I/O; KHÔNG block thread
    return ten;
}

string kq = await TaiDuLieuAsync("A", 300);
```

Quy ước: tên phương thức bất đồng bộ kết thúc bằng **Async** ; kiểu trả về là **Task** , **Task<T>** (hoặc **ValueTask<T>**). **Main** /top-level statements có thể dùng **await** trực tiếp.

Bên trong, trình biên dịch biến phương thức **async** thành một **máy trạng thái**; bạn viết code tuần tự dễ đọc mà không phải lồng callback.

Chạy tuần tự hay song song?

```
// Tuần tự: ~600 ms – đợi A xong mới bắt đầu B
await TaiDuLieuAsync("A", 300);
await TaiDuLieuAsync("B", 300);

// Song song: ~300 ms – bắt đầu cả hai TRƯỚC, rồi mới đợi
var tA = TaiDuLieuAsync("A", 300);
var tB = TaiDuLieuAsync("B", 300);
await Task.WhenAll(tA, tB);
```

Điểm mấu chốt: **gọi phương thức async bắt đầu Task ngay**; **await** mới là lúc chờ. Muốn song song, đừng **await** ngay từng cái. **await Task.WhenAll(...)** chờ tất cả (và nếu Task trả **T** , kết quả là mảng **T[]**); **await Task.WhenAny(...)** chờ **cái đầu tiên** xong (dùng cho timeout, "lấy nguồn nhanh nhất").

Với danh sách: **var kq = await Task.WhenAll(ds.Select(x => XuLyAsync(x)));** . Nhớ giới hạn số lượng đồng thời (**SemaphoreSlim** , **Parallel.ForEachAsync**) nếu danh sách lớn.

Xử lý lỗi

Ngoại lệ trong phương thức **async** được **lưu vào Task** và ném lại khi bạn **await** :

```
try { await LoiAsync(); }
catch (InvalidOperationException e) { ... }    // bắt bình thường như code đồng bộ
```

Với **Task.WhenAll** , **await** ném ngoại lệ **đầu tiên**; các lỗi còn lại nằm trong **task.Exception** (**AggregateException**). Nếu không bao giờ **await** một Task lỗi, lỗi bị "nuốt" lặng lẽ (fire-and-forget) — luôn **await** hoặc xử lý kết quả.

Hủy việc: CancellationToken

Việc chạy lâu phải cho phép **hủy** (người dùng bấm Cancel, request bị ngắt, hết timeout):

```
using var cts = new CancellationTokenSource(TimeSpan.FromMilliseconds(250)); // tự hủy
try { await ViiecDaiAsync(cts.Token); }
catch (OperationCanceledException) { Console.WriteLine("bi huy"); }

static async Task ViiecDaiAsync(CancellationToken ct)
{
    for (int i = 0; i < 10; i++)
    {
        ct.ThrowIfCancellationRequested(); // tự kiểm tra
        await Task.Delay(100, ct);        // hầu hết API async nhận token
    }
}
```

Quy ước: phương thức async có thể mất thời gian nên nhận `CancellationToken ct = default` là **tham số cuối** và truyền nó xuống mọi lời gọi async bên trong. Trong [ASP.NET Core](#), framework cung cấp sẵn token gắn với request.

IAsyncEnumerable<T> — luồng bất đồng bộ

Sinh dãy giá trị mà mỗi phần tử cần chờ:

```
static async IAsyncEnumerable<int> SinhSoAsync(int n)
{
    for (int i = 1; i <= n; i++)
    {
        await Task.Delay(50);
        yield return i;
    }
}

await foreach (var so in SinhSoAsync(3)) Console.WriteLine(so);
```

Hợp cho đọc dữ liệu từng phần (dòng file lớn, kết quả CSDL, dữ liệu phân trang). `await using` giải phóng tài nguyên `IAsyncDisposable` (Chương 21).

Task.Run — việc CPU nặng

`await` giúp với I/O. Với **tính toán nặng** trên luồng giao diện, dùng `Task.Run` để đẩy sang thread pool:

```
long tong = await Task.Run(() => TinhToanNang());
```

Trong [ASP.NET](#) Core **không** bọc code đồng bộ bằng `Task.Run` chỉ để "trông có vẻ async" — nó chỉ chiếm thread khác thay vì thread hiện tại, không đem lại lợi ích.

Quy tắc vàng

1. **"Async all the way"**: đã async thì async xuyên suốt, từ trên xuống dưới; đừng trộn.
2. **Không chặn** Task bằng `.Result`, `.Wait()`, `.GetAwaiter().GetResult()` — có thể gây **deadlock** (nhất là ở UI/ ngữ cảnh cũ) và làm mất lợi ích. Hãy `await`.
3. **Tránh async void** — chỉ dành cho event handler; lỗi trong `async void` không bắt được và làm sập tiến trình. Dùng `async Task`.
4. Phương thức chỉ chuyển tiếp Task mà không cần `await` có thể trả thẳng Task (`return repo.GetAsync(id);`) — nhưng nếu có `using / try/catch` bao quanh thì phải `await`.
5. Trong thư viện dùng chung, cân nhắc `ConfigureAwait(false)`; trong [ASP.NET](#) Core không cần (không có `SynchronizationContext`).
6. **Luôn truyền** `CancellationToken`.

Lỗi thường gặp

- Quên `await` → Task chạy nhưng code phía sau chạy tiếp mà không chờ; lỗi bị bỏ sót (cảnh báo CS4014).
- `.Result` / `.Wait()` → deadlock hoặc chặn thread.
- `async void` ngoài event handler.
- Hai `await` liên tiếp khi lẽ ra chạy song song (mất hiệu năng).
- Bắt Exception chung chung nuốt cả `OperationCanceledException` — bắt riêng và bỏ qua khi huỷ hợp lệ.
- Lạm dụng `Task.Run` trong web.
- Quên `Dispose` `CancellationTokenSource` (`using var cts`).

Bài tập

1. Viết `Task<string> DocFileAsync(string path)` dùng `File.ReadAllTextAsync`; gọi 3 file **song song**.
2. Đo thời gian của 5 lời gọi `Task.Delay(200)` chạy tuần tự vs `Task.WhenAll`.
3. Thêm timeout 1 giây cho một việc chạy lâu bằng `CancellationTokenSource` hoặc `Task.WaitAsync(TimeSpan)`.
4. Dùng `Task.WhenAny` để "chạy đua" hai nguồn dữ liệu, lấy kết quả của nguồn nhanh hơn và huỷ nguồn còn lại.
5. Cố tình gọi `.Result` trên một Task trong ứng dụng console và giải thích vì sao ở đó không deadlock nhưng vẫn là thói quen xấu.

Chương 34 — Đa luồng và đồng bộ hoá

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **thread**, **thread pool** và khi nào cần chạy song song.
- Nhận ra và sửa **race condition** bằng `lock`, `Interlocked`, collection an toàn đa luồng.
- Dùng `Parallel`, `SemaphoreSlim` và `Channel<T>` (mô hình producer/consumer).
- Hiểu **deadlock** và cách tránh; phân biệt với `async/await`.

Code mẫu: [code/ch34-da-luong/](#).

Thread và thread pool

Một **thread** là một luồng thực thi. Chương trình luôn có một thread chính; bạn có thể tạo thêm để làm nhiều việc *cùng lúc* (trên nhiều lõi CPU):

```
var t = new Thread(() => Console.WriteLine("chay tren thread phu"));
t.Start();
t.Join();    // chờ thread kết thúc
```

Tạo thread thủ công đắt (mỗi thread tốn ~1 MB ngăn xếp) và khó quản lý. Thực tế dùng **thread pool** — một nhóm thread tái sử dụng do .NET quản lý — thông qua `Task.Run`, `Parallel`, `async/await`. Hiếm khi bạn cần `new Thread` (chỉ khi cần thread chạy lâu, ưu tiên riêng).

Chọn công cụ:

Bài toán	Dùng
Chờ I/O (mạng, đĩa, CSDL)	<code>async/await</code> (Chương 33)
Tính toán nặng, chia đều trên nhiều lõi	<code>Parallel.For/ForEach</code> , <code>PLINQ (.AsParallel())</code>
Việc nền một lần	<code>Task.Run</code>
Luồng dữ liệu producer → consumer	<code>Channel<T></code>

Race condition — vấn đề gốc

Khi **nhiều thread cùng đọc/ghi một dữ liệu chung** mà không phối hợp, kết quả phụ thuộc vào thứ tự "may rủi" giữa các thread:

```
int dem = 0;
Parallel.For(0, 100_000, _ => dem++);
Console.WriteLine(dem); // mong 100000, nhưng thường nhỏ hơn (vd 63.421) – và mỗi lần
```

Vì `dem++` không "nguyên tử": nó gồm ba bước **đọc – cộng – ghi**. Hai thread có thể cùng đọc 5, cùng ghi 6 → mất một lần đếm. Lỗi này **không nhất quán**, khó tái hiện và hay biến mất khi debug — nên phải phòng ngừa chứ không thể "thử xem có đúng không".

Giải pháp 1: lock

`lock` chỉ cho **một thread tại một thời điểm** vào đoạn code găng (critical section):

```
object khoa = new();
Parallel.For(0, 100_000, _ => { lock (khoa) { dem++; } }); // luôn đúng: 100000
```

Quy tắc dùng `lock` :

- Khoá trên một **đối tượng `private readonly`** riêng (hoặc `System.Threading.Lock` trong .NET 9+); **không** khoá trên `this`, `typeof(...)`, hay chuỗi — code khác có thể khoá cùng đối tượng và gây deadlock.
- Giữ khoá **càng ngắn càng tốt**; không gọi code lạ, không `await` bên trong `lock` (dùng `SemaphoreSlim` cho tình huống async).

Giải pháp 2: Interlocked

Phép nguyên tử, không cần khoá, nhanh hơn cho thao tác đơn giản trên biến số:

```
Parallel.For(0, 100_000, _ => Interlocked.Increment(ref dem));
Interlocked.Add(ref tong, 5);
Interlocked.CompareExchange(ref x, giaTriMoi, giaTriMongDoi);
```

Giải pháp 3: collection an toàn đa luồng

`List<T>`, `Dictionary<K,V>` **không** an toàn khi nhiều thread cùng ghi. Dùng `System.Collections.Concurrent` :

```
var dem = new ConcurrentDictionary<string, int>();
Parallel.ForEach(tu, t => dem.AddOrUpdate(t, 1, (_, cu) => cu + 1));
```

Có sẵn `ConcurrentQueue<T>`, `ConcurrentBag<T>`, `ConcurrentStack<T>`, `BlockingCollection<T>`. Lưu ý: từng thao tác an toàn, nhưng **chuỗi hai thao tác** (kiểm tra rồi

thêm) vẫn có thể bị xen — dùng phương thức gộp như `GetOrAdd`, `AddOrUpdate`.

Cách tốt nhất tránh race: **không chia sẻ trạng thái có thể thay đổi**. Dùng dữ liệu bất biến (`record`, Chương 20) và mỗi thread giữ dữ liệu riêng rồi gộp kết quả cuối cùng.

Parallel và PLINQ

```
// Mỗi thread cộng dồn vào biến cục bộ, gộp một lần cuối => ít tranh chấp
long tongChan = 0;
Parallel.For(1, 1_000_001, () => 0L,
    (i, _, cucBo) => i % 2 == 0 ? cucBo + i : cucBo,
    cucBo => Interlocked.Add(ref tongChan, cucBo));

var kq = so.AsParallel().Where(LaNguyenTo).ToList(); // PLINQ
await Parallel.ForEachAsync(urls, opts, async (url, ct) => { ... }); // song song +
```

Song song hoá chỉ đáng khi **mỗi đơn vị công việc đủ nặng** và độc lập; với việc nhỏ, chi phí phối hợp lớn hơn lợi ích (thậm chí chậm hơn). Luôn **đo** trước và sau.

SemaphoreSlim — giới hạn số việc đồng thời

Dùng khi muốn chỉ cho phép tối đa N việc chạy cùng lúc (giới hạn kết nối, gọi API có rate limit). Hỗ trợ `await`:

```
var cho = new SemaphoreSlim(2);
await cho.WaitAsync();
try { await LamViecAsync(); }
finally { cho.Release(); } // LUÔN Release trong finally
```

Channel<T> — producer / consumer

Một bên **sản xuất** dữ liệu, bên kia **tiêu thụ**, ngăn cách bởi kênh có bộ đệm, an toàn đa luồng và hỗ trợ async:

```
var kenh = Channel.CreateBounded<int>(3); // đệm tối đa 3 phần tử: "áp suất ngược"

var sanXuat = Task.Run(async () =>
{
    for (int i = 1; i <= 5; i++) await kenh.Writer.WriteAsync(i); // đầy thì chờ
    kenh.Writer.Complete(); // báo hết dữ liệu
});
var tieuThu = Task.Run(async () =>
{

```

```
await foreach (var x in kênh.Reader.ReadAllAsync()) Console.WriteLine(x);
});
await Task.WhenAll(sanXuat, tieuThu);
```

Đây là mô hình nền cho hàng đợi công việc, xử lý sự kiện, background service trong [ASP.NET Core](#).

Deadlock

Deadlock = hai (hay nhiều) thread cùng chờ nhau mãi, không ai chạy được:

```
// Thread 1: lock(A) { lock(B) { ... } }
// Thread 2: lock(B) { lock(A) { ... } } // → mỗi bên giữ một khoá và chờ khoá kia
```

Cách tránh: (1) luôn khoá theo **cùng một thứ tự**; (2) giữ khoá ngắn, không gọi code ngoài khi đang giữ; (3) dùng `Monitor.TryEnter` với timeout; (4) thiết kế ít khoá bằng cách tránh chia sẻ trạng thái. Deadlock kinh điển thứ hai: `.Result` / `.Wait()` trên Task trong ngữ cảnh đồng bộ (Chương 33).

Ghi nhớ

- Chạy song song là **tối ưu hiệu năng có giá**: thêm độ phức tạp và lỗi khó bắt. Chỉ dùng khi đã đo và thật sự cần.
- **async cho I/O, Parallel cho CPU**.
- Thứ tự ưu tiên khi phải chia sẻ dữ liệu: bất biến → không chia sẻ → collection concurrent → `Interlocked` → `lock`.
- Kiểm thử đa luồng rất khó; thiết kế để đoạn dùng chung nhỏ và dễ kiểm chứng bằng mắt.

Lỗi thường gặp

- Nhiều thread ghi `List<T>` / `Dictionary` không khoá → dữ liệu hỏng, `IndexOutOfRangeException`, vòng lặp vô hạn.
- `lock(this)` / `lock("chuoi")`.
- `await` bên trong `lock` (lỗi biên dịch) — dùng `SemaphoreSlim`.
- Quên `Release()` semaphore trong `finally` → treo vĩnh viễn.
- Tưởng `volatile` / `static` là đủ an toàn.
- Dùng `Parallel.For` cho việc quá nhỏ — chậm hơn vòng `for`.
- Không gọi `Writer.Complete()` khiến consumer đợi mãi.

Bài tập

1. Chạy đoạn race condition 10 lần và ghi lại kết quả; sửa bằng `lock`, rồi bằng `Interlocked`; so sánh tốc độ.
2. Dùng `Parallel.ForEach` + `ConcurrentDictionary` đếm tần suất từ trong nhiều chuỗi.
3. Viết chương trình hai thread gây **deadlock** có chủ đích, rồi sửa bằng thứ tự khoá cố định.
4. Dùng `Channel` mô phỏng nhà bếp: 1 đầu bếp (producer) và 3 người phục vụ (consumer) xử lý đơn.
5. Đo thời gian đếm số nguyên tố $\leq 5.000.000$ bằng vòng `for`, `Parallel.For`, và PLINQ.

Chương 35 — File I/O

Mục tiêu học

Sau chương này, bạn sẽ:

- Đọc/ghi file văn bản và nhị phân bằng `File`, `StreamReader/StreamWriter`, `FileStream`.
- Làm việc với đường dẫn và thư mục một cách an toàn, **đa nền tảng** (`Path`, `Directory`).
- Biết chọn giữa đọc **cả file** và đọc **từng dòng**; đọc/ghi **bất đồng bộ**.
- Bắt và xử lý lỗi I/O; đảm bảo file luôn được đóng.

Code mẫu: [code/ch35-file-io/](#).

Chương trình mẫu làm việc trong thư mục tạm của hệ điều hành (`Path.GetTempPath()`) và **tự dọn dẹp** khi xong, nên chạy được nhiều lần mà không làm bừa máy.

Các lớp chính (namespace `System.IO`)

Lớp	Vai trò
<code>File</code>	thao tác nhanh trên một file (static): đọc, ghi, sao chép, xóa...
<code>Directory / DirectoryInfo</code>	thư mục: tạo, liệt kê, xóa
<code>Path</code>	xử lý chuỗi đường dẫn (ghép, tách, đuôi file)
<code>StreamReader / StreamWriter</code>	đọc/ghi văn bản theo luồng
<code>FileStream</code>	đọc/ghi byte ở mức thấp
<code>FileInfo</code>	thông tin một file (kích thước, ngày sửa...)

Đọc và ghi toàn bộ file (file nhỏ)

```
File.WriteAllText(path, "Dong 1\nDong 2\n", Encoding.UTF8); // ghi đè
string noiDung = File.ReadAllText(path); // đọc cả file thành n
File.AppendAllText(path, "them dong\n"); // ghi nối vào cuối
string[] dong = File.ReadAllLines(path); // mảng các dòng
File.WriteAllLines(path, ["a", "b"]);
byte[] data = File.ReadAllBytes(path);
```

Tiện, nhưng **nạp cả file vào bộ nhớ** → chỉ nên dùng cho file nhỏ (vài MB). **Luôn chỉ rõ mã hoá** (`Encoding.UTF8`) khi ghi văn bản tiếng Việt để tránh vỡ dấu ở máy khác (mặc định .NET Core dùng UTF-8, nhưng nên tường minh khi trao đổi dữ liệu).

Đọc từng dòng, xử lý file lớn

```
foreach (var dong in File.ReadLines(path))    // LƯỖI: đọc từng dòng theo yêu cầu
    if (dong.Contains("ERROR")) dem++;
```

`File.ReadLines` trả `IEnumerable<string>` đọc dần (Chương 27) — dùng được cho file nhiều GB. (`ReadAllLines` thì đọc hết vào mảng.) Vì lười, file **được mở suốt lúc duyệt**; và duyệt hai lần là đọc file hai lần.

StreamReader / StreamWriter và using

Khi cần kiểm soát chi tiết (ghi dần, đọc theo điều kiện):

```
using (var ghi = new StreamWriter(path, append: false, Encoding.UTF8))
{
    ghi.WriteLine("Bat dau");
    ghi.Write("Ket thuc");
} // kết thúc khối using: tự Flush + đóng file

using var doc = new StreamReader(path);
string? line;
while ((line = doc.ReadLine()) is not null)
    Console.WriteLine(line);
```

Luôn bọc trong `using` (Chương 21): file là tài nguyên hệ điều hành; nếu không đóng, nó bị khoá (process khác không mở được), dữ liệu có thể chưa ghi hết (chưa `Flush`), và rò rỉ handle. `ReadLine()` trả `null` khi hết file.

Đường dẫn: Path

```
Path.Combine("a", "b", "c.txt");           // "a\b\c.txt" (Windows) hay "a/b/c.t
Path.GetFileName(p);                       // "thang-3.xlsx"
Path.GetFileNameWithoutExtension(p);       // "thang-3"
Path.GetExtension(p);                     // ".xlsx"
Path.GetDirectoryName(p);
Path.GetTempPath(); Path.GetRelativePath(goc, f); Path.GetFullPath("x");
```

Đừng tự nối chuỗi đường dẫn bằng `+` `"\\"` + hay `"/"` — dùng `Path.Combine` để chạy đúng trên mọi hệ điều hành. Chuỗi đường dẫn kiểu Windows (`C:\...`) chỉ phân tích đúng trên Windows; trên Linux `\` không phải dấu phân cách.

Bảo mật: không ghép trực tiếp tên file do người dùng cung cấp vào đường dẫn (`../../etc/passwd` — *path traversal*). Kiểm tra bằng `Path.GetFileName(input)` hoặc so sánh `Path.GetFullPath` với thư mục gốc cho phép.

Thư mục và thao tác hệ thống tập tin

```
Directory.CreateDirectory(dir); // tạo (cả các cấp cha; không lỗi r
File.Copy(nguồn, đích, overwrite: true);
File.Move(nguồn, đích);
File.Delete(path); // không lỗi nếu file không tồn tại
Directory.Delete(dir, recursive: true); // xoá cả nội dung
File.Exists(path); Directory.Exists(dir);

foreach (var f in Directory.EnumerateFiles(dir, "*.cs", SearchOption.AllDirectories))
    Console.WriteLine(new FileInfo(f).Length);
```

Ưu tiên `Enumerate...` (lười) hơn `Get...` (dựng cả mảng) với thư mục lớn.

Bất đồng bộ

```
await File.WriteAllTextAsync(path, "Noi dung");
string s = await File.ReadAllTextAsync(path);
await using var fs = new FileStream(path, FileMode.Open, FileAccess.Read, FileShare.Re
```

Trong ứng dụng web/giao diện, dùng phiên bản `Async` để không chặn thread (Chương 33).

FileStream — byte và truy cập ngẫu nhiên

```
await File.WriteAllBytesAsync(path, [1, 2, 3, 255]);

using var fs = new FileStream(path, FileMode.OpenOrCreate, FileAccess.ReadWrite);
fs.Seek(2, SeekOrigin.Begin); // nhảy tới vị trí
int b = fs.ReadByte();
```

`FileMode` (`Create`, `Open`, `OpenOrCreate`, `Append`, `Truncate...`), `FileAccess`, `FileShare` điều khiển cách mở. Ảnh, video, file nén... đều là byte; xử lý qua `Stream` (kể cả `MemoryStream`, `GZipStream`, `NetworkStream` có cùng giao diện `Stream`).

Xử lý lỗi I/O

Các ngoại lệ thường gặp:

Ngoại lệ	Nguyên nhân
FileNotFoundException / DirectoryNotFoundException	không tồn tại
UnauthorizedAccessException	không đủ quyền / đường dẫn là thư mục
IOException	file đang bị process khác khoá, đĩa đầy, đường dẫn sai
PathTooLongException, ArgumentException	đường dẫn không hợp lệ

```
try { File.ReadAllText(path); }
catch (FileNotFoundException e) { Console.WriteLine($"Không tìm thấy: {e.FileName}"); }
catch (IOException e) { Console.WriteLine($"Lỗi I/O: {e.Message}"); }
```

Đừng kiểm tra `File.Exists` rồi mới mở và tin là an toàn — giữa hai lệnh, file có thể bị xoá (**race condition** TOCTOU); vẫn phải `try/catch` khi mở.

Ghi file an toàn

Ghi trực tiếp đè lên file quan trọng mà chương trình sập giữa chừng → file hỏng. Mẫu an toàn: ghi vào **file tạm**, xong thì `File.Move(tam, dich, overwrite: true)` (thao tác đổi tên gần như nguyên tử).

Lỗi thường gặp

- Quên `using` → file bị khoá / dữ liệu chưa ghi.
- Dùng `ReadAllText` cho file khổng lồ → hết bộ nhớ (`OutOfMemoryException`).
- Ghép đường dẫn bằng `+` và dấu `\\` / `///` cứng.
- Đường dẫn tương đối phụ thuộc "thư mục làm việc hiện tại" (thay đổi tùy cách chạy).
Dùng `AppContext.BaseDirectory` hoặc đường dẫn tuyệt đối/cấu hình.
- Sai mã hoá → tiếng Việt vỡ dấu.
- Ghi vào thư mục cài đặt (`Program Files`) → `UnauthorizedAccessException`; dùng thư mục dữ liệu người dùng
(`Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData)`).
- Đọc file đang được chương trình khác ghi (cần `FileShare.ReadWrite`).

Bài tập

1. Viết chương trình đếm số dòng, số từ, số ký tự của một file văn bản (dùng `ReadLines`).
2. Sao chép toàn bộ file `.txt` từ thư mục A sang B, giữ nguyên cấu trúc thư mục con.
3. Viết nhật ký (logger) ghi thêm dòng có timestamp vào file `app.log`, mỗi lần chạy nối tiếp.

4. Tìm 5 file lớn nhất trong một thư mục (kèm thư mục con), in tên và kích thước.
5. Viết hàm `GhiAnToan(path, noiDung)` ghi qua file tạm rồi đổi tên.

Chương 36 — JSON và CSV

Mục tiêu học

Sau chương này, bạn sẽ:

- Chuyển object ↔ JSON bằng `System.Text.Json` (serialize/deserialize), tùy chỉnh bằng options và attribute.
- Đọc JSON không có sẵn class bằng `JsonDocument`.
- Đọc/ghi **CSV** đúng (dấu phẩy, ngoặc kép, xuống dòng trong ô) và biết khi nào nên dùng thư viện.
- Xử lý an toàn dữ liệu đầu vào không tin cậy.

Code mẫu: [code/ch36-json-csv/](#).

JSON là gì?

JSON là định dạng văn bản trao đổi dữ liệu phổ biến nhất hiện nay (API web, file cấu hình):

```
{ "ma": 1, "hoTen": "Nguyen Van An", "diem": 8.5, "monHoc": ["Toan", "Ly"] }
```

Gồm đối tượng `{ }`, mảng `[ ]`, chuỗi, số, `true/false`, `null`. Trong [ASP.NET Core](#), request/response JSON được chuyển đổi tự động bằng đúng cơ chế trong chương này (Tập 2). .NET có sẵn `System.Text.Json` (nhẹ, mặc định); bạn cũng sẽ gặp `Newtonsoft.Json` trong code cũ.

Serialize và Deserialize

```
using System.Text.Json;

var tuyChon = new JsonSerializerOptions
{
    WriteIndented = true,                // in đẹp, thật lè
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase, // Ten -> "ten"
};

string json = JsonSerializer.Serialize(sv, tuyChon);                // object -> json
SinhVien? sv2 = JsonSerializer.Deserialize<SinhVien>(json, tuyChon); // json -> object
```

Quy tắc: chỉ **public property** (có `get / set`) được xử lý mặc định; field public cần `IncludeFields = true`. `record`, `init`, `required` đều dùng được. Tên khớp **phân biệt**

hoa/thường trừ khi bật `PropertyNameCaseInsensitive = true` (hoặc dùng `JsonSerializerDefaults.Web` — camelCase + không phân biệt hoa/thường, chuẩn cho web).

Tái sử dụng `JsonSerializerOptions` (tạo một lần, để `static readonly`): tạo mới mỗi lần rất tốn hiệu năng.

Tuỳ chỉnh bằng attribute

```
class SinhVien
{
    public int Ma { get; set; }

    [JsonPropertyName("hoTen")] // đổi tên trong JSON
    public string Ten { get; set; } = "";

    [JsonConverter(typeof(JsonStringEnumConverter))] // enum ghi ra chữ ("Gioi")
    public XepLoai XepLoai { get; set; }

    [JsonIgnore] // KHÔNG đưa vào JSON (dù)
    public string MatKhai { get; set; } = "";
}
```

Chạy chương trình mẫu: `MatKhai` không xuất hiện trong JSON, và khi đọc lại nó là chuỗi rỗng.

Các tuỳ chọn hữu ích khác: `[JsonPropertyOrder]`, `DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull`, `Converters`, `NumberHandling.DateOnly`, `TimeOnly`, `DateTime`, `Guid`, `decimal` được hỗ trợ sẵn (ngày dùng chuẩn ISO 8601).

Đừng bao giờ serialize thẳng entity CSDL chứa mật khẩu/băm/dữ liệu nội bộ ra ngoài — dùng DTO/ record riêng cho đầu ra (Tập 2).

Đọc JSON không có class: JsonDocument

Khi cấu trúc thay đổi hoặc chỉ cần lấy vài giá trị:

```
using var doc = JsonDocument.Parse(jsonChuoi);
var goc = doc.RootElement;
string ten = goc.GetProperty("cua_hang").GetProperty("ten").GetString();
decimal tong = goc.GetProperty("cua_hang").GetProperty("sach")
    .EnumerateArray().Sum(x => x.GetProperty("gia").GetDecimal());
```

Dùng `TryGetProperty` nếu thuộc tính có thể vắng. `JsonDocument` chỉ đọc và nên `Dispose`.

Cần vừa đọc vừa sửa: `JsonNode` (`JsonNode.Parse(...)`, chỉ mục `node["a"]["b"]`).

Lỗi và dữ liệu xấu

```
try { JsonSerializer.Deserialize<SinhVien>("{ \"ma\": \"abc\" }", tuyChon); }
catch (JsonException e) { ... } // sai cú pháp hoặc sai kiểu
```

JSON từ bên ngoài **không đáng tin**: kiểm tra sau khi đọc (giá trị null, khoảng hợp lệ — `required`, `validation` của Tập 2), đặt giới hạn kích thước/độ sâu (`MaxDepth`) nếu nhận từ người lạ.

File và luồng

```
await File.WriteAllTextAsync(path, json);
await using var fs = File.OpenRead(path);
var t = await JsonSerializer.DeserializeAsync<SinhVien>(fs, tuyChon); // đọc thẳng t
```

`DeserializeAsyncEnumerable<T>` đọc mảng JSON rất lớn theo từng phần tử.

CSV

CSV (giá trị phân tách bằng dấu phẩy) là định dạng bảng đơn giản, mở được bằng Excel:

```
ma,ten,diem
1,An,8.5
2,"Binh, Nguyen",7.0
3,"Chi ""Cool""",9.0
```

Nghe đơn giản nhưng có bẫy: `Split(',')` **sai** vì ô có thể chứa dấu phẩy (`"Binh, Nguyen"`), dấu ngoặc kép (`"` biểu diễn một `"`) và cả xuống dòng. Quy tắc chuẩn (RFC 4180): ô chứa `,` , `"` hoặc xuống dòng phải bọc trong `"..."` , và mỗi `"` bên trong nhân đôi.

Code mẫu có bộ tách dòng đơn giản dạng **máy trạng thái** (đang trong hay ngoài ngoặc kép) để bạn hiểu nguyên lý, và hàm ghi `DongGoiCsv` escape đúng. Với dữ liệu thật (xuống dòng trong ô, mã hoá lạ, dấu phân cách `;` , file lớn), hãy dùng thư viện đã kiểm chứng như

CsvHelper:

```
dotnet add package CsvHelper
```

```
using var reader = new StreamReader("data.csv");
using var csv = new CsvReader(reader, CultureInfo.InvariantCulture);
var dong = csv.GetRecords<SinhVien>().ToList();
```

Lưu ý **culture**: số thực 8.5 hay 8,5 ? Excel tiếng Việt thường dùng dấu phẩy thập phân và ; làm dấu phân cách. Khi trao đổi dữ liệu máy-máy, dùng `CultureInfo.InvariantCulture` (Chương 11) và ghi rõ định dạng.

Với mọi dữ liệu người dùng tải lên: cẩn thận **CSV injection** — ô bắt đầu bằng = , + , - , @ có thể bị Excel hiểu là công thức; khi xuất ra cho người khác mở bằng Excel, thêm ký tự ' phía trước các ô đó.

JSON hay CSV hay XML?

	JSON	CSV	XML
Cấu trúc	lồng nhau, linh hoạt	bảng phẳng	lồng nhau, nặng nề
Dùng cho	API, cấu hình, lưu object	trao đổi bảng tính, báo cáo	hệ thống cũ, tài liệu
Kiểu dữ liệu	có (số, bool, null)	chỉ chuỗi	chỉ chuỗi (+ schema)

Lỗi thường gặp

- Quên `[JsonIgnore]` → lộ mật khẩu/dữ liệu nhạy cảm.
- Tên thuộc tính lệch hoa/thường → giá trị mặc định (không lỗi!) — kiểm tra kết quả; bật case-insensitive hoặc dùng `JsonSerializerDefaults.Web`.
- `JsonException` do sai kiểu ("abc" vào int).
- Deserialize ra null (đầu vào "null") → `NullReferenceException`.
- Dùng `Split(',')` cho CSV.
- Đọc double bằng Parse mà không chỉ culture → sai trên máy khác.
- Tạo `JsonSerializerOptions` mới mỗi lần gọi.
- Vòng tham chiếu (A trở B, B trở A) → `JsonException`; đặt `ReferenceHandler.IgnoreCycles` hoặc tách DTO.

Bài tập

1. Serialize một record có `List<record>` lồng nhau ra JSON đẹp; đọc lại và so sánh (==) hai object.
2. Đọc JSON từ chuỗi gồm mảng người dùng, dùng `JsonDocument` in tên những người trên 18 tuổi.
3. Ghi danh sách sinh viên ra file CSV và đọc lại đúng, kể cả tên có dấu phẩy và ngoặc kép.
4. Thêm thuộc tính NgayTao và cấu hình để bỏ qua giá trị null khi ghi.
5. Viết converter tùy chỉnh `JsonConverter<T>` đọc/ghi ngày theo định dạng dd/MM/yyyy.

Chương 37 — Reflection, Attribute và Serialization

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng **reflection** để khảo sát kiểu, thành viên và gọi chúng lúc chạy.
- Viết và đọc **attribute** tùy chỉnh (metadata gắn vào code).
- Hiểu vì sao framework ([ASP.NET](#) Core, EF Core, JSON, test) dựa nhiều vào hai thứ này.
- Nắm khái niệm **serialization** và khi nào nên/không nên dùng.

Code mẫu: [code/ch37-reflection-attribute/](#).

Reflection — chương trình tự soi mình

Reflection cho phép khảo sát **thông tin về kiểu lúc chạy**: có những property/method nào, kiểu gì, có attribute gì, và cả gọi/gán chúng mà không biết trước lúc biên dịch.

```
Type kieu = typeof(SanPham);           // hoặc obj.GetType()
foreach (var pr in kieu.GetProperties())
    Console.WriteLine($"{pr.Name}: {pr.PropertyType.Name}");

var sp = (SanPham)Activator.CreateInstance(kieu)!;           // tạo đối tượng từ Type
kieu.GetProperty("Ten")!.SetValue(sp, "Chuo");              // gán bằng tên
kieu.GetMethod("MoTa")!.Invoke(sp, null);                    // gọi bằng tên
```

Các API chính (`System.Reflection`): `Type` , `PropertyInfo` , `MethodInfo` , `FieldInfo` , `ConstructorInfo` , `Assembly` . `BindingFlags` lọc thành viên (`Public` | `NonPublic` | `Instance` | `Static` | `DeclaredOnly`).

Ai dùng reflection?

Bạn hiếm khi viết reflection trực tiếp, nhưng **mọi framework lớn đều dựa vào nó**:

- [ASP.NET](#) Core: tìm controller, gán giá trị request vào tham số (model binding).
- JSON serializer: liệt kê property để chuyển đổi.
- EF Core: ánh xạ class ↔ bảng.
- DI container, xUnit/NUnit (tìm `[Fact]` , `[Test]`), AutoMapper...

Cái giá của reflection

- **Chậm hơn** gọi trực tiếp (hàng chục–trăm lần) — tránh trong vòng lặp nóng; nếu cần, cache `PropertyInfo` / `MethodInfo` , hoặc dùng `Expression` /delegate biên dịch sẵn, hoặc **source generator** (cách .NET hiện đại thay thế reflection).
- **Mất an toàn kiểu lúc biên dịch:** gõ sai tên chuỗi `"Ten"` chỉ lỗi lúc chạy → dùng `nameof(SanPham.Ten)` .
- Có thể truy cập cả thành viên `private` → phá vỡ đóng gói; và không tương thích tốt với **trimming/AOT** (biên dịch ahead-of-time) vì công cụ cắt code không biết bạn sẽ gọi gì.

Quy tắc: **có lựa chọn thì đừng dùng reflection**; dùng khi thật sự cần tính linh hoạt kiểu plugin/framework.

Attribute — metadata gắn vào code

Attribute là "nhãn" thông tin đặt trong `[ ]` trước class, phương thức, property... Bản thân nó không làm gì; **có ai đó (trình biên dịch, framework, code của bạn) đọc nó** rồi hành động.

Attribute có sẵn bạn đã gặp: `[Flags]` (enum), `[Obsolete("...")]` (cảnh báo dùng API cũ), `[JsonPropertyName]` , `[JsonIgnore]` , `[Required]` , `[HttpGet]` , `[Fact]` , `[Serializable]` .

Tự định nghĩa attribute

Là class kế thừa `Attribute` , tên thường kết thúc bằng `Attribute` (khi dùng có thể bỏ hậu tố):

```
[AttributeUsage(AttributeTargets.Property)]    // chỉ đặt được trên property
class BatBuocAttribute : Attribute;

[AttributeUsage(AttributeTargets.Property)]
class KhoangAttribute(double min, double max) : Attribute
{
    public double Min { get; } = min;
    public double Max { get; } = max;
}

class SanPham
{
    [BatBuoc] public string Ten { get; set; } = "";
    [Khoang(0, 1000)] public int TonKho { get; set; }
}
```

Đọc attribute bằng reflection — mini validator

```
static List<string> KiemTra(object o)
{
    var loi = new List<string>();
```

```

foreach (var pr in o.GetType().GetProperties())
{
    var giaTri = pr.GetValue(o);
    if (pr.GetCustomAttribute<BatBuocAttribute>() is not null && (giaTri is null |
        loi.Add($"{pr.Name} la bat buoc");
    if (pr.GetCustomAttribute<KhoangAttribute>() is { } k && giaTri is IConvertible
    {
        double v = c.ToDouble(null);
        if (v < k.Min || v > k.Max) loi.Add($"{pr.Name} phai trong [{k.Min}, {k.Ma
    }
}
return loi;
}

```

Đây chính xác là cách `[Required]`, `[Range]` của **Data Annotations** hoạt động trong [ASP.NET Core](#): framework quét property, đọc attribute, kiểm tra giá trị. Bạn được dùng validator có sẵn thay vì tự viết (Tập 2), nhưng nay bạn hiểu bên trong.

Mẫu plugin: tìm kiểu cài đặt một interface

```

var cacPlugin = Assembly.GetExecutingAssembly().GetTypes()
    .Where(t => typeof(IPlugin).IsAssignableFrom(t) && t is { IsClass: true, IsAbstrac
    .Select(t => (IPlugin)Activator.CreateInstance(t!));

```

Thêm class `PluginC : IPlugin` là tự động được nhận ra, không sửa dòng nào — cơ chế "quét và đăng ký" mà các DI container hay dùng. (Với thư viện bên ngoài: `Assembly.LoadFrom(path)`.)

Serialization — tổng quan

Serialization = chuyển **đối tượng trong bộ nhớ** thành dạng có thể **lưu hoặc gửi đi** (chuỗi, byte), và **deserialization** là chiều ngược lại. Bạn đã làm với JSON (Chương 36).

Định dạng	Ưu	Nhược
JSON (<code>System.Text.Json</code>)	đọc được bằng mắt, chuẩn web, đa ngôn ngữ	kích thước lớn hơn nhị phân
XML (<code>XmlSerializer</code>)	hệ thống cũ, có schema	dài dòng
Protobuf, MessagePack	nhỏ, nhanh, có schema	không đọc được bằng mắt, cần thư viện
BinaryFormatter	—	ĐÃ BỊ LOẠI BỎ — nguy hiểm, tuyệt đối không dùng

Cảnh báo bảo mật: `BinaryFormatter` (và vài serializer nhị phân "ma thuật" tương tự) cho phép kẻ tấn công **thực thi mã tùy ý** khi deserialize dữ liệu độc hại; nó đã bị loại khỏi .NET hiện đại. Nguyên tắc chung: **không bao giờ deserialize dữ liệu không tin cậy** bằng cơ chế có thể tạo kiểu tùy ý. Dùng JSON với kiểu đích cố định (`Deserialize<SinhVien>`), tránh `object` /kiểu động và cấu hình cho phép kiểu đa hình tự do.

Khi thiết kế dữ liệu để serialize: dùng DTO/ `record` đơn giản, tránh tham chiếu vòng, không đưa thông tin nhạy cảm, và nghĩ đến **phiên bản hoá** (thêm trường mới không được làm hỏng người đọc cũ).

Lỗi thường gặp

- Gõ sai tên thuộc tính trong `GetProperty("Tên")` → trả về `null` → `NullReferenceException` (dùng `nameof`).
- `GetProperties()` không trả thành viên `private` / `static` (cần `BindingFlags`).
- Dùng reflection trong vòng lặp lớn mà không cache.
- Quên `[AttributeUsage]` khiến attribute đặt sai chỗ mà không báo lỗi.
- Nghĩ rằng attribute tự chạy: nó chỉ là dữ liệu, cần có code đọc nó.
- Dùng `BinaryFormatter` , hoặc deserialize dữ liệu người lạ sang kiểu tự do.
- Quên rằng trimming/AOT có thể loại bỏ thành viên chỉ dùng qua reflection.

Bài tập

1. Viết hàm `InThongTin(object o)` in tên và giá trị của mọi property public bằng reflection.
2. Thêm attribute `[DoDaiToiDa(50)]` vào validator và kiểm tra chuỗi.
3. Viết `[Ten("...")]` attribute cho enum và hàm đọc lấy tên hiển thị bằng reflection.
4. Đo thời gian gán 1 triệu lần một property bằng cách gọi trực tiếp và bằng `PropertyInfo.SetValue` ; sau đó cache `PropertyInfo` và đo lại.
5. Tạo thêm `PluginC : IPlugin` và chứng minh nó tự xuất hiện trong danh sách mà không sửa chỗ quét.

Chương 38 — Unit test với xUnit

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **unit test** để làm gì và các nguyên tắc của một test tốt.
- Viết test bằng **xUnit**: `[Fact]`, `[Theory]`, `Assert`, mẫu **Arrange-Act-Assert**.
- Dùng **đối tượng giả (mock/stub)** với `NSubstitute` để cô lập code cần test.
- Chạy test bằng `dotnet test` / IDE và đọc kết quả.

Code mẫu: [code/ch38-unit-test/](#) (thư viện `ShopLib` + project `ShopLib.Tests`).

Vì sao phải test?

Phần mềm thay đổi liên tục. Mỗi lần sửa, làm sao biết bạn không làm hỏng thứ đang chạy tốt? Bấm thử bằng tay thì chậm, dễ sót, không lặp lại được. **Test tự động** là chương trình kiểm tra chương trình khác — chạy trong vài giây, mỗi lần commit.

- **Phát hiện lỗi sớm**, rẻ hơn nhiều so với lỗi lên production.
- **Tự tin sửa/tái cấu trúc code** (Chương 42): test xanh nghĩa là hành vi vẫn đúng.
- **Tài liệu sống**: tên test mô tả code phải làm gì.
- Ép bạn viết code **để kiểm thử** — thường cũng là code thiết kế tốt (tách phụ thuộc bằng interface, Chương 18).

Các loại test (kim tự tháp): **unit test** (nhỏ, nhanh, nhiều nhất — một lớp/hàm) → **integration test** (nhiều phần ghép nhau, có CSDL/HTTP) → **end-to-end/UI test** (ít, chậm). Chương này tập trung unit test.

Tạo project test

```
dotnet new classlib -n ShopLib
dotnet new xunit -n ShopLib.Tests
dotnet add ShopLib.Tests reference ShopLib
dotnet sln add ShopLib ShopLib.Tests
```

Các thư viện thường dùng: **xUnit** (khung test, phổ biến nhất trong .NET; NUnit và MSTest là lựa chọn khác với ý tưởng tương tự), **NSubstitute** hoặc **Moq** (đối tượng giả),

FluentAssertions/Shouldly (assert dễ đọc, tùy chọn). Quy ước: project test đặt tên

`<Project>.Tests`, class test `<Lớp>Tests`.

Chạy: `dotnet test` (hoặc **Test Explorer** trong IDE). Kết quả liệt kê test đạt/hỏng và thông báo lỗi chi tiết.

Test đầu tiên: [Fact]

```
public class GioHangTests
{
    [Fact]
    public void TongTien_GioHangMoi_BangKhong()
    {
        var gio = new GioHang();

        Assert.Equal(0m, gio.TongTien);
    }
}
```

- `[Fact]` đánh dấu một test luôn đúng. Test là một phương thức `public`, không tham số.
- Test **đạt** nếu không ném ngoại lệ; `Assert.X` ném ngoại lệ khi sai.

Mẫu Arrange – Act – Assert (AAA)

```
[Fact]
public void Them_HaiSanPhamKhacNhau_TinhDungTongTien()
{
    // Arrange: chuẩn bị dữ liệu và đối tượng
    var gio = new GioHang();

    // Act: thực hiện đúng MỘT hành động cần kiểm tra
    gio.Them("A", "Chuot", 150_000m, 2);
    gio.Them("B", "Ban phim", 500_000m);

    // Assert: kiểm tra kết quả
    Assert.Equal(800_000m, gio.TongTien);
    Assert.Equal(2, gio.Dong.Count);
}
```

Đặt tên test nói rõ cái gì – trong tình huống nào – kết quả mong đợi:

`TenPhuongThuc_TinhHuong_KetQuaMongDoi`. Khi test đỏ, chỉ đọc tên đã biết hỏng ở đâu.

Các Assert thường dùng

<code>Assert.Equal(mongDoi, thucTe);</code>	<code>Assert.NotEqual(...)</code>
<code>Assert.True(dk);</code>	<code>Assert.False(dk);</code>
<code>Assert.Null(x);</code>	<code>Assert.NotNull(x);</code>

```
Assert.Empty(ds);                Assert.Single(ds);                Assert.Contains("abc", ch
Assert.Throws<ArgumentException>(() => gio.Them("", "x", 1m));    // kiểm tra NÉM lỗi đ
Assert.Equal(1.23, ketQua, precision: 2);                        // so sánh số thực
```

Thứ tự tham số: `Assert.Equal(mongDoi, thucTe)` (mong đợi trước). Đảo ngược làm thông báo lỗi gây hiểu nhầm.

[Theory] — một test, nhiều bộ dữ liệu

```
[Theory]
[InlineData(0, 1_000_000, 1_000_000)]
[InlineData(10, 1_000_000, 900_000)]
[InlineData(100, 1_000_000, 0)]
public void TinhTienSauGiamGia_PhanTramHopLe_TraVeDung(int phanTram, int gia, int mong
{
    var gio = new GioHang();
    gio.Them("A", "Laptop", gia);

    Assert.Equal((decimal)mongDoi, gio.TinhTienSauGiamGia(phanTram));
}
```

Mỗi `[InlineData]` sinh một test riêng. Với dữ liệu phức tạp dùng `[MemberData]` / `[ClassData]`. Nhớ kiểm tra cả **giá trị biên** (0, 100, âm, 101) — nơi hay có lỗi nhất.

Vòng đời và độc lập giữa các test

xUnit tạo **một đối tượng class test mới cho mỗi test**. Constructor là "setup", `IDisposable.Dispose` là "teardown":

```
public class DonHangServiceTests : IDisposable
{
    public DonHangServiceTests() { /* chạy trước MỖI test */ }
    public void Dispose() { /* chạy sau MỖI test */ }
}
```

Test phải **độc lập**: không phụ thuộc thứ tự chạy, không dùng chung trạng thái thay đổi được (biến `static`, file, CSDL thật). Test có thể chạy song song.

Cô lập phụ thuộc: mock và stub

Lớp `DonHangService` cần kho hàng (CSDL) và gửi email (mạng). Unit test không nên chạm vào chúng: chậm, không ổn định, có tác dụng phụ (gửi email thật!). Nhờ **lập trình theo interface**

(Chương 18), ta thay bằng **đối tượng giả**:

```
public class DonHangService(IKhoHang kho, IGuiEmail email) { ... } // phụ thuộc vào

// Trong test:
private readonly IKhoHang _kho = Substitute.For<IKhoHang>();
private readonly IGuiEmail _email = Substitute.For<IGuiEmail>();

[Fact]
public void DatHang_DuHang_GiamTonVaGuiEmail()
{
    var gio = new GioHang();
    gio.Them("A", "Chuot", 100_000m, 2);
    _kho.SoLuongTon("A").Returns(10); // STUB: dàn kết quả trả về

    var kq = _service.DatHang(gio, "khach@mail.com");

    Assert.True(kq.OK);
    _kho.Received(1).GiamTon("A", 2); // MOCK: kiểm tra hàm ĐÃ được gọi đúng
    _email.Received(1).Gui("khach@mail.com", Arg.Any<string>(), Arg.Any<string>());
}

[Fact]
public void DatHang_KhongDuHang_KhongGiamTonKhongGuiEmail()
{
    ...
    _kho.DidNotReceive().GiamTon(Arg.Any<string>(), Arg.Any<int>());
}
```

- **Stub**: trả dữ liệu định sẵn để dẫn code vào nhánh cần test.
- **Mock**: ghi lại lời gọi để bạn kiểm tra "có gọi/không gọi" — dùng khi hành vi quan sát được là *tác dụng phụ* (gửi email).
- Đừng lạm dụng mock cho mọi thứ: test bám sát chi tiết cài đặt sẽ hỏng mỗi lần tái cấu trúc. Ưu tiên kiểm tra **kết quả** trước, chỉ verify lời gọi khi đó chính là hành vi cần bảo đảm.
- Đây là lý do **Dependency Injection** (nhận phụ thuộc qua constructor) quan trọng: nó cho phép thay thế phụ thuộc khi test.

Nguyên tắc test tốt (FIRST)

- **Fast** — nhanh (mili-giây), để chạy liên tục.
- **Independent** — độc lập nhau.
- **Repeatable** — chạy lúc nào, ở đâu cũng cùng kết quả (không phụ thuộc giờ hiện tại, mạng, dữ liệu thật: bọc `DateTime.Now` sau interface `TimeProvider`).
- **Self-validating** — tự báo đạt/hỏng, không cần người đọc kết quả.

- **Timely** — viết cùng lúc/trước code.

Một test nên kiểm tra **một hành vi** (nhiều `Assert` cho cùng một hành vi là được), không có `if` /vòng lặp/logic phức tạp, và **đọc như tài liệu**. Test cũng là code: giữ sạch, tránh lặp.

TDD (Test-Driven Development): viết test *thất bại* trước → viết code tối thiểu cho test đạt → tái cấu trúc → lặp lại (Red – Green – Refactor). Không bắt buộc, nhưng luyện tập vài lần giúp thiết kế tốt hơn.

Độ phủ (code coverage)

`dotnet test --collect:"XPlat Code Coverage"` đo bao nhiêu dòng code được test chạy qua. Độ phủ **cao không có nghĩa là test tốt** (test không có `Assert` vẫn tăng độ phủ), nhưng **độ phủ thấp chắc chắn là vùng rủi ro**. Đừng đuổi theo con số 100%; hãy phủ kỹ nghiệp vụ quan trọng và các nhánh lỗi.

Lỗi thường gặp

- Test phụ thuộc thứ tự chạy hoặc trạng thái chung → lúc đạt lúc hỏng ("flaky test").
- Test gọi CSDL/mạng/ `DateTime.Now` thật.
- `Assert.Equal(thucTe, mongDoi)` đảo tham số.
- Một test làm quá nhiều việc; tên test mơ hồ (`Test1`).
- Bắt ngoại lệ bằng `try/catch` thay vì `Assert.Throws` (test luôn đạt nếu không ném lỗi).
- Mock quá nhiều → test gắn chặt vào cài đặt.
- Quên đặt `[Fact]` / `[Theory]` → test không bao giờ chạy (kiểm tra số test được phát hiện).
- So sánh số thực bằng `Assert.Equal(0.3, 0.1 + 0.2)` (dùng tham số `precision`).

Bài tập

1. Viết test cho `GioHang.Xoa` khi sản phẩm không tồn tại, và test gộp số lượng khi thêm cùng sản phẩm 3 lần.
2. Dùng `[Theory]` với `[MemberData]` kiểm tra hàm kiểm tra năm nhuận cho 10 năm khác nhau.
3. Viết `DonHangService` test cho trường hợp `IKhoHang` ném ngoại lệ (`_kho.When(...).Do(_ => throw ...)`); kiểm tra không gửi email.
4. Làm theo TDD: viết test cho `TinhPhiVanChuyen(khoiLuong)` (miễn phí dưới 1kg, 20k đến 5kg, 50k trên đó) trước rồi mới viết hàm.
5. Đo độ phủ và tìm các dòng chưa được test trong `GioHang` ; viết thêm test cho chúng.

Chương 39 — Debug và logging

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng debugger hiệu quả: breakpoint có điều kiện, Watch, Call Stack, Step, Exception settings.
- Ghi **log** đúng cách với `ILogger` : cấp độ, structured logging, exception, scope.
- Biết log cái gì và **không** log cái gì.
- Có quy trình hệ thống để tìm và sửa lỗi.

Code mẫu: [code/ch39-debug-logging/](#) .

Quy trình tìm lỗi

Đừng đoán mò. Làm theo trình tự:

1. **Tái hiện** lỗi ổn định (các bước cụ thể, dữ liệu cụ thể).
2. **Thu hẹp** phạm vi: lỗi nằm ở tầng nào, đầu vào nào?
3. **Đặt giả thuyết** rồi **kiểm chứng** bằng debugger/log/test — không sửa thử lung tung.
4. **Sửa nguyên nhân gốc**, không chỉ triệu chứng.
5. **Viết test** chứng minh lỗi đã sửa và không quay lại (Chương 38).

Đọc kỹ **thông báo lỗi và stack trace**: dòng trên cùng cho biết *chỗ nổ*, các dòng dưới cho biết *ai gọi đến đó*. Tìm dòng đầu tiên thuộc **code của bạn** — thường là nguồn gốc.

Debugger

Chạy ở chế độ **Debug** (`F5` , cấu hình Debug — khác Release: Release tối ưu code nên biến bị "mất", thứ tự chạy khó theo dõi).

Breakpoint

- **Breakpoint** (`F9`): dừng trước khi chạy dòng đó.
- **Conditional breakpoint**: chuột phải vào chấm đỏ → *Conditions*, ví dụ `i == 500` hay `ten == "An"` ; chỉ dừng khi đúng. Vô giá khi lỗi chỉ xảy ra ở vòng lặp thứ 500.
- **Hit count**: dừng ở lần thứ *n*. **Tracepoint / Logpoint**: in thông tin ra cửa sổ Output **mà không dừng** và không sửa code.
- **Exception breakpoint**: dừng ngay khi ném một loại exception (Debug → *Windows* → *Exception Settings*), kể cả khi nó được `catch` .

Điều khiển chạy

Thao tác	Phím (VS / VS Code)	Ý nghĩa
Continue	F5	chạy tới breakpoint kế
Step Over	F10	chạy hết dòng, không vào hàm
Step Into	F11	đi vào hàm được gọi
Step Out	Shift+F11	chạy hết hàm hiện tại, quay ra nơi gọi
Run to Cursor	Ctrl+F10	chạy tới dòng đang trỏ

Xem trạng thái

- **Locals / Autos**: biến cục bộ hiện tại. **Watch**: gõ biểu thức bạn quan tâm (`danhSach.Count` , `x * 2`).
- **Rê chuột** lên biến; ghim (pin) để theo dõi.
- **Call Stack**: chuỗi hàm dẫn tới đây — bấm vào từng khung để xem biến ở tầng đó.
- **Immediate/Debug Console**: chạy thử biểu thức, gọi hàm, **đổi giá trị biến** ngay lúc dừng.
- **Edit and Continue** (Visual Studio): sửa code khi đang dừng rồi chạy tiếp mà không khởi động lại.
- **Debug.WriteLine/ Debug.Assert** : chỉ hoạt động ở bản Debug; bị loại khi build Release.

Gỡ lỗi ứng dụng web/đang chạy

Attach to Process gắn debugger vào tiến trình đang chạy. Với [ASP.NET](#) Core, chạy dự án bằng `F5` là debug luôn; lỗi trên server thật thì **log** là công cụ chính (bạn không thể đặt breakpoint trên production).

Vì sao cần logging?

Debugger chỉ dùng được khi *bạn ngồi trước máy đang chạy*. Khi ứng dụng chạy trên server lúc 3 giờ sáng, **log** là "hộp đen" duy nhất ghi lại chuyện gì đã xảy ra.

ILogger — logging chuẩn của .NET

Thư viện: `Microsoft.Extensions.Logging` (là chuẩn; [ASP.NET](#) Core dùng sẵn, nhận `ILogger<T>` qua dependency injection).

```
using var factory = LoggerFactory.Create(b => b.SetMinimumLevel(LogLevel.Debug).AddSink<Program>());
ILogger<Program> log = factory.CreateLogger<Program>();
```

```
log.LogInformation("Nguoi dung {NguoiDung} them {SoLuong} san pham", nguoiDung, soLuor
```

Trong lớp nghiệp vụ, nhận `ILogger<T>` qua constructor (Tập 2 sẽ dùng DI tự cấp):

```
class DichVuTinhToan(ILogger<DichVuTinhToan> log) { ... }
```

Các cấp độ (LogLevel)

Cấp	Dùng cho
Trace	cực kỳ chi tiết, có thể chứa dữ liệu nhạy cảm (chỉ dev)
Debug	thông tin cho lập trình viên khi điều tra
Information	luồng bình thường của ứng dụng (đã đăng nhập, đã đặt hàng)
Warning	bất thường nhưng vẫn chạy được (dữ liệu thiếu, dùng giá trị mặc định, sắp hết dung lượng)
Error	một thao tác thất bại (exception đã bắt)
Critical	lỗi nghiêm trọng, ứng dụng/hệ thống có nguy cơ sập

Mức tối thiểu cấu hình theo môi trường (`Debug` khi dev, `Information / Warning` khi production) — **không đổi code** mà đổi cấu hình (`appsettings.json` , Chương 40; chương trình mẫu đọc biến môi trường `LOG_LEVEL`).

Structured logging — dùng tham số đặt tên, không nội suy chuỗi

```
log.LogInformation("Nguoi dung {NguoiDung} them {SoLuong} san pham", nguoiDung, soLuor
log.LogInformation($"Nguoi dung {nguoiDung} them {soLuong} san pham");
```

Bản đúng giữ lại **mẫu thông điệp** và **các tham số riêng biệt** → công cụ (Seq, Elasticsearch, Application Insights) lọc/tìm theo `NguoiDung = "an"` được, và tiết kiệm bộ nhớ khi mức log bị tắt (không phải dựng chuỗi). Bản nội suy chỉ ra một chuỗi trơn.

Ghi exception

```
catch (DivideByZeroException e)
{
    log.LogError(e, "Khong the chia {A} cho {B}", a, b);    // truyền exception làm th
    throw;
}
```

Truyền cả `Exception` để log giữ đầy đủ stack trace. **Đừng vừa log vừa throw ở mọi tầng** — cùng một lỗi bị log nhiều lần gây nhiễu. Quy tắc: log **một lần** ở nơi cuối cùng xử lý lỗi (ví dụ middleware toàn cục của web), hoặc log ở nơi có thêm ngữ cảnh hữu ích rồi ném lại.

Scope — gắn ngữ cảnh chung

```
using (log.BeginScope("RequestId={RequestId}", requestId))
{
    log.LogInformation("Bat dau xu ly");    // mọi dòng log trong khối đều kèm RequestId
    ...
}
```

Rất hữu ích để nối các dòng log của cùng một request (correlation id).

Đích ghi log (provider) và Serilog

Mặc định .NET có provider Console, Debug, EventLog. Cho ứng dụng thật, nhiều người dùng **Serilog** (hoặc NLog) — cắm vào `ILogger` nên code của bạn không đổi — để ghi ra file (có xoay vòng), Seq, Elasticsearch, Application Insights:

```
dotnet add package Serilog.AspNetCore
```

Ở Tập 2 ([ASP.NET](#) Core) ta sẽ cấu hình Serilog và OpenTelemetry (Tập 3).

Log cái gì, không log cái gì?

Nên log: khởi động/tắt ứng dụng, các sự kiện nghiệp vụ quan trọng, lỗi và cảnh báo (kèm ID/ngữ cảnh: mã đơn hàng, user id), thời gian xử lý các thao tác chậm, lời gọi dịch vụ ngoài và kết quả.

TUYỆT ĐỐI KHÔNG log: mật khẩu, token, khoá API, số thẻ, dữ liệu cá nhân nhạy cảm (số CMND, y tế...), toàn bộ body request chứa dữ liệu như vậy. Log thường được lưu lâu và nhiều người truy cập được → rò rỉ. Che/băm dữ liệu nhạy cảm (`sdt: 09****111`).

Log **đúng lượng:** quá ít không đủ điều tra, quá nhiều (log trong vòng lặp nóng) làm chậm và tốn dung lượng. Mỗi dòng log nên giúp trả lời được: *cái gì xảy ra, với đối tượng nào, kết quả ra sao*.

Lỗi thường gặp

- Debug ở cấu hình Release rồi thắc mắc vì sao breakpoint không dừng/biến "optimized away".
- Nội suy chuỗi trong `Log...` thay vì tham số đặt tên.

- Log mật khẩu/token.
- `catch (Exception e) { log.LogError(e.Message); }` — mất stack trace; truyền `e` làm tham số đầu.
- Log lỗi ở mọi tầng → một sự cố ra hàng chục dòng.
- Đặt tất cả ở `Information`, không phân cấp → không lọc được; hoặc để `Debug/Trace` trên production.
- Dùng `Console.WriteLine` thay `ILogger` trong ứng dụng thật (không có cấp độ, không cấu hình được).
- Sửa lỗi bằng cách "thử đại" thay vì tái hiện và xác định nguyên nhân.

Bài tập

1. Chạy chương trình mẫu với `LOG_LEVEL=Debug` rồi `Warning`; quan sát dòng nào biến mất.
2. Đặt breakpoint có điều kiện trong `TinhTong` chỉ dừng khi `x == 2`; dùng Watch theo dõi `tong`.
3. Thêm log vào `TaiKhoan.RutTien` (Chương 14): `Information` khi thành công, `Warning` khi không đủ tiền — có nên log số dư?
4. Cố tình gây `NullReferenceException`, đọc stack trace và tìm dòng gốc bằng Call Stack.
5. Dùng Logpoint (không sửa code) để in giá trị một biến trong vòng lặp.

Chương 40 — Git, NuGet và cấu hình

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng **Git** cho quy trình làm việc hằng ngày: `commit`, `branch`, `merge`, `.gitignore`.
- Quản lý gói **NuGet** và hiểu `PackageReference`, phiên bản, `dotnet restore`.
- Tách **cấu hình** khỏi code: `appsettings.json`, biến môi trường, **user secrets**, **options pattern**.
- Không bao giờ để **bí mật** (mật khẩu, khoá API) lọt vào Git.

Code mẫu: [code/ch40-git-nuget-cau-hinh/](#).

Git — quản lý phiên bản

Git ghi lại lịch sử thay đổi của dự án, cho phép quay lại, thử nghiệm song song (branch) và cộng tác. **GitHub/GitLab/ Azure DevOps** lưu repository trên mạng.

Lệnh hằng ngày

```
git clone <url>           # lấy repo về
git status                 # xem thay đổi
git add <file> | git add . # đưa vào "staging"
git commit -m "Mô tả ngắn" # chốt thành một commit
git log --oneline          # xem lịch sử
git diff                   # xem chi tiết chưa commit
git pull                   # lấy thay đổi mới từ server (fetch + merge)
git push                   # đẩy commit lên server
```

Ba "vùng": **working directory** (file đang sửa) → **staging** (`add`) → **repository** (`commit`). Commit là ảnh chụp toàn bộ trạng thái tại một thời điểm.

Branch (nhánh)

Làm tính năng mới trên nhánh riêng để không đụng `main` :

```
git switch -c tinh-nang-gio-hang # tạo và chuyển sang nhánh mới
# ... sửa, commit ...
git switch main
git merge tinh-nang-gio-hang     # gộp vào main
```

Quy trình phổ biến (**GitHub Flow**): mọi công việc trên nhánh ngắn hạn → mở **Pull Request** → review → CI chạy test → merge vào `main`. Xung đột khi merge (hai người sửa cùng dòng) phải giải quyết bằng tay: Git đánh dấu `<<<<<< / ===== / >>>>>>`, bạn chọn nội dung đúng, `git add`, rồi `commit`.

Viết commit tốt

- Mỗi commit **một thay đổi logic**, build được, test đạt.
- Thông điệp: dòng đầu ngắn (≤ 72 ký tự), thể hiện *cái gì và vì sao*; ví dụ `Thêm kiểm tra so luong am cho GioHang`.
- Commit **thường xuyên**, đừng gom cả tuần vào một commit.

.gitignore — không đưa gì vào Git

File `.gitignore` liệt kê thứ **không** theo dõi. Kho này đã có sẵn mẫu Visual Studio. Chủ yếu loại: `bin/`, `obj/`, `.vs/`, `*.user`, thư mục gói tải về, file log/tạm. Tạo mẫu cho dự án .NET: `dotnet new gitignore`.

Bài học đắt giá: thứ đã commit (dù xoá ở commit sau) **vẫn nằm trong lịch sử**. Bí mật lỡ commit phải coi là **đã lộ** — đổi ngay (rotate) chứ không chỉ xoá file.

NuGet — quản lý gói thư viện

Bạn đã dùng NuGet ở Chương 4. Nhắc lại và mở rộng:

```
dotnet add package Serilog           # thêm bản mới nhất ổn định
dotnet add package Serilog --version 4.2.0 # chỉ định phiên bản
dotnet list package                  # đã dùng gói nào
dotnet list package --outdated       # gói nào có bản mới
dotnet list package --vulnerable     # gói có lỗ hổng bảo mật đã biết
dotnet remove package Serilog
dotnet restore                       # tải gói theo .csproj (build/run tự gọi)
```

Trong `.csproj`:

```
<ItemGroup>
  <PackageReference Include="Microsoft.Extensions.Configuration.Json" Version="10.0.0" />
</ItemGroup>
```

- **Phiên bản (SemVer)** `Major.Minor.Patch`: tăng *Major* = có thể phá vỡ tương thích; *Minor* = thêm tính năng tương thích; *Patch* = sửa lỗi. Đọc *release notes* trước khi nâng Major.
- Gói kéo theo **phụ thuộc bắc cầu** (transitive): thêm một gói có thể tải hàng chục gói khác. Kiểm tra gói lỗ hổng định kỳ.

- **Central Package Management** (`Directory.Packages.props`): ghi phiên bản một chỗ cho cả solution nhiều project.
- Chọn thư viện có uy tín: nhiều lượt tải, cập nhật gần đây, giấy phép rõ ràng, mã nguồn mở. Mỗi gói là **rủi ro chuỗi cung ứng**.
- Bạn cũng có thể đóng gói thư viện của mình (`dotnet pack`) và đẩy lên NuGet (công khai hoặc nguồn riêng của công ty).

Cấu hình ứng dụng

Nguyên tắc (*Twelve-Factor App*): **cấu hình tách khỏi code**. Chuỗi kết nối CSDL, URL dịch vụ, cờ bật/tắt tính năng, mức log... thay đổi giữa máy dev / test / production **mà không phải sửa code hay build lại**.

`Microsoft.Extensions.Configuration` hợp nhất nhiều **nguồn** cấu hình thành một cây khoá-giá trị:

```
IConfiguration cauHinh = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddJsonFile($"appsettings.{moiTruong}.json", optional: true)
    .AddUserSecrets<Program>(optional: true)
    .AddEnvironmentVariables()
    .Build();
```

Nguồn thêm sau ghi đè nguồn thêm trước — thứ tự ở trên nghĩa là biến môi trường có ưu tiên cao nhất, rồi user secrets, rồi file theo môi trường, cuối cùng `appsettings.json` (giá trị mặc định). (Với `WebApplication.CreateBuilder` của [ASP.NET](#) Core, việc này đã được cấu hình sẵn — Tập 2.)

appsettings.json và theo môi trường

```
{
  "CuaHang": { "Ten": "Sach Xanh", "ThueVat": 0.08, "KenhHoTro": [ "email", "dien thoai" ],
  "KetNoi": { "MayChu": "localhost", "MatKhai": "" }
}
```

`appsettings.Development.json` chỉ chứa phần **khác biệt** ở môi trường dev. Môi trường lấy từ biến `DOTNET_ENVIRONMENT` (hoặc `ASPNETCORE_ENVIRONMENT` với web): `Development`, `Staging`, `Production`. (Trong csproj cần đặt file `CopyToOutputDirectory` để file JSON nằm cạnh file chạy.)

Đọc cấu hình

```
cauHinh["CuaHang:Ten"] // khoá lồng nhau dùng dấu ':'
cauHinh.GetValue<int>("CuaHang:SoNgayDoiTra", 7) // có kiểu, kèm giá trị mặc định
var cuaHang = cauHinh.GetSection("CuaHang").Get<CuaHangOptions>(); // BIND vào class
```

Options pattern: gom cấu hình liên quan vào một class đơn giản (`CuaHangOptions`) và truyền vào nơi cần qua `IOptions<CuaHangOptions>` . Code nghiệp vụ không đọc chuỗi khoá rải rác — an toàn kiểu, dễ test, dễ kiểm tra tính hợp lệ (`ValidateDataAnnotations` , `ValidateOnStart` — Tập 2).

Biến môi trường

Ghi đè bất kỳ khoá nào bằng biến môi trường, dùng `__` (hai gạch dưới) thay cho `:` :

```
CuaHang__Ten="Sach Do" # tương ứng CuaHang:Ten
```

Đây là cách chuẩn để cấu hình khi chạy trên container/Docker/cloud (Tập 3) mà không đụng file.

Bí mật: user-secrets và kho bí mật

Không bao giờ đặt mật khẩu, khoá API, chuỗi kết nối chứa mật khẩu trong `appsettings.json` rồi commit.

- **Lúc phát triển: User Secrets** — lưu ngoài thư mục dự án (trong thư mục hồ sơ người dùng), không bị đưa vào Git:

```
dotnet user-secrets init
dotnet user-secrets set "KetNoi:MatKhau" "bi-mat-cua-toi"
dotnet user-secrets list
```

- **Production:** biến môi trường do nền tảng cấp, hoặc kho bí mật chuyên dụng (**Azure Key Vault**, AWS Secrets Manager, HashiCorp Vault) qua provider cấu hình tương ứng.
- **User Secrets không mã hoá**; chỉ để tiện dev, không phải giải pháp production.

Checklist trước khi push

- [] Không có mật khẩu/khoá API/token trong code hay file cấu hình được commit (tìm `password` , `secret` , `apikey`).
- [] `.gitignore` đã loại `bin/` , `obj/` , `.vs/` , file bí mật cục bộ.
- [] Build và test đạt (`dotnet build` , `dotnet test`).
- [] Commit nhỏ, thông điệp rõ ràng.

Lỗi thường gặp

- Commit `appsettings.json` chứa mật khẩu thật; commit `bin/`, `obj/`.
- `FileNotFoundException: appsettings.json` — quên `CopyToOutputDirectory` hoặc đặt sai `SetBasePath`.
- Cấu hình đọc ra `null` /mặc định vì gõ sai khoá hoặc sai cấp lồng nhau (không có lỗi báo!). Bind vào class và kiểm tra bắt buộc.
- Hiểu sai thứ tự ưu tiên nguồn cấu hình (nguồn sau thắng).
- Sai dấu phân cách biến môi trường: dùng `__`, không phải `:` (một số shell không cho `:`).
- Không `git pull` trước khi `git push` → bị từ chối (non-fast-forward).
- Dùng `git add .` mà không xem `git status` → đưa cả file rác/bí mật vào commit.
- Dùng `git push --force` lên nhánh chung (ghi đè lịch sử của người khác).

Bài tập

1. Tạo repo mới, commit `.gitignore`, một dự án console, rồi tạo nhánh `feature/x`, sửa, merge về `main`.
2. Tạo cố ý một **xung đột merge** (hai nhánh sửa cùng một dòng) và giải quyết nó.
3. Thêm `Serilog` vào project bằng CLI; dùng `dotnet list package --outdated` và `--vulnerable`.
4. Ghi đè `CuaHang:Ten` bằng biến môi trường và bằng `user-secrets`; quan sát ai thắng khi cả hai cùng có.
5. Thêm class `KetNoiOptions` và bind mục `KetNoi`; ném lỗi khi `MayChu` rỗng ngay lúc khởi động.

Chương 41 — Design pattern cơ bản

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **design pattern** là gì và dùng chúng để làm gì (và không phải để làm gì).
- Cài đặt và nhận ra 5 pattern nền tảng: **Singleton**, **Factory**, **Strategy**, **Observer**, **Decorator**.
- Biết cách .NET hiện đại thể hiện các pattern này (DI lifetime, `event`, `delegate`, `middleware`).
- Tránh lạm dụng pattern.

Code mẫu: [code/ch41-design-pattern/](#).

Design pattern là gì?

Design pattern (mẫu thiết kế) là **lời giải đã được kiểm chứng cho một vấn đề thiết kế thường gặp**, có tên gọi chung để lập trình viên trao đổi ("dùng Strategy ở đây"). Nó **không phải** đoạn code copy-paste, mà là ý tưởng bạn hiện thực hoá trong ngữ cảnh của mình. Bộ mẫu kinh điển là 23 pattern của "Gang of Four" (GoF), chia ba nhóm:

- **Creational** (khởi tạo): Singleton, Factory, Builder...
- **Structural** (cấu trúc): Decorator, Adapter, Facade...
- **Behavioral** (hành vi): Strategy, Observer, Command...

Cảnh báo: pattern là công cụ, không phải mục tiêu. Nhồi pattern vào code đơn giản chỉ làm nó rối. Bắt đầu bằng code đơn giản nhất chạy đúng; áp dụng pattern khi thấy **vấn đề cụ thể** (lập `switch`, cứng nhắc, khó test) mà pattern giải quyết. Nhiều pattern cổ điển đã có sẵn trong ngôn ngữ/framework hiện đại (`event`, `delegate`, DI, `middleware`, LINQ).

1. Singleton — một thể hiện duy nhất

Vấn đề: chỉ được có **một** đối tượng của class (cấu hình dùng chung, bộ nhớ đệm, kết nối tốn kém).

```
sealed class CauHinhUngDung
{
    private static readonly Lazy<CauHinhUngDung> _instance = new(() => new CauHinhUngDung());
    public static CauHinhUngDung Instance => _instance.Value;
```

```
private CauHinhUngDung() { }           // cấm new từ bên ngoài
}
```

`Lazy<T>` bảo đảm khởi tạo **an toàn đa luồng** và **chỉ khi cần**. Khuyết điểm của Singleton tự viết: là **trạng thái toàn cục ẩn** — code nào cũng gọi `CauHinhUngDung.Instance` được, phụ thuộc bị giấu, **khó test** (không thay bằng đối tượng giả được) và dễ gây lỗi đa luồng.

Cách hiện đại: để **DI container** quản lý thời gian sống:

`builder.Services.AddSingleton<ICauHinh, CauHinh>();` . Bạn vẫn có một thể hiện duy nhất, nhưng nhận nó qua constructor dưới dạng interface — test thay thế dễ dàng (Tập 2). Nên hiểu Singleton như **một lifetime**, không phải chuyện tự viết `static Instance` .

2. Factory — che giấu việc tạo đối tượng

Vấn đề: chọn *lớp cụ thể* nào để `new` phụ thuộc vào điều kiện (tham số, cấu hình), và bạn không muốn logic đó rải khắp nơi.

```
static class XuatBaoCaoFactory
{
    public static IXuatBaoCao Tao(string loai) => loai.ToLowerInvariant() switch
    {
        "pdf" => new XuatPdf(),
        "excel" => new XuatExcel(),
        "csv" => new XuatCsv(),
        _ => throw new NotSupportedException($"Khong ho tro '{loai}'"),
    };
}

IXuatBaoCao xuat = XuatBaoCaoFactory.Tao("pdf");    // người dùng chỉ biết interface
```

Bên gọi **chỉ phụ thuộc vào interface** `IXuatBaoCao` ; thêm định dạng mới chỉ sửa nhà máy. Biến thể: **Abstract Factory** (nhà máy tạo cả họ đối tượng liên quan), **Factory Method**, và **Builder** khi đối tượng có nhiều tham số tùy chọn. Trong DI, mẫu tương đương là `Func<string, IXuatBaoCao>` hoặc "keyed services" (`[FromKeyedServices("pdf")]`).

3. Strategy — hoán đổi thuật toán

Vấn đề: cùng một việc có nhiều cách làm (tính giảm giá, tính phí ship, sắp xếp) và cách nào dùng được chọn lúc chạy. Đừng để một `switch / if` khổng lồ:

```
interface IChinhSachGiamGia { string Ten { get; } decimal TinhTien(decimal tienHang);

class GiamPhanTram(decimal phanTram) : IChinhSachGiamGia
```

```

{
    public string Ten => $"Giam {phanTram}%";
    public decimal TinhTien(decimal t) => t * (100 - phanTram) / 100;
}
class GiamCoDinh(decimal soTien) : IChinhSachGiamGia { ... }

foreach (var cs in chinhSach)
    Console.WriteLine($"{cs.Ten}: {cs.TinhTien(1_000_000m):N0}");

```

Thêm chính sách mới = thêm class mới, không sửa code đang chạy (Open/Closed, Chương 42). Khi thuật toán chỉ là một hàm nhỏ, **delegate** gọn hơn interface — chính là Strategy trong C#:

```

class DonHang(decimal tienHang, Func<decimal, decimal> chinhSach) { public decimal Tor
new DonHang(1_000_000m, tien => tien * 0.85m);

```

`List<T>.Sort(Comparison<T>)`, `Where(Func<T, bool>)` đều là Strategy.

4. Observer — thông báo cho nhiều bên

Vấn đề: khi một đối tượng (Subject) thay đổi, nhiều đối tượng khác (Observer) cần biết mà Subject **không nên phụ thuộc** vào từng người nghe.

Trong C#, đây chính là **event** (Chương 31):

```

class CuaHangGiaCoPhieu
{
    public event EventHandler<GiaThayDoiEventArgs>? GiaThayDoi;
    public void DatGia(string ma, decimal giaMoi) { ...; GiaThayDoi?.Invoke(this, new(
}

cuaHang.GiaThayDoi += (s, e) => Console.WriteLine($"[Bang dien] {e.Ma}: {e.GiaCu} -> {
cuaHang.GiaThayDoi += (s, e) => { if (e.GiaMoi > e.GiaCu * 1.05m) Console.WriteLine("C

```

Publisher không biết ai đang nghe; thêm/bớt người nghe không đụng vào publisher. Lưu ý huỷ đăng ký (-=) để tránh rò rỉ bộ nhớ. Với luồng sự kiện phức tạp/bất đồng bộ: `IObservable<T>` (Reactive Extensions), `Channel<T>` (Chương 34), hoặc message bus (MediatR, hàng đợi thông điệp — Tập 3).

5. Decorator — bọc thêm hành vi

Vấn đề: thêm chức năng (log, cache, thời gian, mã hoá) cho một đối tượng mà **không sửa nó** và không nổ tung số lớp con.

```

interface IThongBao { void Gui(string noiDung); }

class ThongBaoConsole : IThongBao { public void Gui(string s) => Console.WriteLine(s);

class ThongBaoCoThoiGian(IThongBao ben) : IThongBao
{
    public void Gui(string s) => ben.Gui($"[08:30] {s}");    // thêm hành vi rồi giao
}
class ThongBaoVietHoa(IThongBao ben) : IThongBao
{
    public void Gui(string s) => ben.Gui(s.ToUpperInvariant());
}

IThongBao tb = new ThongBaoVietHoa(new ThongBaoCoThoiGian(new ThongBaoConsole()));
tb.Gui("he thong da san sang");    // [08:30] HE THONG DA SAN SANG

```

Mỗi decorator **cài đúng interface** và **giữ một tham chiếu tới đối tượng cùng interface**; xếp chồng như búp bê Nga. Bạn sẽ gặp lại ở **middleware pipeline** của [ASP.NET](#) Core, `Stream` (`GZipStream` bọc `FileStream`), decorator cache/log cho repository (Tập 3).

Chọn pattern nào khi nào?

Dấu hiệu trong code	Cân nhắc
Cần đúng một thể hiện dùng chung	Singleton (qua DI lifetime)
new rải rác kèm if/switch chọn lớp	Factory
switch khổng lồ theo "loại" cùng làm một việc	Strategy
Một thay đổi cần thông báo nhiều nơi	Observer (event)
Cần thêm hành vi xuyên suốt (log, cache) mà không sửa lớp gốc	Decorator
Chuẩn bị đối tượng phức tạp nhiều bước/tham số	Builder
Giao diện không khớp thư viện ngoài	Adapter
Che phức tạp của nhiều lớp sau một cửa	Facade
Đóng gói yêu cầu thành đối tượng (undo, hàng đợi)	Command

Lỗi thường gặp

- **Lạm dụng pattern:** ba lớp và hai interface cho việc một hàm làm được ("over-engineering").
- Singleton tự viết mang trạng thái thay đổi được → lỗi đa luồng, test không độc lập.

- Factory trả kiểu cụ thể thay vì interface → mất ý nghĩa.
- Strategy khi chỉ có một cách làm (không có biến thể thật).
- Observer không hủy đăng ký; ngoại lệ trong một handler làm hỏng các handler sau.
- Decorator lồng quá sâu, thứ tự bọc thay đổi kết quả mà không ai để ý.
- Học thuộc tên pattern mà không hiểu **vấn đề** nó giải quyết.

Bài tập

1. Thêm định dạng `json` vào Factory và chứng minh không phải sửa code gọi.
2. Thêm `GiamTheoNgayLe` (giảm 30% nếu tháng 12) vào Strategy; dùng `TimeProvider` để test được.
3. Viết `CacheDecorator` bọc `IKhoSanPham` (interface có `LaySanPham(int id)`) để lưu kết quả vào `Dictionary`.
4. Dùng Observer mô phỏng "nút Đặt hàng": khi bấm, một handler gửi email, một handler trừ kho, một handler ghi log.
5. Tìm ba chỗ trong .NET (không phải code bạn viết) thể hiện Strategy, Observer, Decorator.

Chương 42 — Nguyên lý SOLID

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu và áp dụng 5 nguyên lý **SOLID** qua ví dụ tái cấu trúc thực tế.
- Nhận ra "mùi" code vi phạm từng nguyên lý.
- Thấy mối liên hệ giữa SOLID, interface, đa hình, **Dependency Injection** và khả năng **kiểm thử**.
- Biết khi nào áp dụng vừa đủ, không "quá tay".

Code mẫu: [code/ch42-solid/](#) — mỗi nguyên lý có cặp "Xấu" và "Tốt".

SOLID là gì?

Năm nguyên lý thiết kế hướng đối tượng giúp code **dễ hiểu, dễ sửa, dễ mở rộng, dễ kiểm thử**:

Chữ	Nguyên lý	Một câu
S	Single Responsibility	Một lớp chỉ có một lý do để thay đổi
O	Open/Closed	Mở để mở rộng, đóng để sửa đổi
L	Liskov Substitution	Lớp con phải thay thế được lớp cha mà không phá hành vi
I	Interface Segregation	Nhiều interface nhỏ tốt hơn một interface to
D	Dependency Inversion	Phụ thuộc vào trừu tượng , không vào chi tiết cụ thể

Đây là *nguyên tắc chỉ đạo*, không phải luật cứng: dùng để đánh giá thiết kế, cân nhắc chi phí/lợi ích.

S — Single Responsibility (đơn trách nhiệm)

Vi phạm: một lớp vừa tính thuế, vừa định dạng, vừa in, vừa lưu CSDL. Đổi cách in cũng phải mở lớp tính thuế → dễ làm hỏng chỗ khác.

```
class HoaDonXau
{
    public void InVaLuu(decimal tien)
    {
        decimal thue = tien * 0.1m;           // logic thuế
        string s = $"Hoa don: {tien + thue:N0}"; // định dạng
    }
}
```

```

        Console.WriteLine(s); // in
        // ... rồi lưu CSDL, gửi email ...
    }
}

```

Tái cấu trúc: tách theo **lý do thay đổi** (thuế đổi / mẫu in đổi / nơi lưu đổi):

```

class HoaDonService(ITinhThue thue, IInHoaDon inAn, ILuuTru luu)
{
    public void XuLy(decimal tien)
    {
        string noiDung = $"Hoa don: {tien + thue.Thue(tien):N0}";
        inAn.In(noiDung);
        luu.Luu(noiDung);
    }
}

```

`HoaDonService` chỉ **điều phối**. Dấu hiệu vi phạm: tên lớp có "Manager/Helper/Utils" ôm mọi thứ; lớp dài hàng nghìn dòng; phải mô tả bằng chữ "và". Đứng cực đoan: một "trách nhiệm" là *một lý do thay đổi*, không phải một phương thức.

O — Open/Closed (đóng-mở)

Vi phạm: mỗi lần thêm loại khách phải **sửa** hàm cũ (đã chạy tốt, đã test):

```

decimal Giam(string loai, decimal tien) => loai switch
{
    "vip" => tien * 0.8m,
    "sinhvien" => tien * 0.9m,
    _ => tien, // thêm "nhanvien"? phải sửa hàm này (rủi ro hồi quy)
};

```

Tái cấu trúc: đa hình — thêm lớp mới thay vì sửa lớp cũ:

```

interface IGiamGia { decimal Giam(decimal tien); }
class GiamVip : IGiamGia { public decimal Giam(decimal t) => t * 0.8m; }
class GiamSinhVien : IGiamGia { public decimal Giam(decimal t) => t * 0.9m; }
// Thêm GiamNhanVien? Chỉ VIẾT lớp mới, code đang chạy không đổi.

```

Nền tảng: đa hình (Chương 17), interface (18), Strategy (41). Không cần "thiết kế mở" cho mọi thứ — chỉ ở những điểm **đã từng/hay đổi**. Lần đầu cứ viết đơn giản; khi thấy `switch` thứ hai cùng kiểu xuất hiện, hãy tách.

L — Liskov Substitution (thay thế Liskov)

Nếu B kế thừa A thì **mọi nơi dùng A phải chạy đúng khi truyền B**. Ví dụ kinh điển: Hình vuông "là một" hình chữ nhật?

```
class HCNXau { public virtual double Rong { get; set; } public virtual double Cao { get; set; }  
class VuongXau : HCNXau { /* đặt Rong thì Cao đổi theo, và ngược lại */  
  
HCNXau h = new VuongXau();  
h.Rong = 2; h.Cao = 3;  
h.DienTich(); // mong 6, nhưng ra 9 → phá vỡ hợp đồng của HCNXau
```

Trong toán "vuông là chữ nhật", nhưng với đối tượng **có thể thay đổi**, hành vi khác nhau → kế thừa sai. Sửa: bỏ quan hệ kế thừa; cùng cài một interface chung nhỏ:

```
interface IHinh { double DienTich(); }  
record HCN(double Rong, double Cao) : IHinh { ... }  
record HinhVuong(double Canh) : IHinh { ... }
```

Dấu hiệu vi phạm: lớp con `throw new NotImplementedException() / NotSupportedException` cho phương thức của cha; lớp con thu hẹp điều kiện đầu vào, nới lỏng điều kiện đầu ra; code phải `if (x is LopCon)` để xử lý riêng. Quy tắc: kế thừa để **thể hiện "là một" về hành vi**, không chỉ về khái niệm — nghi ngờ thì dùng composition.

I — Interface Segregation (tách interface)

Vi phạm: một interface "béo" ép lớp cài đặt phải có cả phương thức nó không cần:

```
interface IMayXau { void In(string s); void Quet(string s); void Fax(string s); }  
class MayInDon : IMayXau { /* phải viết Quet/Fax rỗng hoặc ném NotSupportedException */
```

Tái cấu trúc: chia theo **vai trò**; lớp nào cần bao nhiêu vai trò thì cài bấy nhiêu:

```
interface IMayIn { void In(string s); }  
interface IMayQuet { void Quet(string s); }  
  
class MayInDon : IMayIn { ... }  
class MayDaNangLuc : IMayIn, IMayQuet { ... }
```

Bên dùng cũng chỉ phụ thuộc vào phần nó cần (`void Xuat(IMayIn may)`), nên đổi `IMayQuet` không ảnh hưởng nó. Ví dụ .NET: `IReadOnlyList<T>`, `IEnumerable<T>`, `IDisposable` — mỗi cái một khả năng nhỏ. Đây cũng là nền để kiểm thử: mock interface nhỏ dễ hơn nhiều.

D — Dependency Inversion (đảo ngược phụ thuộc)

Vi phạm: lớp "cấp cao" (ng nghiệp vụ) tự `new` lớp "cấp thấp" (email, CSDL) → dính chặt vào chi tiết, không test được, không thay được:

```
class DatHangXau
{
    private readonly GuiEmailSmtplib _email = new GuiEmailSmtplib(); // phụ thuộc lớp cụ t
    public void DatHang(string sp) { ...; _email.Gui(...); } // test là gửi email
}
```

Tái cấu trúc: cả hai phía cùng phụ thuộc vào **abstraction (interface)**; đối tượng cụ thể được **tiêm vào** từ ngoài:

```
interface IGuiEmail { void Gui(string den, string noiDung); }

class DatHangService(IGuiEmail email) // Dependency Injection qua constructor
{
    public void DatHang(string sp) { ...; email.Gui("khach@mail.com", $"Da dat {sp}"); }
}

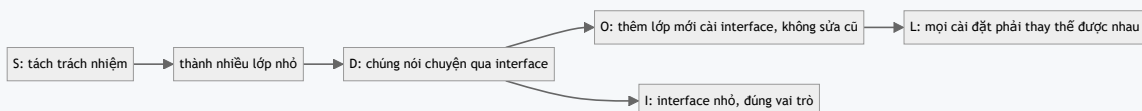
var dichVu = new DatHangService(new GuiEmailGia()); // test: dùng bản giả
```

"Đảo ngược" nghĩa là: thay vì nghiệp vụ *phụ thuộc vào* email, cả hai phụ thuộc vào interface do **ng nghiệp vụ định nghĩa**. Nơi lắp ráp đối tượng gọi là **composition root** (thường ở `Program.cs`). Trong [ASP.NET Core](#), **DI container** làm việc lắp ráp đó (Tập 2):

```
builder.Services.AddScoped<IGuiEmail, GuiEmailSmtplib>();
```

Phân biệt: **DIP** (nguyên lý thiết kế) ≠ **DI** (kỹ thuật tiêm phụ thuộc) ≠ **IoC container** (công cụ tự động hoá DI).

SOLID gắn kết với nhau



Kết quả cuối: code **lỏng lẻo (loosely coupled)**, mỗi mảnh có thể thay/kiểm thử độc lập — nền tảng cho kiến trúc phân lớp/Clean Architecture ở Tập 3.

Áp dụng vừa đủ

- SOLID **giảm chi phí thay đổi trong tương lai**, nhưng chi phí hiện tại là nhiều lớp/interface hơn. Nếu code nhỏ, đơn giản, ít đổi, **đừng ép**.
- Quy tắc ba lần (Rule of Three): lập lần thứ ba mới tái cấu trúc/tách trừu tượng.
- **Đo bằng câu hỏi**: Sửa yêu cầu X thì phải đụng bao nhiêu chỗ? Có test được không mà không cần CSDL/mạng thật? Người mới có hiểu được không?
- Tránh interface "một-một" vô nghĩa (`IUserService` chỉ có một cài đặt, không bao giờ đổi, không cần mock). Nhưng ở ranh giới I/O (CSDL, mạng, đồng hồ, file) thì hầu như luôn đáng.
- Tái cấu trúc **cần test làm lưới an toàn** (Chương 38): có test xanh trước, sửa, test xanh sau.

Các nguyên tắc bổ sung để nhớ

- **DRY** (Don't Repeat Yourself): đừng lặp tri thức — nhưng đừng gộp code chỉ vì trông giống nhau tình cờ.
- **KISS**: giữ đơn giản. **YAGNI**: đừng viết thứ chưa cần.
- **Composition over inheritance, Law of Demeter** (a.B.C.D dài là mùi xấu), **Tell, Don't Ask** (bảo đối tượng làm, đừng hỏi dữ liệu rồi tự quyết).

Lỗi thường gặp

- Hiểu SRP là "mỗi lớp một phương thức" → phân mảnh vô nghĩa.
- Interface cho mọi lớp "phòng khi", không có lý do thay thế.
- `new` các phụ thuộc bên trong lớp nghiệp vụ → không test được.
- Kế thừa chỉ để tái sử dụng code, sinh lớp con phá hợp đồng lớp cha.
- Interface quá to, cài đặt rỗng/ném `NotImplementedException`.
- Dùng Service Locator (`ServiceProvider.GetService<T>()` gọi rải rác) thay vì tiêm qua constructor.
- Constructor nhận 8–10 phụ thuộc → lớp đang làm quá nhiều (vi phạm SRP).

Bài tập

1. Tách một lớp `QuanLyDonHang` làm cả tính tiền, lưu file JSON và gửi email thành các lớp theo SRP + DIP.
2. Viết lại hàm `TinhPhiVanChuyen(string loai, double kg)` dùng `switch` thành Strategy để đóng với việc thêm loại mới.
3. Cho `interface IDongVat { void Bay(); void Boi(); void Chay(); }` — chỉ ra vấn đề và tách theo ISP (Chim, Ca, Cho).
4. Tìm ví dụ vi phạm Liskov trong `Stack<T>` / `ReadOnlyCollection` hoặc trong chính code bạn (một lớp con ném `NotSupportedException`).

5. Viết unit test cho `DatHangService` bằng `IGuiEmail` giả (Chương 38) và chứng minh không cần gửi email thật.

Chương 43 — Dự án tổng hợp Tập 1: Quản lý kho hàng

Mục tiêu học

Sau chương này, bạn sẽ:

- Ghép toàn bộ kiến thức Tập 1 thành **một ứng dụng hoàn chỉnh**: OOP, record, collection, LINQ, async, JSON, exception, interface/DI, unit test.
- Biết **quy trình** đi từ yêu cầu đến thiết kế, cài đặt và kiểm thử.
- Tổ chức solution nhiều project theo hướng **tách nghiệp vụ khỏi giao diện và lưu trữ**.
- Có một dự án nền để mở rộng và mang lên web ở Tập 2.

Code đầy đủ: [code/ch43-du-an-tong-hop/](#) .

Bài toán

Cửa hàng nhỏ cần một chương trình console **quản lý kho hàng**:

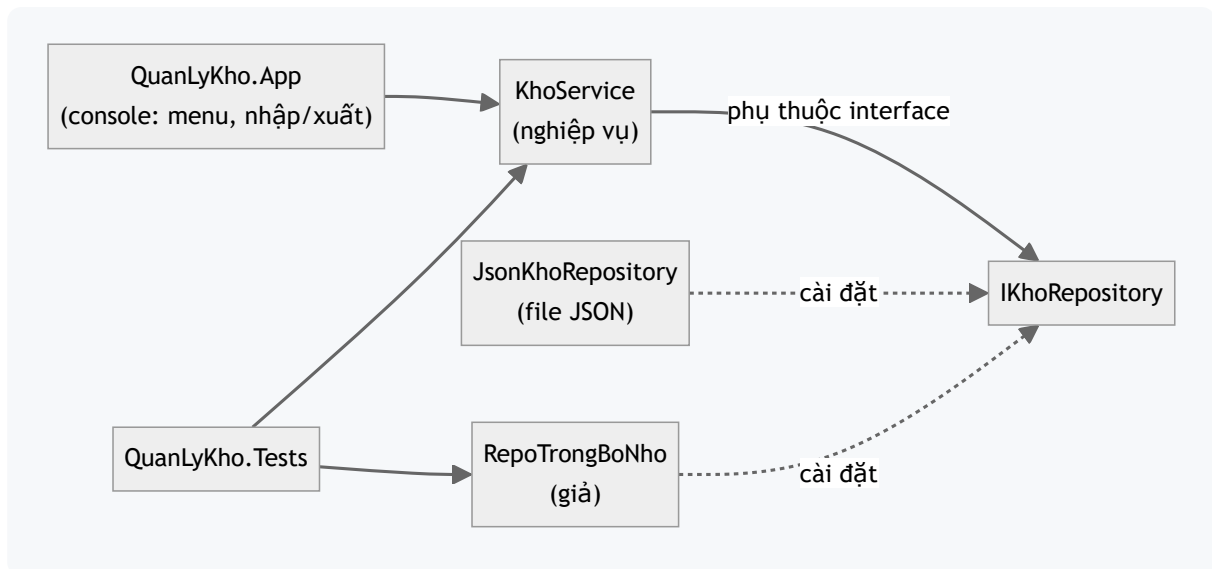
#	Yêu cầu
1	Quản lý sản phẩm: mã (duy nhất), tên, nhóm hàng, đơn giá, tồn kho, mức cảnh báo
2	Nhập kho (tăng tồn) và xuất kho (giảm tồn; không được xuất quá số đang có)
3	Ghi lịch sử giao dịch (thời điểm, loại, số lượng, ghi chú)
4	Tìm kiếm theo tên/mã, liệt kê sản phẩm sắp hết hàng
5	Báo cáo theo nhóm: số sản phẩm, tổng tồn, tổng giá trị
6	Lưu bền dữ liệu vào file JSON, mở lại vẫn còn
7	Nhập liệu thân thiện: sai dữ liệu thì báo lỗi rõ ràng, không sập chương trình

Bước 1 — Phân tích và thiết kế

Áp dụng cách nghĩ của Chương 12: tìm **danh từ** → class; **động từ** → phương thức.

- Danh từ: **sản phẩm, giao dịch, kho, nhóm hàng**.
- Động từ: thêm, nhập, xuất, tìm, báo cáo, lưu.
- Quy tắc nghiệp vụ (đây là "bất biến" cần bảo vệ): mã không trùng, số lượng nhập/xuất > 0, **tồn không âm**.

Kiến trúc phân lớp



Ba project trong một solution (Chương 4, 15):

Project	Vai trò	Phụ thuộc
QuanLyKho.Core	mô hình + nghiệp vụ + interface lưu trữ + cài đặt JSON	không
QuanLyKho.App	giao diện console, composition root	Core
QuanLyKho.Tests	unit test xUnit	Core

Nguyên tắc dẫn đường: Core **không biết** console hay file JSON tồn tại — chỉ biết interface `IKhoRepository` (Dependency Inversion, Chương 42). Nhờ vậy: đổi JSON sang SQL/Web chỉ viết thêm một lớp; test thay bằng repository giả trong bộ nhớ; và nghiệp vụ dùng lại nguyên vẹn cho ứng dụng web ở Tập 2.

Bước 2 — Mô hình dữ liệu (record)

```

public record SanPham(string Ma, string Ten, string Nhom, decimal DonGia, int TonKho,
{
    public decimal GiaTriTon => DonGia * TonKho;    // thuộc tính tính toán
}

public enum LoaiGiaoDich { Nhap, Xuat }
public record GiaoDich(DateTime ThoiGian, string MaSanPham, LoaiGiaoDich Loai, int Sol

```

Vì sao **record bất biến**: (Chương 14, 20) đối tượng không bị sửa tùy tiện từ bên ngoài; "thay đổi" là tạo bản mới bằng `with : sp with { TonKho = sp.TonKho + soLuong }`. Dùng `decimal` cho tiền (Chương 5), `enum` cho loại giao dịch (Chương 19), tham số mặc định cho `MucCanhBao`.

Lỗi nghiệp vụ dùng exception riêng (Chương 21):

```
public class KhoException(string message) : Exception(message);
public class KhôngTimThayException(string ma) : KhoException($"Không tìm thấy sản phẩm {ma}");
public class KhôngDuHangException(string ma, int can, int con) : KhoException($"Không đủ hàng {ma} để bán {can} sản phẩm {con}");
```

Lớp gốc `KhoException` cho phép giao diện `catch (KhoException)` một lần để hiển thị mọi lỗi nghiệp vụ — còn lỗi hệ thống (file hỏng, hết bộ nhớ) không bị "nuốt" nhầm.

Bước 3 — Nghiệp vụ: `KhoService`

```
public class KhoService(IKhoRepository repo, TimeProvider dongHo) // nhận phụ thuộc
{
    private DuLieuKho _du = new();

    public async Task XuatKhoAsync(string ma, int soLuong, string? ghiChu = null, CancellationToken ct)
    {
        if (soLuong <= 0) throw new KhoException("Số lượng xuất phải lớn hơn 0");
        var sp = LayHoacNem(ma);
        if (sp.TonKho < soLuong) throw new KhôngDuHangException(sp.Ma, soLuong, sp.TonKho);

        CapNhat(sp with { TonKho = sp.TonKho - soLuong });
        GhiGiaoDich(sp.Ma, LoaiGiaoDich.Xuat, soLuong, ghiChu);
        await repo.LuuAsync(_du, ct); // lưu SAU KHI thay đổi hợp lệ
    }
    ...
}
```

Những kiến thức được dùng trong lớp này:

- **Validate trước, thay đổi sau** (guard clauses, Chương 21): dữ liệu sai không bao giờ lọt vào trạng thái.
- **Async** (Chương 33): lưu file là I/O nên `async Task`, nhận `CancellationToken`.
- **TimeProvider** (đồng hồ trừu tượng): test có thể "đóng băng" thời gian thay vì phụ thuộc `DateTime.Now`.
- **LINQ** (Chương 30) cho truy vấn:

```
public IEnumerable<SanPham> SapHet()
=> _du.SanPham.Where(s => s.TonKho <= s.MucCanhBao).OrderBy(s => s.TonKho);

public IEnumerable<NhomBaoCao> BaoCaoTheoNhom()
=> _du.SanPham
    .GroupBy(s => s.Nhom)
    .Select(g => new NhomBaoCao(g.Key, g.Count(), g.Sum(s => s.TonKho), g.Sum(s => s.GiaTri)
    .OrderByDescending(x => x.GiaTri);
```

- **So sánh chuỗi** bằng `StringComparison.OrdinalIgnoreCase` (Chương 11): mã `a1` và `A1` là một.
- Trả `ReadOnlyList<SanPham> TatCa` thay vì `List` để bên ngoài không phá dữ liệu (Chương 14).

Bước 4 — Lưu trữ: interface và cài đặt JSON

```
public interface IKhoRepository
{
    Task<DuLieuKho> TaiAsync(CancellationToken ct = default);
    Task LuuAsync(DuLieuKho duLieu, CancellationToken ct = default);
}
```

`JsonKhoRepository` cài đặt bằng `System.Text.Json` (Chương 36) và **ghi qua file tạm rồi đổi tên** (Chương 35) để không hỏng dữ liệu nếu chương trình sập giữa lúc ghi:

```
string tam = duongDan + ".tmp";
await using (var fs = File.Create(tam))
    await JsonSerializer.SerializeAsync(fs, duLieu, Options, ct);
File.Move(tam, duongDan, overwrite: true);
```

`JsonSerializerOptions` được tạo **một lần** (`static readonly`) và dùng `JsonStringEnumConverter` để `Nhap/Xuat` hiện thành chữ trong file.

Bước 5 — Ứng dụng console và composition root

`Program.cs` là **nơi duy nhất** biết lớp cụ thể nào được dùng — nơi lắp ráp (composition root):

```
var dịchVuKho = new KhoService(new JsonKhoRepository(duongDan), TimeProvider.System);
await dịchVuKho.KhoiTaoAsync();
```

- **Chế độ `--demo`**: chạy kịch bản có sẵn với file tạm rồi xóa — để xem chương trình hoạt động ngay, cũng là "bản chạy thử" của CI.
- **Chế độ menu**: vòng `while (true)`, `switch` trên lựa chọn (Chương 7, 8), đọc số an toàn bằng `TryParse` trong vòng lặp cho đến khi hợp lệ (Chương 5).
- **Xử lý lỗi ở biên giới** (Chương 21): `catch (KhoException e)` ngay trong vòng menu để in thông báo thân thiện và tiếp tục, thay vì sập.

Chạy thử:

```
dotnet run --project QuanLyKho.App -- --demo
```

Dữ liệu tương tác lưu vào `ApplicationData/QuanLyKho/kho.json` ; đổi bằng `--file duong-dan.json` .

Bước 6 — Kiểm thử

`KhoService` phụ thuộc interface, nên test thay lưu trữ thật bằng một repository **trong bộ nhớ** (Chương 38):

```
class RepoTrongBoNho : IKhoRepository
{
    public DuLieuKho DuLieu { get; private set; } = new();
    public int SoLanLuu { get; private set; }
    public Task<DuLieuKho> TaiAsync(CancellationToken ct = default) => Task.FromResult
    public Task LuuAsync(DuLieuKho duLieu, CancellationToken ct = default) { DuLieu =
}

[Fact]
public async Task XuatKho_KhongDuHang_NemLoiVaTonKhongDoi()
{
    await _kho.ThemSanPhamAsync(SanPhamMau("A1", ton: 3));

    var loi = await Assert.ThrowsAsync<KhongDuHangException>(() => _kho.XuatKhoAsync("

    Assert.Equal(3, _kho.Tim("A1")!.TonKho);        // tồn KHÔNG bị thay đổi khi lỗi
}
```

Bộ test phủ: thêm hợp lệ / trùng mã, nhập, xuất đủ/thiếu hàng, số lượng không dương (`[Theory]`), sản phẩm không tồn tại, sắp hết, báo cáo nhóm, tìm kiếm. Chạy `dotnet test` — mọi thay đổi sau này đều có lưới an toàn.

Đối chiếu kiến thức Tập 1

Chương	Ở đâu trong dự án
5–11 Nền tảng	biến, decimal, chuỗi, vòng lặp, switch, phương thức trong Program.cs
12–15 OOP cơ bản	class, property, namespace/project (Core/App/Tests)
16–20 OOP nâng cao	interface IKhoRepository, kế thừa exception, record, enum
21 Exception	KhoException và hai lớp con, guard clause
22–23 Generics, null	Task<T>, IEnumerable<T>, string?, SanPham?
24–30 Collection, LINQ	List, GroupBy, Where, OrderBy, Sum
31–34 Delegate, async	lambda trong LINQ, async/await, CancellationToken

Chương	Ở đâu trong dự án
35–37 I/O, JSON	JsonKhoRepository, file tạm, options
38–40 Test, cấu hình	xUnit, repo giả, tham số dòng lệnh/đường dẫn
41–42 Pattern, SOLID	Repository, DI qua constructor, SRP/DIP

Mở rộng (bài tập lớn)

Chọn tối thiểu ba việc:

1. **Cấu hình**: đọc đường dẫn file, ngưỡng cảnh báo mặc định từ `appsettings.json` (Chương 40).
2. **Logging**: thay `Console.WriteLine` trong nghiệp vụ bằng `ILogger` (Chương 39).
3. **Xoá/sửa sản phẩm**, đổi tên nhóm; có test cho từng tính năng.
4. **Xuất báo cáo CSV** (Chương 36) cho báo cáo nhóm và lịch sử.
5. **Nhiều kho** (`MaKho`) — thiết kế lại mô hình và migrate file JSON cũ.
6. **Strategy** tính giá bán theo nhóm (Chương 41): giá vốn + lợi nhuận theo quy tắc khác nhau.
7. **Repository SQLite**: viết `SqliteKhoRepository` cài đặt `IKhoRepository` (chuẩn bị cho EF Core ở Tập 2) — không sửa `KhoService` !
8. **Giao dịch song song**: hai người cùng xuất một mặt hàng — dùng `SemaphoreSlim` /khóa để bảo đảm tồn không âm (Chương 34).

Hướng tới Tập 2

Cùng lớp `QuanLyKho.Core` này sẽ được **dùng lại nguyên vẹn** trong một dự án **ASP.NET Core Web API**: `KhoService` được đăng ký vào DI container, `IKhoRepository` được cài đặt bằng **Entity Framework Core**, và menu console được thay bằng các endpoint HTTP. Đó là phần thưởng của thiết kế tách lớp: nghiệp vụ không phụ thuộc giao diện.

Chúc mừng bạn đã hoàn thành Tập 1! Bạn đã có nền tảng vững chắc của C# và .NET — sẵn sàng bước vào web với **ASP.NET Core**.

Phụ lục A — Bảng lỗi thường gặp và cách xử lý

Tra cứu nhanh theo **triệu chứng** (thông báo lỗi). Có hai nhóm: **lỗi biên dịch** (mã CSxxxx, trình biên dịch báo ngay) và **ngoại lệ lúc chạy** (chương trình đã chạy rồi mới ném lỗi). Cột cuối chỉ chương liên quan.

A.1 Đọc thông báo lỗi thế nào

1. Đọc **dòng đầu**: mã lỗi + mô tả. Mã lỗi (CS0103 ...) tra được trên learn.microsoft.com.
2. Xem **vị trí**: Program.cs(12,5) = dòng 12, cột 5. Lỗi thật đôi khi nằm **ngay trước** dòng đó (thiếu ; , thiếu }).
3. Sửa lỗi **đầu tiên** rồi biên dịch lại — các lỗi sau thường là hệ quả dây chuyền.
4. Với ngoại lệ lúc chạy: đọc **StackTrace**, tìm dòng đầu tiên thuộc code của bạn.

A.2 Lỗi biên dịch thường gặp

Thông báo	Nguyên nhân thường gặp	Cách xử lý	Chương
CS1002: ; expected	thiếu dấu ; cuối lệnh	thêm ; (kiểm tra cả dòng trước)	3
CS1513: } expected	thiếu/dư ngoặc nhọn	căn thật lề, để IDE định dạng lại (Ctrl+K,D)	3
CS0103: The name 'x' does not exist in the current context	sai tên/hoa thường, biến ngoài phạm vi	kiểm tra chính tả; khai báo biến ở đúng scope	5, 10
CS0246: type or namespace 'X' could not be found	thiếu using/tham chiếu project/gói NuGet	thêm using; dotnet add reference/package	4, 15
CS0266: Cannot implicitly convert 'double' to 'int'	ép kiểu mất dữ liệu	ép tường minh (int) hoặc Math.Round	5
CS0165: Use of unassigned local variable	dùng biến chưa gán	gán giá trị khởi tạo	5
CS0136: A local variable named 'x' cannot be declared in this scope	trùng tên biến với scope ngoài	đổi tên	10
CS0161: not all code paths return a value	có nhánh không return	thêm return/throw cho mọi nhánh	10

Thông báo	Nguyên nhân thường gặp	Cách xử lý	Chương
CS7036: There is no argument that corresponds to the required parameter	gọi constructor/phương thức thiếu đối số	truyền đủ đối số hoặc thêm constructor	13, 16
CS1061: 'X' does not contain a definition for 'Y'	gọi thành viên không có (sai tên, sai kiểu, thiếu using cho extension method)	kiểm tra kiểu và tên; using System.Linq;	17, 19
CS0122: 'X' is inaccessible due to its protection level	truy cập thành viên private/internal	đổi access modifier hoặc dùng property	14
CS0120: An object reference is required for the non-static field	gọi thành viên thường từ static	tạo đối tượng, hoặc đổi thành static	13
CS0534: does not implement inherited abstract member	chưa override đủ thành viên abstract	cài đặt đủ	18
CS0535: does not implement interface member	chưa cài đủ interface	cài đủ (nhớ public)	18
CS0144: Cannot create an instance of the abstract type or interface	new một abstract class/interface	new lớp cụ thể	18
CS0506: cannot override because it is not marked virtual	lớp cha thiếu virtual	thêm virtual ở lớp cha	16
CS0108: hides inherited member	vô tình che thành viên cha	dùng override hoặc new có chủ đích	16
CS0104: 'X' is an ambiguous reference	hai namespace cùng có tên X	dùng tên đầy đủ/alias	15
CS8600/CS8602/CS8604: possible null ...	cảnh báo nullable	kiểm tra null, dùng ?./??, hoặc đổi kiểu T?	23
CS8618: Non-nullable property must contain a non-null value	property string chưa khởi tạo	khởi tạo, required, hoặc ?	14, 23
CS1503: cannot convert from 'X' to 'Y' (đối số)	truyền sai kiểu	ép/chuyển đổi đúng kiểu	10

Thông báo	Nguyên nhân thường gặp	Cách xử lý	Chương
CS0029: Cannot implicitly convert type	gán sai kiểu	chuyển đổi tường minh	5
CS4014: the call is not awaited (cảnh báo)	gọi async mà quên await	thêm await (hoặc xử lý có chủ đích)	33
CS4033: 'await' operator can only be used within an async method	thiếu async ở phương thức	thêm async Task	33
CS0019: Operator cannot be applied to operands	toán tử với kiểu không phù hợp (vd T > T)	dùng IComparable<T>, ép kiểu	6, 22
CS0029/CS0037: Cannot convert null to non-nullable value type	gán null cho int...	dùng int?	23
CS8509: switch expression does not handle all possible values	thiếu nhánh _	thêm nhánh mặc định	7
CS8120: The switch case is unreachable / already handled	nhánh cụ thể đặt sau nhánh tổng quát	đổi thứ tự (cụ thể trước)	7, 17
CS0017: more than one entry point	nhiều Main/top-level statements	chỉ giữ một	3
CS9035: Required member must be set	quên gán property required	gán trong object initializer	14, 32

A.3 Ngoại lệ lúc chạy thường gặp

Ngoại lệ	Nguyên nhân	Cách xử lý / phòng tránh	Chương
NullReferenceException	dùng đối tượng đang null	bật nullable; ?. , ??; kiểm tra is null; xem StackTrace	13, 23
IndexOutOfRangeException	chỉ số ngoài mảng	duyệt < Length; kiểm tra biên	9

Ngoại lệ	Nguyên nhân	Cách xử lý / phòng tránh	Chương
ArgumentOutOfRangeException	chỉ số/giá trị ngoài phạm vi (List, Substring)	kiểm tra Count, độ dài trước	11, 25
KeyNotFoundException	dict[key] khi khoá chưa có	TryGetValue, ContainsKey	26
InvalidCastException	ép kiểu sai	dùng is/as	17
FormatException	int.Parse("abc")	TryParse	5
OverflowException	tràn số trong checked / chuyển kiểu	dùng kiểu lớn hơn, kiểm tra	5
DivideByZeroException	chia số nguyên cho 0	kiểm tra mẫu số	6
InvalidOperationException: Collection was modified	thêm/xoá khi đang foreach	duyệt ngược, RemoveAll, duyệt bản sao ToList()	25, 26
InvalidOperationException: Sequence contains no elements	First()/Max() trên dãy rỗng	FirstOrDefault, Any()	30
InvalidOperationException: Nullable object must have a value	.Value trên int? đang null	kiểm tra HasValue/??	23
ArgumentNullException	truyền null vào tham số không cho phép	kiểm tra đầu vào; ThrowIfNull	21
ArgumentException	đối số không hợp lệ / khoá trùng trong Dictionary.Add	kiểm tra trước, dùng TryAdd/dict[k]=v	21, 26
StackOverflowException	đệ quy không điều kiện dừng / quá sâu	thêm điều kiện dừng, chuyển sang vòng lặp/Stack	10, 29
OutOfMemoryException	nạp quá nhiều dữ liệu vào bộ nhớ	xử lý theo luồng/lười (ReadLines, yield)	27, 35
FileNotFoundException / DirectoryNotFoundException	đường dẫn sai, cwd khác dự kiến	kiểm tra đường dẫn tuyệt đối, Path.Combine	35
UnauthorizedAccessException	thiếu quyền, hoặc đường dẫn là thư mục	ghi vào thư mục được phép	35

Ngoại lệ	Nguyên nhân	Cách xử lý / phòng tránh	Chương
IOException (file in use)	file đang bị chương trình khác/ chính bạn khoá	đóng bằng using; FileShare	35
JsonException	JSON sai cú pháp/sai kiểu	kiểm tra đầu vào; bắt và báo lỗi	36
OperationCanceledException	tác vụ bị huỷ (CancellationToken)	bắt riêng; thường là hợp lệ	33
TaskCanceledException/TimeoutException	hết hạn chờ	thử lại/báo người dùng	33
AggregateException	lỗi gộp từ nhiều Task / .Result	await (lỗi được "mở" ra); xem InnerExceptions	33
ObjectDisposedException	dùng đối tượng đã Dispose	giữ phạm vi using đủ rộng	21
NotImplementedException	phần chưa viết	cài đặt	18
NotSupportedException	thao tác không hỗ trợ (vd sửa collection chỉ đọc)	dùng cấu trúc phù hợp	18
SwitchExpressionException	switch expression không khớp nhánh nào	thêm nhánh _	7
TypeInitializationException	lỗi trong hàm khởi tạo tĩnh (static)	xem InnerException	13

A.4 Lỗi logic — "không báo lỗi nhưng kết quả sai"

Triệu chứng	Nguyên nhân	Chương
1 / 2 ra 0	chia nguyên	6
0.1 + 0.2 != 0.3	sai số số thực	5
int.MaxValue + 1 ra số âm	tràn số không báo lỗi	5
s.Trim(); mà s không đổi	string bất biến, quên gán kết quả	11
Sửa b mà a cũng đổi	class/mảng là kiểu tham chiếu	9, 13
Hai đối tượng "giống nhau" nhưng == ra false	so sánh tham chiếu	13, 20

Triệu chứng	Nguyên nhân	Chương
HashSet.Contains không tìm thấy phần tử vốn có	thiếu Equals/GetHashCode	20, 26
Truy vấn LINQ cho kết quả khác dự đoán / chạy lại nhiều lần	thực thi trì hoãn	27, 30
dem++ từ nhiều thread ra kết quả nhỏ hơn	race condition	34
Lệnh sau async chạy trước khi xong	quên await	33
Số/ngày in ra dấu sai trên máy khác	khác culture	11, 36
Tiếng Việt vỡ dấu khi đọc/ghi file	sai mã hoá	35
Cấu hình đọc ra rỗng/mặc định	sai khoá, sai môi trường	40
Test lúc đạt lúc hỏng	phụ thuộc thứ tự/thời gian/trạng thái chung	38

A.5 Quy trình khi gặp lỗi mà bạn chưa hiểu

1. Copy **nguyên văn** thông báo lỗi (kèm mã) và tìm kiếm — thường đã có người gặp.
2. Thu nhỏ vấn đề: tạo project console mới, tái hiện lỗi bằng vài dòng.
3. Dùng debugger (Chương 39) thay vì đoán.
4. Hỏi (Stack Overflow, cộng đồng): nêu **phiên bản .NET**, đoạn code tối thiểu, thông báo lỗi đầy đủ, điều đã thử.

Phụ lục B — Tài liệu tham khảo và nguồn học thêm

Danh sách dưới đây ưu tiên **nguồn chính thức, miễn phí và được duy trì thường xuyên**. Địa chỉ web có thể thay đổi; nếu link không mở được, hãy tìm theo tên.

B.1 Tài liệu chính thức

- **Microsoft Learn — C#**: <https://learn.microsoft.com/dotnet/csharp/> — hướng dẫn, tham chiếu ngôn ngữ, "What's new in C#".
- **Microsoft Learn — .NET**: <https://learn.microsoft.com/dotnet/> — kiến trúc, CLI, thư viện lớp cơ sở.
- **.NET API Browser**: <https://learn.microsoft.com/dotnet/api/> — tra từng lớp/phương thức (ví dụ `List<T>`, `File`).
- **Tải .NET SDK**: <https://dotnet.microsoft.com/download> — kèm lịch hỗ trợ (LTS/STS).
- **C# language specification** và đề xuất tính năng: <https://github.com/dotnet/csharplang>.
- **Mã nguồn .NET Runtime**: <https://github.com/dotnet/runtime> — đọc cách `List<T>`, `Dictionary` được cài đặt.
- **NuGet Gallery**: <https://www.nuget.org> — tìm thư viện.
- **ASP.NET Core** (cho Tập 2): <https://learn.microsoft.com/aspnet/core/>.
- **Entity Framework Core** (Tập 2): <https://learn.microsoft.com/ef/core/>.

B.2 Sách nên đọc thêm

Sách	Tác giả	Đọc khi
<i>C# in Depth</i> (bản mới nhất)	Jon Skeet	muốn hiểu sâu ngôn ngữ, các phiên bản mới
<i>Pro C# and .NET</i>	Andrew Troelsen, Philip Japikse	tham khảo toàn diện
<i>Clean Code</i>	Robert C. Martin	viết code dễ đọc, dễ bảo trì
<i>Refactoring</i>	Martin Fowler	tái cấu trúc code (kèm Chương 42)
<i>Design Patterns</i> (GoF)	Gamma, Helm, Johnson, Vlissides	gốc của các pattern (Chương 41)
<i>Head First Design Patterns</i>	Freeman & Robson	tiếp cận design pattern dễ hiểu
<i>The Pragmatic Programmer</i>	Hunt, Thomas	tư duy và thói quen nghề nghiệp

Sách	Tác giả	Đọc khi
<i>Concurrency in C# Cookbook</i>	Stephen Cleary	async/đa luồng thực dụng (Chương 33–34)
<i>Unit Testing Principles, Practices, and Patterns</i>	Vladimir Khorikov	test tốt (Chương 38)
<i>Dependency Injection Principles, Practices, and Patterns</i>	Steven van Deursen, Mark Seemann	DI đúng cách (Chương 42)
<i>Algorithms</i>	Sedgewick & Wayne	cấu trúc dữ liệu và thuật toán (Chương 24–29)
<i>Introduction to Algorithms</i> (CLRS)	Cormen và cộng sự	tham khảo thuật toán nâng cao

B.3 Blog, video, cộng đồng

- **.NET Blog:** <https://devblogs.microsoft.com/dotnet/> — thông báo tính năng, bài viết hiệu năng (Stephen Toub).
- **Nick Chapsas, Milan Jovanović, Tim Corey** (YouTube) — video thực hành C#/.NET.
- **Stephen Cleary's blog** — async chuyên sâu; **Andrew Lock** — [ASP.NET](#) Core; **Jon Skeet** — ngôn ngữ C#.
- **Stack Overflow** (thẻ `c#`, `.net`) — hỏi đáp; **Reddit r/dotnet**, **r/csharp**; **Discord C# / .NET**.
- **.NET Conf** (hàng năm, video miễn phí) và **.NET Community Standup**.

B.4 Công cụ

Nhu cầu	Công cụ
IDE	Visual Studio, VS Code + C# Dev Kit, JetBrains Rider
Thử code nhanh	LINQPad , <code>dotnet-script</code> , SharpLab.io (xem code sau khi biên dịch)
Kiểm thử	xUnit, NUnit, MSTest; NSubstitute, Moq; FluentAssertions
Độ phủ / chất lượng	Coverlet, ReportGenerator; SonarQube; <code>dotnet format</code> ; Roslyn analyzers
Hiệu năng	BenchmarkDotNet , <code>dotnet-counters</code> , <code>dotnet-trace</code> , PerfView
Xem/giải mã assembly	ILSpy, dnSpy (mã nguồn mở)
Logging	Serilog, NLog; Seq (xem log)
Quản lý mã nguồn	Git, GitHub Desktop, GitKraken

B.5 Luyện tập

- **Exercism** (C# track): bài tập có mentor. **LeetCode**, **HackerRank**, **Codewars**, **Advent of Code**: thuật toán và tư duy.
- **Microsoft Learn Modules**: lộ trình có chấm bài. **Roslyn / dotnet/samples**: mẫu code chính thức.
- Tự làm dự án: mở rộng dự án Chương 43; đóng góp cho dự án mã nguồn mở .NET (tìm issue `good first issue`).

B.6 Lộ trình sau Tập 1

1. **Tập 2 của bộ sách này**: [ASP.NET](#) Core, EF Core, MVC/Razor Pages/Blazor.
2. **Tập 3**: kiến trúc sạch, DDD, CQRS, bảo mật, hiệu năng, Docker, CI/CD, triển khai cloud.
3. Học nền tảng bổ trợ: **SQL**, **HTTP/REST**, **Git**, **Docker**, kiến thức cơ bản về **cloud** (Azure).
4. Chuyên sâu tùy hướng: API/backend, dịch vụ nền, **Blazor** (frontend C#), game (**Unity**), ứng dụng đa nền tảng (**.NET MAUI**), AI (thư viện .NET cho ML/LLM).

Phụ lục C — Cheat sheet C#

Tra cứu nhanh cú pháp và lệnh thường dùng. Nhớ đây là **tóm tắt**; giải thích nằm trong các chương tương ứng (ghi trong ngoặc).

C.1 Lệnh dotnet (Ch. 2–4, 40)

```
dotnet --version          dotnet --info          dotnet new list
dotnet new console -n Ten dotnet new classlib -n Ten dotnet new xunit -n Ten
dotnet new sln -n Ten     dotnet sln add Proj     dotnet add A reference
dotnet add package X [--version v] dotnet remove package X dotnet list package
dotnet run [--project P] [-- args] dotnet build            dotnet test            dotnet publish
dotnet user-secrets set "K" "V"     dotnet format          dotnet new gitignore
```

C.2 Kiểu dữ liệu (Ch. 5)

Kiểu	Ví dụ	Ghi chú
int, long, byte	25, 8_000L, 200	số nguyên 32/64/8 bit
double, float, decimal	1.5, 1.5f, 19.99m	decimal cho tiền
bool, char, string	true, 'A', "chao"	
var	var x = 5;	suy kiểu lúc biên dịch
T?	int? x = null;; string? s	nullable value / reference type
const, readonly	const double Pi = 3.14;	hằng lúc biên dịch / lúc chạy

Ép kiểu: (int)9.9 (cắt) · Convert.ToInt32(x) · int.Parse(s) · int.TryParse(s, out int n)
· x as T · x is T t.

C.3 Toán tử (Ch. 6)

```
+ - * / %    ++ --    += -= *= /=    == != < > <= >=    && || !    & | ^ ~ << >>
c ? a : b    x ?? d    x ??= d    x?.Y    x?[i]    x!    a..b    ^1    is not null
```

Độ ưu tiên (cao → thấp): ! ++ -- → * / % → + - → so sánh → == != → && → || → ??
→ ?: → gán. Khi nghi ngờ: dùng ngoặc.

C.4 Điều khiển (Ch. 7–8)

```

if (a) { } else if (b) { } else { }
switch (x) { case 1: ...; break; default: ...; break; }
var s = x switch { 1 => "a", >= 2 and <= 5 => "b", _ => "c" };

for (int i = 0; i < n; i++) { }      while (dk) { }      do { } while (dk);
foreach (var x in ds) { }          break; continue; return;

```

Pattern: `is int n`, `is { Length: > 3 }`, `is [1, .., var last]`, `not null`, `x and > 0`, `x or 1`.

C.5 Chuỗi (Ch. 11)

```

$"Xin chao {ten}, {gia:N2}, {ngay:dd/MM/yyyy}, [{s,10}], [{s,-10}]"
s.Length s.Trim() s.ToUpper() s.Contains("x") s.IndexOf("x") s.Replace("a","b")
s.Split(',') string.Join(", ", ds) string.IsNullOrEmpty(s) s.StartsWith("x")
string.Equals(a, b, StringComparison.OrdinalIgnoreCase)
"" "raw multi-line" "" @"C:\verbatim" new StringBuilder().Append(x).ToString()

```

C.6 Mảng và collection (Ch. 9, 24–27)

```

int[] a = [1, 2, 3]; int[,] m = new int[2,3]; int[][] j = new int[3][]; a[^1]
Array.Sort(a) Array.Reverse(a) Array.IndexOf(a, x) Array.BinarySearch(a, x)

var l = new List<T> { } l.Add l.Insert(i,x) l.Remove(x) l.RemoveAt(i) l.RemoveA
var d = new Dictionary<K,V> d[k] = v d.TryGetValue(k, out var v) d.ContainsKey(k)
var h = new HashSet<T> h.Add(x) h.Contains(x) h.UnionWith(o) h.IntersectWi
new Queue<T>() .Enqueue .Dequeue .Peek new Stack<T>() .Push .Pop .Peek

```

Cần	Dùng	Truy cập/tìm
Danh sách	List<T>	[i] O(1) · tìm O(n)
Tra cứu theo khoá	Dictionary<K,V>	O(1)
Không trùng	HashSet<T>	O(1)
Có thứ tự khoá	SortedDictionary/SortedSet	O(log n)

C.7 LINQ (Ch. 30)

```

.Where(x => ...) .Select(x => ...) .SelectMany(...) .OrderBy(k) .OrderByDescending
.GroupBy(k) .Join(o, a => a.K, b => b.K, (a, b) => ...) .Distinct() .Skip(n).Take(r
.First() .FirstOrDefault() .Single() .Last() .ElementAt(i) .Any() .All(p) .Cont

```

Nhớ: thực thi trì hoãn — chạy khi duyệt/ `ToList()` .

[illegible]

C.11 Generics, delegate, event (Ch. 22, 31)

```
class Kho<T> where T : class, IComparable<T>, new() { }      T Max<T>(IEnumerable<T> d)
Func<int,int> f = x => x * 2;   Action<string> a = Console.WriteLine;   Predicate<int>
public event EventHandler<MyArgs>? DaXong;   DaXong?.Invoke(this, new MyArgs());   obj
```

C.12 Async và đa luồng (Ch. 33–34)

```
async Task<int> LayAsync(CancellationToken ct = default) { await Task.Delay(100, ct);
var kq = await LayAsync();                                await Task.WhenAll(t1, t2);      await Task.
await foreach (var x in nguonAsync) { }      await Task.Run(() => TinhNang());
lock (khoa) { }      Interlocked.Increment(ref n);      Parallel.For(0, n, i => { });      r
new ConcurrentDictionary<K,V>()      Channel.CreateBounded<int>(10)
```

Quy tắc: async xuyên suốt; không `.Result` / `.Wait()`; tránh `async void`; luôn truyền `CancellationToken`.

C.13 I/O và JSON (Ch. 35–36)

```
File.ReadAllText(p) File.WriteAllText(p, s) File.ReadLines(p) File.AppendAllText(p,
await File.ReadAllTextAsync(p)      Directory.EnumerateFiles(d, "*.cs", SearchOption.A
Path.Combine(a, b) Path.GetFileName(p) Path.GetExtension(p) Path.GetTempPath()
using var w = new StreamWriter(p); w.WriteLine("x");      using var r = new StreamRe

string json = JsonSerializer.Serialize(obj, opts);      var o = JsonSerializer.Deser
[JsonPropertyName("ten")] [JsonIgnore]      new JsonSerializerOptions { WriteIndented
```

C.14 Test xUnit (Ch. 38)

```
[Fact] public void Ten_TinhHuong_KetQua() { /* Arrange - Act - Assert */ }
[Theory][InlineData(1, 2, 3)] public void Cong(int a, int b, int mongDoi) { Assert.Equ
Assert.Equal(mongDoi, thucTe) Assert.True(x) Assert.Null(x) Assert.Empty(ds) Asser
var m = Substitute.For<IX>(); m.Ham().Returns(5); m.Received(1).Ham();
```

C.15 Quy ước đặt tên (Ch. 15)

Thành phần	Quy ước
Class, record, struct, enum, namespace, method, property, event	PascalCase

Thành phần	Quy ước
Interface	IPascalCase
Tham số, biến cục bộ	camelCase
Field private	_camelCase
Hằng	PascalCase
Phương thức async	hậu tố Async
Kiểu generic	T, TKey, TValue

C.16 Phím tắt IDE hay dùng

Việc	Visual Studio	VS Code
Chạy debug / không debug	F5 / Ctrl+F5	F5 / Ctrl+F5
Breakpoint	F9	F9
Step Over / Into / Out	F10 / F11 / Shift+F11	F10 / F11 / Shift+F11
Gợi ý nhanh / sửa lỗi	Ctrl+.	Ctrl+.
Định dạng code	Ctrl+K, Ctrl+D	Shift+Alt+F
Đổi tên (rename)	F2	F2
Đến định nghĩa	F12	F12
Tìm mọi tham chiếu	Shift+F12	Shift+F12
Chạy test	Ctrl+R, T	qua Test Explorer

